

# **Biobuz lessons**

*Release*

**Catholic Central Spires Robotics**

Oct 03, 2026



|          |                                                                |          |
|----------|----------------------------------------------------------------|----------|
| <b>1</b> | <b>How to use it</b>                                           | <b>3</b> |
| <b>2</b> | <b>The tasks</b>                                               | <b>5</b> |
| 2.1      | Are the motors turning the right way? . . . . .                | 5        |
| 2.2      | L2a: read the sticks. . . . .                                  | 7        |
| 2.3      | L2: the sticks drive the wheels. . . . .                       | 13       |
| 2.4      | L3: make the sticks feel right . . . . .                       | 17       |
| 2.5      | L4: one stick to drive, one to turn . . . . .                  | 19       |
| 2.6      | L5: slide it sideways . . . . .                                | 20       |
| 2.7      | Watch the numbers while it drives . . . . .                    | 23       |
| 2.8      | Check your work without a robot. . . . .                       | 25       |
| 2.9      | L6: let the path follower hold the drivetrain . . . . .        | 29       |
| 2.10     | Mount the Pinpoint and plug it in . . . . .                    | 32       |
| 2.11     | L8: work out where the robot is from the drive wheels. . . . . | 35       |
| 2.12     | L9: the first autonomous . . . . .                             | 38       |
| 2.13     | L10: two legs, forward then sideways . . . . .                 | 39       |
| 2.14     | L11: make the sticks mean the field. . . . .                   | 41       |
| 2.15     | L12: give the driver both . . . . .                            | 43       |
| 2.16     | L13: hold the heading the driver left . . . . .                | 45       |
| 2.17     | L14: give the driver a button that parks the robot. . . . .    | 46       |
| 2.18     | L15: one OpMode a driver could take to a match . . . . .       | 48       |
| 2.19     | L16: ask the wheels for a speed . . . . .                      | 50       |
| 2.20     | L17: measure the two numbers L16 trusted . . . . .             | 53       |
| 2.21     | Pedro routes: a cheat sheet. . . . .                           | 55       |
| 2.22     | Glossary . . . . .                                             | 63       |
| 2.23     | Cheat sheet . . . . .                                          | 65       |
| 2.24     | Where this came from . . . . .                                 | 67       |
| 2.25     | Answers . . . . .                                              | 68       |



|                                                                                                                                                                                                                                                                                                 |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1.1: The robot from the front, on the floor, with both gamepads beside it, so a student can match what is in front of them to what the guide calls each part. . . . .                                                                                                                    | 3  |
| Figure 2.1: One wheel from the side, with an arrow on the top of the tyre showing which way it travels when that wheel is driving the robot forward. . . . .                                                                                                                                    | 5  |
| Figure 2.2: The Project window with <code>org.firstinspires.ftc.teamcode</code> open, showing the lessons folder and the mytry folder with only <code>package-info.java</code> in it. . . . .                                                                                                   | 8  |
| Figure 2.3: The Copy Class box with New name <code>L2aSticksOpMode</code> and Destination package <code>org.firstinspires.ftc.teamcode.mytry</code> . . . . .                                                                                                                                   | 9  |
| Figure 2.4: The list under <code>gamepad1</code> , after typing <code>left_st</code> , with <code>left_stick_y</code> highlighted. . .                                                                                                                                                          | 10 |
| Figure 2.5: AdvantageScope's Line Graph with <code>NT:sim/stick/leftY</code> drawn at -1 and <code>NT:/stick/leftY</code> drawn at 1. . . . .                                                                                                                                                   | 11 |
| Figure 2.6: AdvantageScope's Line Graph of the flight log with <code>/stick/leftY</code> in Left Axis, drawn at 1. . . . .                                                                                                                                                                      | 12 |
| Figure 2.7: Gamepad 1 from above, with the left stick, the right stick and the A button labelled, and an arrow showing which way each y axis counts up. . . . .                                                                                                                                 | 13 |
| Figure 2.8: The robot from above with its nose marked, and each of the four wheels labelled front left, front right, back left and back right. . . . .                                                                                                                                          | 15 |
| Figure 2.9: Three graphs side by side, each with the stick from -1 to 1 along the bottom and the power out from -1 to 1 up the side: the raw stick as a straight line, the deadbanded stick as a line with a flat step at the middle, and the squared stick as an S through the middle. . . . . | 17 |
| Figure 2.10: The robot from above with its nose marked, and the top roller of each of the four wheels drawn as a line at 45 degrees, so the four lines make an X across the robot. . . . .                                                                                                      | 21 |
| Figure 2.11: The Panels Graph panel with <code>stick/left_raw</code> and <code>stick/left_shaped</code> both plotted while the stick is pushed slowly to full, so the gap between the two curves is the thing a student is looking for. . . . .                                                 | 24 |
| Figure 2.12: AdvantageScope with a 2D field showing <code>sim/Pose</code> , the topic list open beside it, and a graph of the four <code>sim/wheels</code> values, so a student can match their own screen to what this step describes. . . . .                                                 | 28 |
| Figure 2.13: Two arrows reaching the same four wheels, one from the path follower and one from the driver's sticks, with the commanded-wheels switch between them showing which one gets through. . . . .                                                                                       | 30 |
| Figure 2.14: The Pinpoint on the robot from above, with the forward pod and the strafe pod drawn at right angles to each other, each labelled with the socket it plugs into, and the sticker side marked facing up. . . . .                                                                     | 33 |
| Figure 2.15: One loop of dead reckoning: the four wheel travels since the last loop, the forward and left arrows those add up to, the same pair turned to point along the robot's heading, and the old estimate with that pair added on the end. . . . .                                        | 35 |
| Figure 2.16: The field from above with the driver at the bottom, the robot turned a quarter turn, the stick's direction as an arrow on the field, and the same arrow drawn again on the robot showing what forward and sideways it turns into. . . . .                                          | 42 |
| Figure 2.17: The loop drawn once round: the stick asking for 40 in/s, the feedforward guess at the power, the encoder's measured speed coming back, and the error being added in. . . . .                                                                                                       | 51 |



This guide walks you through making the robot do things, one task at a time. Each task page says what you will have when it is done, what has to be true before you start, and the steps to get there. Every step tells you what you will see when it worked and what to try when it did not.



---

## How to use it

---

- **Start with the task you need, not with page one.** Every task lists what it needs finished first. If you missed a session, find the task you are up to and read its *Before you start*.
- **Do the step, then read the check.** The check tells you whether to go on. If what you see is not what the check describes, the *if it didn't* line under it is the first thing to try.
- **The answers are in the back.** They are the real code from the solutions branch. Use them when you are stuck, not instead of being stuck.

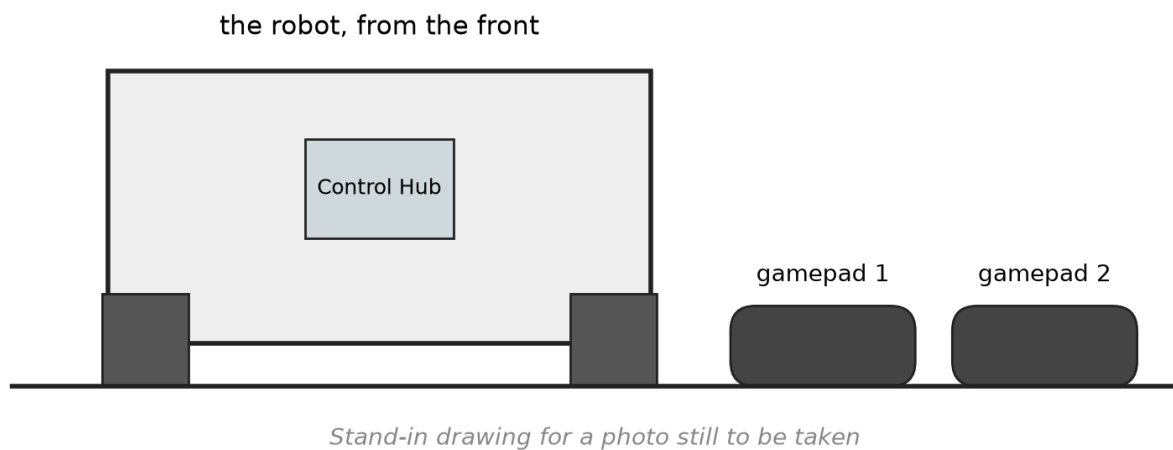


Figure 1.1. The robot from the front, on the floor, with both gamepads beside it, so a student can match what is in front of them to what the guide calls each part.



---

## The tasks

---

### 2.1 Are the motors turning the right way?

#### 2.1.1 What you will have when this is done

Each of the four wheels turns the way the code thinks it does, so a stick pushed forward drives that side forward. You will also know how to spot a wheel that is wired backwards, and where the one line that fixes it lives.

#### 2.1.2 Before you start

- L2 is finished, so L2b Tank drives.
- The robot has about 2 m of clear floor for the driving step.
- For the first step, something to rest the robot on so all four wheels are off the ground. A couple of blocks under the frame is enough.

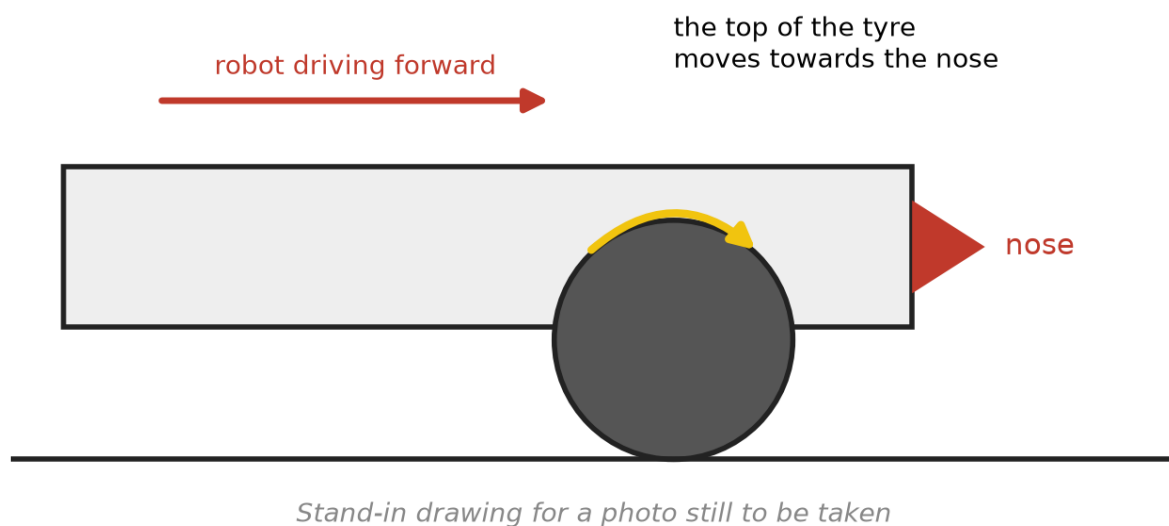


Figure 2.1. One wheel from the side, with an arrow on the top of the tyre showing which way it travels when that wheel is driving the robot forward.

### 2.1.3 The steps

#### Step 1: watch the wheels with the robot held up

Rest the robot so no wheel touches the ground. Start `L2b Tank`. Push the left stick a little way forward, and watch both left wheels. Then let go, push the right stick, and watch both right wheels.

A wheel driving the robot forward has the **top of the tyre travelling toward the nose**. That is the whole test, and you can see it at any speed, so keep the stick gentle.

**You'll know it worked when** all four wheels pass it: the two left ones for the left stick and the two right ones for the right stick.

**If it didn't**, write down which wheels had the top of the tyre going the other way. That is a wheel wired backwards, and step 3 is where it gets fixed. A wheel that does not move at all is a different problem: check that it is plugged in, and that the name in the Robot Configuration matches the name in the code.

#### Step 2: name the wheel by driving one stick at a time

Put the robot on the floor. With the robot held up you can see a backwards wheel directly, so this step is for when you cannot get under the robot, or want to be sure.

Push **only the left stick** forward. The left wheels drive and the right wheels sit still, so the robot should curve forward and to its own right.

Then push **only the right stick** forward. It should curve forward and to its own left.

A backwards wheel changes that in a way that names it:

| What you push    | What the robot does                 | Which wheel is backwards |
|------------------|-------------------------------------|--------------------------|
| Left stick only  | Curves forward, to its right        | Neither left wheel       |
| Left stick only  | Slides to its left, no turn at all  | Front left               |
| Left stick only  | Slides to its right, no turn at all | Back left                |
| Right stick only | Curves forward, to its left         | Neither right wheel      |
| Right stick only | Slides to its right, no turn at all | Front right              |
| Right stick only | Slides to its left, no turn at all  | Back right               |

The slide is the giveaway. One stick on its own should always turn the robot, and a backwards wheel on that side cancels the turn exactly and leaves a sideways push instead.

With both sticks forward a backwards wheel is harder to read. The robot gets half the forward push it should, plus a slide and a spin, so it corkscrews. That tells you something is backwards but not which, which is why one stick at a time is the test.

**You'll know it worked when** both sticks agree with the first two rows of the table.

**If it didn't**, the row that matches names the wheel. Go to step 3.

---

#### Where this table comes from

The simulator, not a floor. Each row was produced by turning one wheel's power around and running `L2b Tank` for 1 s, and the numbers are the wheel powers going in and the pose coming out. No robot has been driven with a wheel backwards, and a real one also scrubs its wheels sideways, so expect the real thing to be messier than the table. The direction of the slide is what to read.

---

#### Step 3: fix it in one line

The four directions live inside `drivetrainConfig`, in

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/pedro/Constants.java
```

```
c.frontLeftDirection.set(DcMotorSimple.Direction.FORWARD);
c.frontRightDirection.set(DcMotorSimple.Direction.REVERSE);
c.backLeftDirection.set(DcMotorSimple.Direction.FORWARD);
c.backRightDirection.set(DcMotorSimple.Direction.REVERSE);
```

Change the wheel you named from FORWARD to REVERSE, or from REVERSE to FORWARD. Send the code to the robot again.

**You'll know it worked when** step 1 and step 2 both pass.

**If it didn't**, and the same wheel is still backwards, the code on the robot is not the code you just changed. Send it again and watch for the message that says it landed.

#### Step 4: let AutoTune settle it

Steps 1 to 3 get the robot driving today. The procedure that settles all four at once is the Mecanum Tuner, and running it is `task.autotune`.

What it does, read from `MecanumTuner.java` rather than from a run: it asks for the four motor names as the Robot Configuration spells them, then spins one motor at a time at half power beside a diagram of which wheel it is and which way forward is, and asks you whether that one spun forward or reversed. At the end it prints a `drivetrainConfig` with all four answers in it, for you to paste into `Constants.java`.

It also catches the mistake steps 1 to 3 cannot tell apart from a backwards wheel. Answer that the motor did not spin, and it stops with this:

```
The wrong motor spun. Check that your motors are plugged into the correct ports,
and that they are configured correctly. Then, try again.
```

Swap two motor names and each of those motors is handed the other one's direction, so both of those wheels turn backwards. Steps 1 to 3 cannot see the difference. This check can.

### 2.1.4 What you just did

Which way a motor turns is not the code's choice. It is set by how the motor is bolted on and wired up. The two sides of the robot face opposite ways. So their motors turn opposite ways for the same power. `frontLeftDirection` and its three neighbours tell the code what each motor already does. That is why the left pair says FORWARD and the right pair says REVERSE. Above that line, nothing has to think about it again. Every lesson after L2 asks for forward and gets forward.

### 2.1.5 Where next

- `task.l3` stops the robot creeping when the sticks are centred.
- `task.autotune` is the full procedure, and it covers more than directions.

`task.autotune` is not written yet.

Getting the robot and the laptop ready has no page yet. Checking the motor directions needs L2 finished, so read it after that one.

## 2.2 L2a: read the sticks

### 2.2.1 What you will have when this is done

You have your own copy of the sticks OpMode, in the `mytry` folder. It runs in the simulator on the laptop. AdvantageScope draws the stick while you push it. The flight log keeps the numbers your code wrote down.

## 2.2.2 Before you start

- Android Studio has the project open. Getting it there has no page yet, so ask a mentor.
- AdvantageScope is installed on the same laptop.
- A gamepad is plugged into the laptop.
- No robot is needed. Nothing in this task turns a wheel.

You write in one folder, `mytry`, inside `TeamCode/java/org.firstinspires.ftc.teamcode`. The `lessons` folder beside it holds the starting files you copy from.

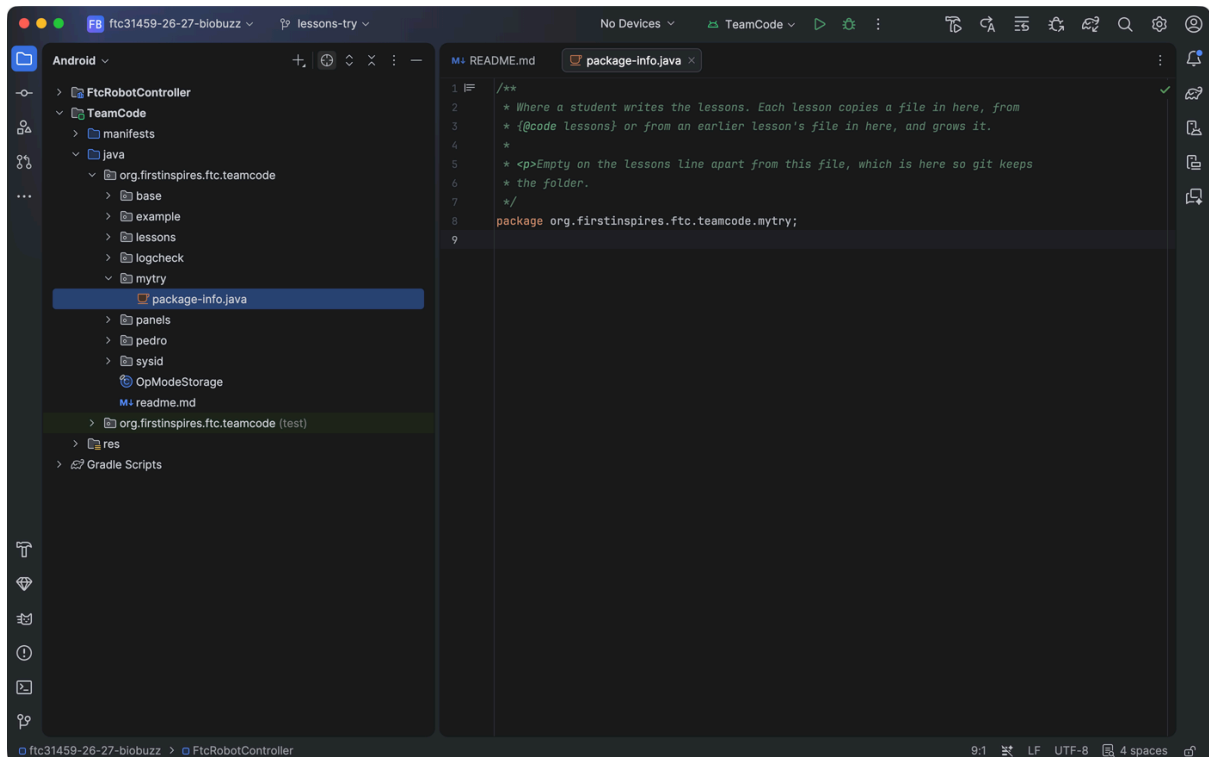


Figure 2.2. The Project window with `org.firstinspires.ftc.teamcode` open, showing the `lessons` folder and the `mytry` folder with only `package-info.java` in it.

## 2.2.3 The steps

### Step 1: copy the starter into mytry

Open `lessons`, click `L2aSticksOpMode` once, and copy it: `⌘C` on a Mac, `Ctrl+C` on Windows. Fold `lessons` up again, click `mytry` once, and paste.

A box called **Copy Class** opens. Look at **Destination package**. It should end in `mytry`. Then click **OK**. If Android Studio asks whether to add the file to Git, click **Add**.

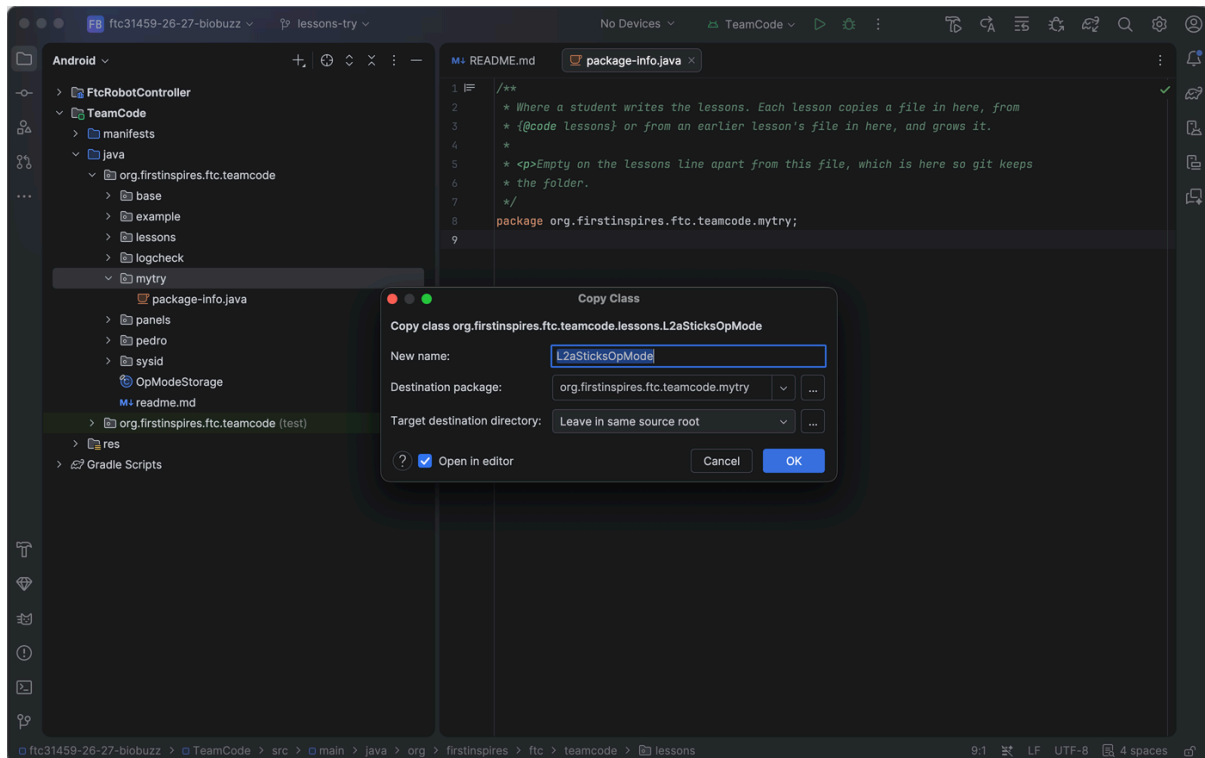


Figure 2.3. The Copy Class box with New name L2aSticksOpMode and Destination package org.firstinspires.ftc.teamcode.mytry.

**You'll know it worked when** mytry holds L2aSticksOpMode, the file is open, and its first line reads `package org.firstinspires.ftc.teamcode.mytry;`. Android Studio changed that line for you.

**If it didn't**, and Destination package ends in lessons, click **Cancel**. Click mytry once more, wait a moment, and paste again.

### Step 2: switch your copy on

Find the line `@Disabled`, just under `@TeleOp(name = "L2a Sticks", group = "Lessons")`. Click on it and delete the whole line.

`@Disabled` hides an OpMode from the Driver Station's list. The starter in lessons keeps it, so the robot only ever lists your copy.

**You'll know it worked when** the line under `@TeleOp` starts `public class L2aSticksOpMode`.

### Step 3: read the sticks

Scroll down to `loop()` and find `TODO 1`. Click at the end of its second line and press Enter. Type this much:

```
double leftSpeed = -gamepad1.
```

A list opens with everything a gamepad has. Type `left_st` and the list shrinks to the left stick. Move the highlight to `left_stick_y` with the arrow keys, then press Tab. Type `;` to end the line.

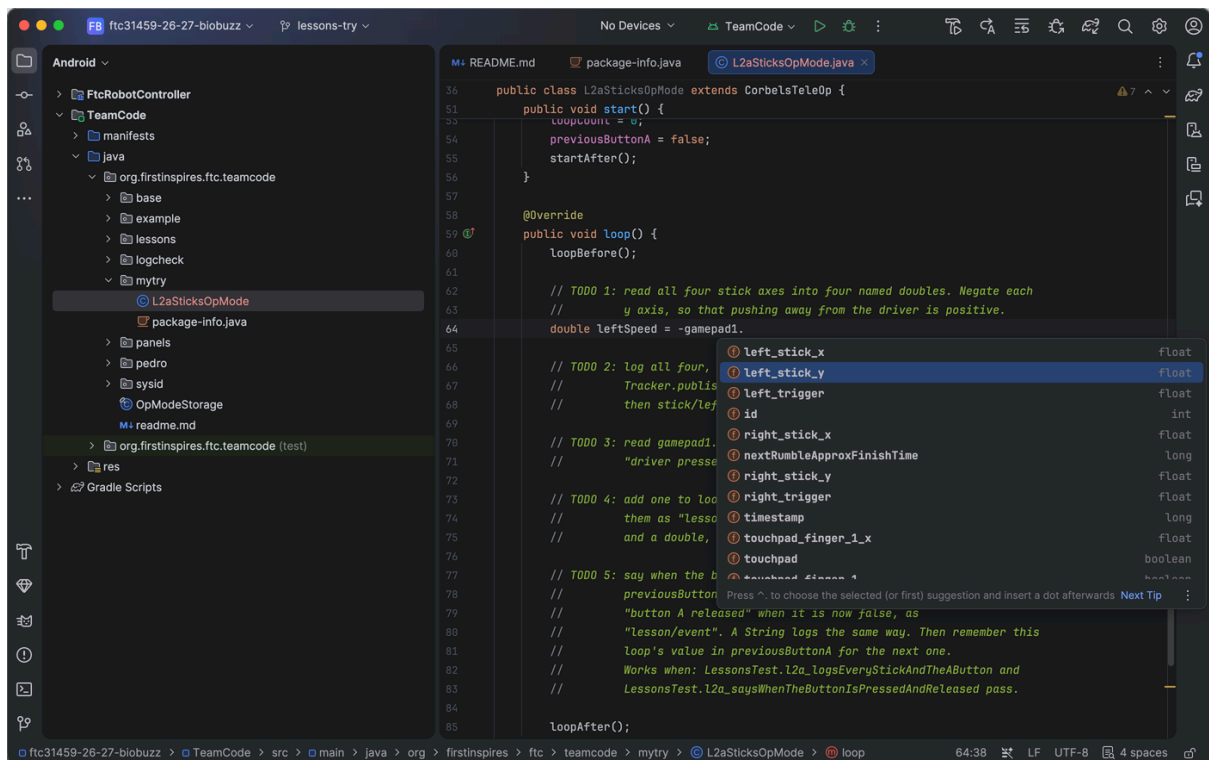


Figure 2.4. The list under gamepad1. after typing left\_st, with left\_stick\_y highlighted.

Now hold the pointer still over `left_stick_y`. A box opens and says what it is: *left analog stick vertical axis*. You can do this with any name you are not sure about.

A stick reads a number from -1 to 1. Pushed away from you it reads negative, which is backwards from the way anybody thinks about driving. The minus sign in front fixes that. After it, forward is positive, and it stays that way for every lesson.

Write the other three the same way: `rightSpeed` from `right_stick_y`, with a minus sign, and `leftSideways` and `rightSideways` from `left_stick_x` and `right_stick_x`, without one.

**You'll know it worked when** all four lines are there and none of them has a red underline.

**If it didn't**, and a line reads `left_stick_x` where you meant `left_stick_y`, the highlight was on the wrong name when you pressed Tab. The list puts the name you picked last at the top, so it is not always in the same place. Change the letter by hand.

#### Step 4: write the numbers down

Under TODO 2, type `Track`. The list shows more than one `Tracker`. Pick the one from `org.firstinspires.ftc.teamcode.base` and press Tab. Android Studio adds the import line for it at the top of the file.

Then finish the line the way TODO 2 shows: `Tracker.publish("stick/leftY", leftSpeed);`. Write one line for each of the other three sticks, with the names TODO 2 gives.

`Tracker.publish` takes a name and a value, and sends both to Panels and to the flight log. Copy each name exactly as the TODO spells it.

Fill TODO 3 and TODO 4 the same way. Four kinds of value go out here: a double for a stick, a boolean for the A button, an int for the loop count, and the seconds as another double. `publish` takes all four.

**You'll know it worked when** `Tracker` is not red anywhere, and there is an import `org.firstinspires.ftc.teamcode.base.Tracker;` line near the top.

**Step 5: say when the A button changes**

Fill TODO 5.

The A button is either down or up, every loop. What you want is the loop it *changed* on, and that takes two values: this loop's and last loop's. `previousButtonA` holds last loop's, and the field is already there. The value you publish here is a `String`, which is text rather than a number.

**Step 6: build it**

Open the **Build** menu and pick the **Compile** item whose name ends in `TeamCode.main`. Then click the hammer on the left edge of the window to open the **Build** window.

**You'll know it worked when** the Build window has a green tick and says `BUILD SUCCESSFUL`.

**Step 7: run it in the simulator**

Click the **Terminal** button near the bottom of the left edge. Type this and press Enter:

```
./gradlew :TeamCode:simRun --args="mytry.L2aSticksOpMode --pad"
```

**You'll know it worked when** it prints `Flight log:` and a folder, then `mytry.L2aSticksOpMode` running., and stays running. The folder is the project's top folder, the one that holds `README.md`.

**Step 8: watch the stick live**

In AdvantageScope, open **File > Connect to Simulator > NetworkTables 4**. Click **Line Graph**. The list on the left has two `stick` folders. The one inside `sim` is the gamepad. The one on its own is what your code published. Open both, and drag each `leftY` down into **Left Axis**.

Push the left stick away from you.

**You'll know it worked when** both lines move as you push. With the stick all the way forward, `sim/stick/leftY` reads -1, the way the gamepad gives it, and your `stick/leftY` reads 1, after your minus sign.

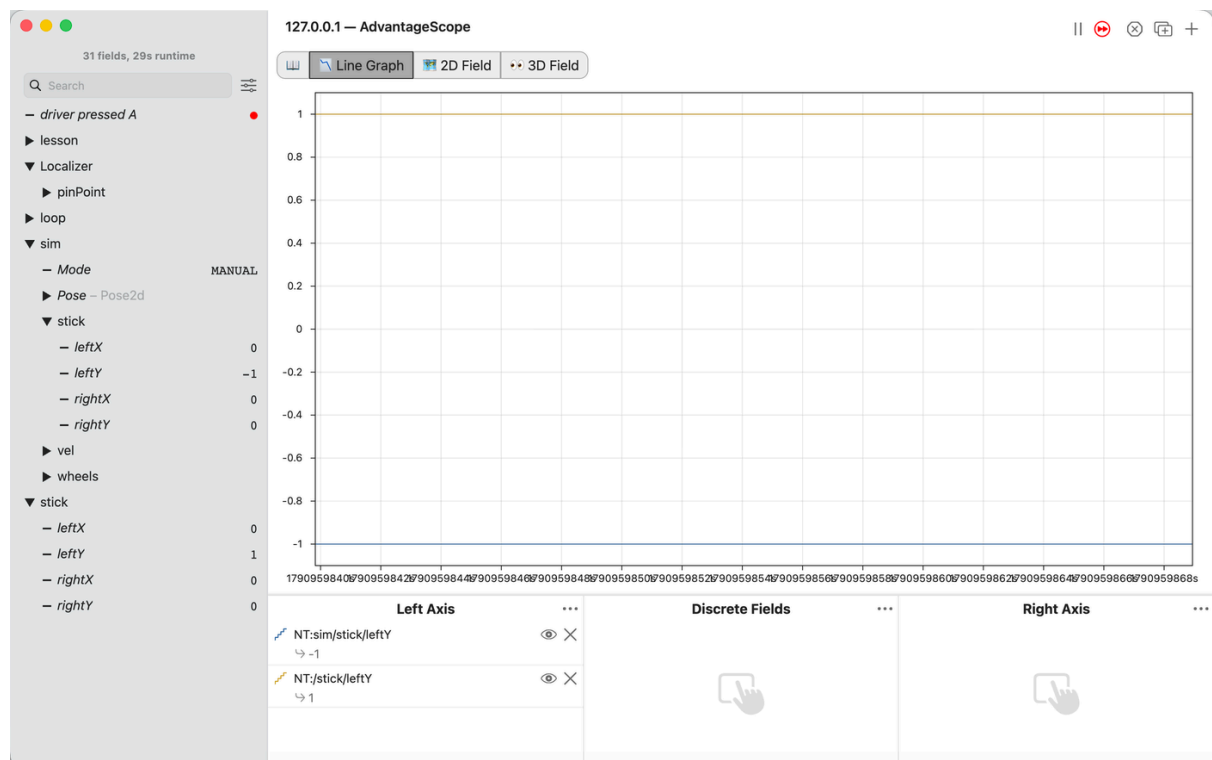


Figure 2.5. AdvantageScope's Line Graph with `NT:sim/stick/leftY` drawn at -1 and `NT:/stick/leftY`

drawn at 1.

### Step 9: read the flight log

Click in the Terminal and press Ctrl+C to stop the simulator. In AdvantageScope, open **File > Open Log(s)...**, go to the project's top folder, and open the newest file whose name starts L2aSticksOpMode. Open `stick`, and drag `leftY` into **Left Axis**.

**You'll know it worked when** the line reads 1 while the stick was forward. It is the same `stick/leftY` you watched live, kept in a file you can open again later.

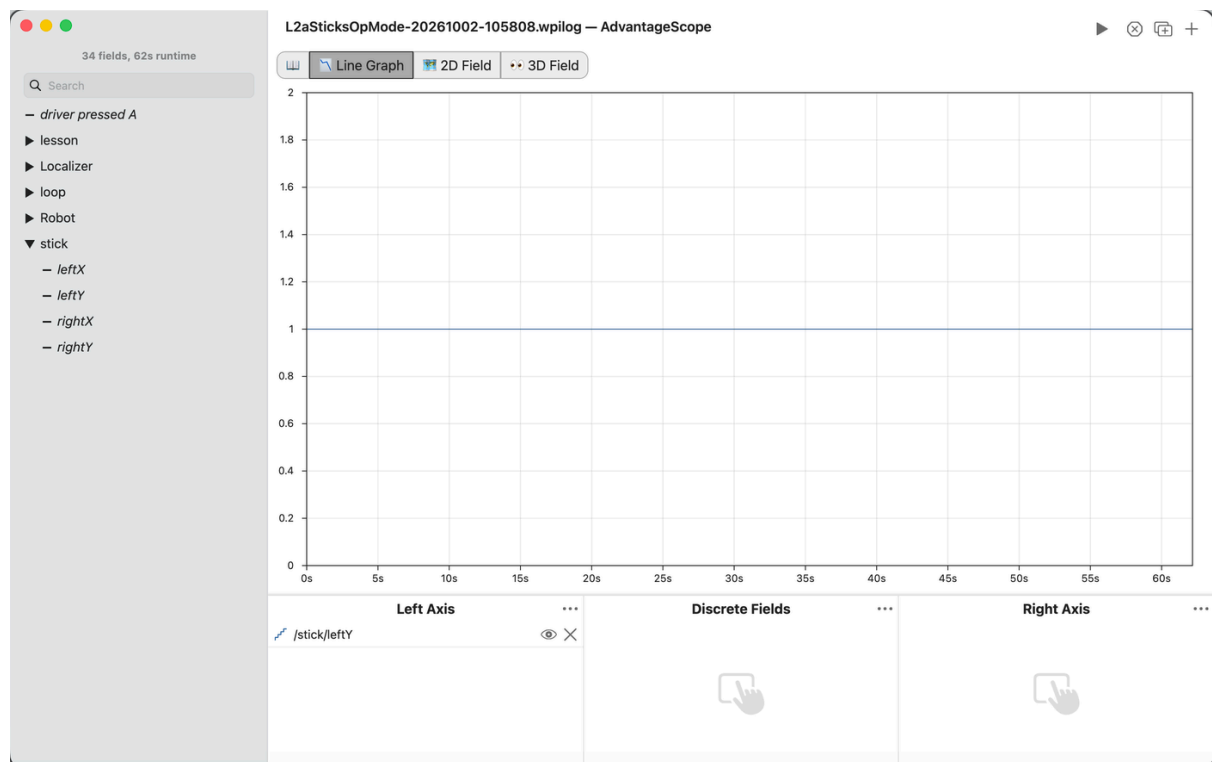


Figure 2.6. AdvantageScope's Line Graph of the flight log with `/stick/leftY` in Left Axis, drawn at 1.

### Step 10: run it on the robot

#### To do

Sending your copy to the robot and running L2a Sticks from the Driver Station is not written yet.

## 2.2.4 What you just did

`loop` runs over and over, from PLAY until STOP, and each pass is one look at the gamepad. Your code read the four sticks and the A button, turned forward into a positive number, and wrote each value down under a name. AdvantageScope showed the stick the way the gamepad reports it beside the value your code wrote, so the same push reads -1 in one and 1 in the other, and the minus sign is the difference. The flight log kept your values, to look at again after the run.

## 2.2.5 Where next

- [task.12](#), from its step 4, makes the sticks drive the wheels.

## 2.3 L2: the sticks drive the wheels

### 2.3.1 What you will have when this is done

The robot drives from the two sticks. The left stick runs the left wheels and the right stick runs the right wheels. Push both forward and it goes straight. Push one each way and it spins.

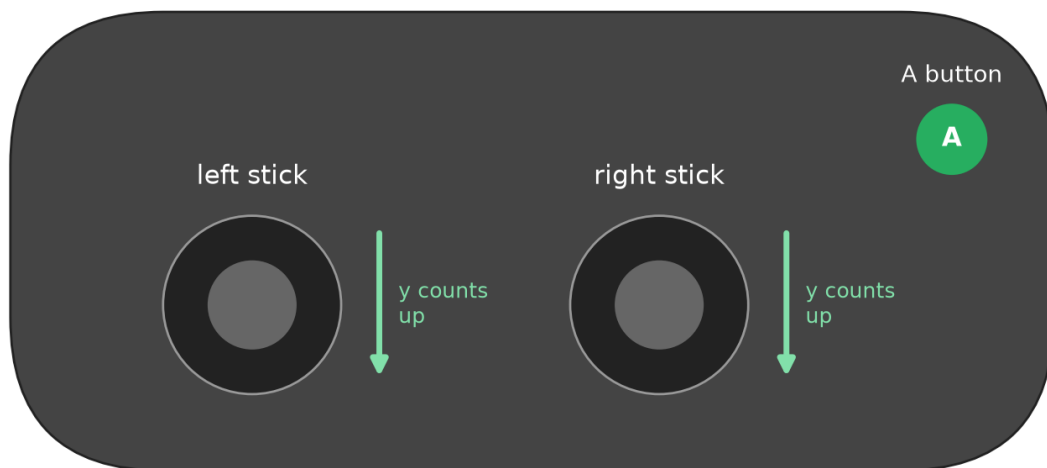
You also write the numbers down. The four stick axes, the A button and the loop count all go somewhere you can watch them.

### 2.3.2 Before you start

- You can build the code and send it to the robot. That task is `task.laptop`.
- You have started an OpMode from the Driver Station once, so you know where INIT, PLAY and STOP are. That task is `task.firstrun`.
- The battery is charged and in, and gamepad 1 is paired.
- Step 6 drives the robot, so leave it about 2 m of clear floor. No step before that one turns a wheel, and the robot can sit where it is.

Everything you write is in one folder: `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/lessons/`.

a stick pushed away from the driver reads -1, pulled back +1



the driver holds it from this side

*Stand-in drawing for a photo still to be taken*

Figure 2.7. Gamepad 1 from above, with the left stick, the right stick and the A button labelled, and an arrow showing which way each y axis counts up.

## 2.3.3 The steps

### Step 1: read the sticks and write the numbers down

Open `L2aSticksOpMode.java`. Fill TODO 1 to TODO 4. Nothing here moves a wheel.

A stick reads a number from -1 to 1. Pushed away from you it reads negative, which is backwards from the way anybody thinks about driving. So put a minus sign in front of each y axis as you read it. After that, forward is positive, and it stays that way for every lesson.

`Tracker.publish` takes a name and a value, and sends both to Panels and to the flight log. Copy each name out of the TODO exactly: the test looks for the name it was given.

Four kinds of value go out here. A `double` for a stick, a `boolean` for the A button, an `int` for the loop count, and the seconds as another `double`. `publish` takes all four the same way.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests
'*LessonsTest.l2a_logsEveryStickAndTheAButton'
```

If it didn't, gradle prints a line starting `See the report at :.` Open it and click the test that failed. The message it shows is the check that did not hold:

- nothing was logged as "stick/leftY" and then a list of names. That name never went out, or it went out spelt differently. The list is what the robot publishes for itself.
- forward is positive after negating the stick expected:<1.0> but was:<-1.0>. A minus sign is missing.

### Step 2: say when the A button changes

Fill TODO 5 in the same file.

The A button is either down or up, every loop. What you want is the loop it *changed* on, and that takes two values: this loop's and last loop's. `previousButtonA` holds last loop's, and the field is already there.

The value you publish here is a `String`, which is text rather than a number. `publish` takes that the same way too.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests
'*LessonsTest.l2a_saysWhenTheButtonIsPressedAndReleased'
```

If it didn't, open the report the same way.

- the loop the button went down on says so expected:<button A pressed> but was:<null>. Nothing was published as lesson/event.
- letting go says so expected:<button A released> but was:<button A pressed>. The last line of the TODO is missing: `previousButtonA` is never given this loop's value, so no loop ever looks like a change back.

### Step 3: run it on the robot

Send the code to the robot. On the Driver Station, pick the `Lessons` group and then `L2a Sticks`. Press INIT, then PLAY.

**You'll know it worked when** the wheels stay still and the Driver Station says the OpMode is running. Push a stick and press A a few times. The numbers are going out, and watching them is `task.panels`.

If it didn't, and `L2a Sticks` is not in the list, the code that is on the robot is not the code you just wrote. Send it again and watch for the message that says it landed.

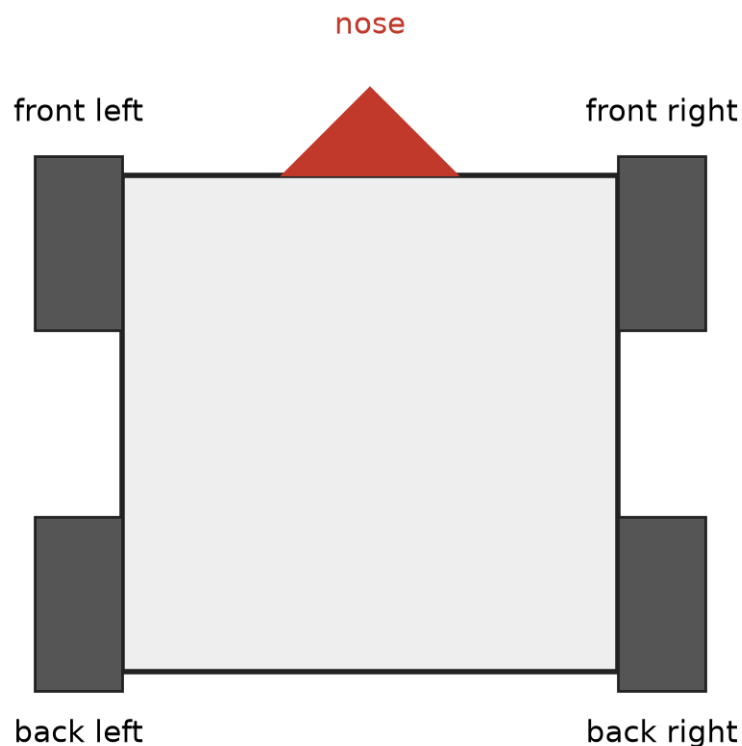
**Step 4: give the wheels their powers**

Three files, and no wheel turns until all three are done.

- `LessonsDriveTrain.java`, TODO 1: `writeWheels`, the only place in any lesson that talks to the four motors. You write it once, and every lesson after this one uses it.
- `LessonsDriveTrain.java`, TODO 2: `driveWheelsNow`, which puts four powers into the four slots and sends them.
- `L2TankDriveTrain.java`: `sticks`, which is what tank drive means. The left stick's number goes to both left wheels and the right stick's to both right wheels, so two of the four are the same.

FL, FR, BL and BR name the four slots: front left, front right, back left, back right. Using them writes the wheel order down once instead of guessing it in every method.

Then fill TODO 1 in `L2bTankOpMode.java`, which reads the two sticks and hands them over.



*Stand-in drawing for a photo still to be taken*

Figure 2.8. The robot from above with its nose marked, and each of the four wheels labelled front left, front right, back left and back right.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l2b*'
```

Two tests run. One watches the four motor powers. The other drives a simulated robot up the field for a second and checks that it got there.

If it didn't, open the report.

- the left stick runs the front left wheel expected:<1.0> but was:<0.0>, and drove forward, and got a fair way: 0.0. Nothing reached the motors at all, so `writeWheels` is still empty, or `driveWheelsNow` never calls it.
- and the back left too expected:<1.0> but was:<0.0>. The four powers are in the wrong order. `driveWheelsNow` takes front left, front right, back left, back right. Handing it left, left, right, right passes the straight-line test and fails this one.

### Step 5: write the numbers down again

Fill TODO 2 in `L2bTankOpMode.java`: the four stick axes, the A button, and the pressed and released events, the same as L2a. The loop count and the seconds are gone, because the robot publishes both for itself.

**You'll know it worked when** you see the names in Panels. No test watches this step: both L2b tests pass whether the names go out or not. A green gate is not a finished lesson, which is what `task.bench` is about.

### Step 6: drive it

Put the robot on the floor with room in front of it. Pick L2b Tank in the Lessons group. INIT, then PLAY.

Push both sticks gently forward.

**You'll know it worked when** the robot drives forward. Pull both back and it comes back. Push one forward and one back and it spins in place.

**If it didn't**, one of these fits what you see. Each is described from the code and not from a robot that was driven that way:

- It spins when both sticks are forward, or drives when they are opposed. One side's wheels turn backwards. That is a motor direction, and it is [task.directions](#).
- One corner drags or fights the others. That corner's power is going to a different wheel than you think. Check the order in `sticks` against the four names in `driveWheelsNow`.
- It creeps when both sticks are centred. The sticks do not read exactly 0 when let go, and nothing in L2 tidies that up. Fixing it is [task.l3](#).

## 2.3.4 What you just did

`loop` runs over and over, from PLAY until STOP, and everything the robot does is one pass through it. Each pass reads the sticks, turns those two numbers into four powers, and hands the powers to the motors. A power is a number from -1 to 1, and it is how hard the motor pushes rather than how fast the wheel turns: 0.5 is half of what the motor can push with, and the wheel it is on will turn slower up a ramp than it does on carpet. Powers are enough for every lesson up to L15, and asking for a speed in inches per second instead is `task.l16`.

## 2.3.5 Where next

- [task.l3](#) stops the creep, and makes a gently pushed stick mean gently.
- [task.directions](#) is where a wheel that turns the wrong way gets sorted out.
- `task.panels` is how you watch the numbers this task published.

`task.panels` is not written yet.

## 2.4 L3: make the sticks feel right

### 2.4.1 What you will have when this is done

The robot sits still when both sticks are let go, and a gently pushed stick makes it move gently. Same wheels, same tank drive as L2. What changes is what happens to the stick number on its way to the motors.

### 2.4.2 Before you start

- L2 is finished, so L2b Tank drives.
- `task.directions` is done, so a wheel that goes the wrong way is not in the way of reading what these two lessons do.
- About 2 m of clear floor.

You write in three files. `LessonsDriveTrain.java` gets the two new methods, and `L3aDeadbandOpMode.java` and `L3bSquaredOpMode.java` each use one more of them than the last.

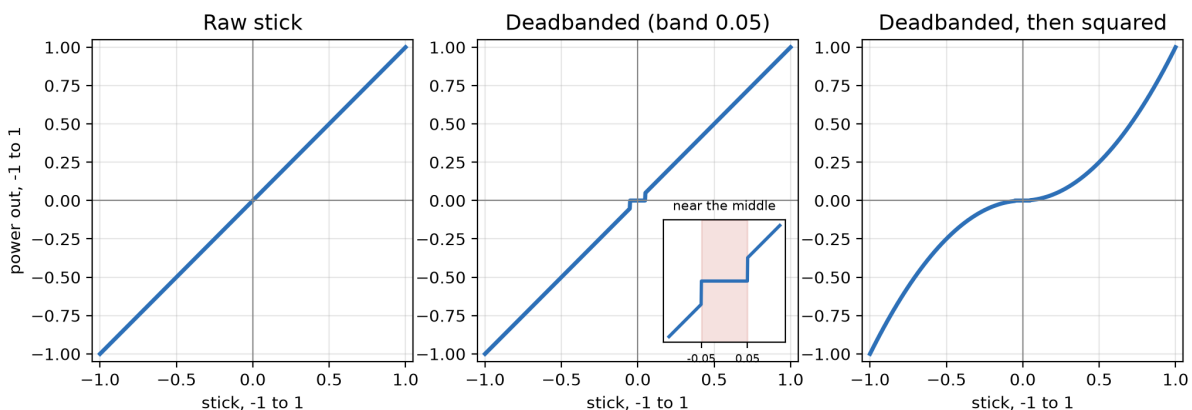


Figure 2.9. Three graphs side by side, each with the stick from -1 to 1 along the bottom and the power out from -1 to 1 up the side: the raw stick as a straight line, the deadbanded stick as a line with a flat step at the middle, and the squared stick as an S through the middle.

### 2.4.3 The steps

#### Step 1: ignore a stick that was let go

A stick that has been let go does not read 0. It reads a few hundredths, and a few hundredths of power is enough to make the robot crawl across the floor on its own. A **deadband** is the answer: any reading smaller than the band counts as let go.

Write `deadband` in `LessonsDriveTrain.java`, TODO 7. It takes the stick and the band, and answers either 0 or the stick itself.

`Math.abs` is the part worth thinking about. A stick pulled back reads -0.5, and -0.5 is smaller than 0.05 the way a thermometer means smaller. What you want to compare is how far from the middle it is, with the minus sign taken off, and that is what `Math.abs` does.

Then open `L3aDeadbandOpMode.java`. TODO 1 is the band itself, and the number is your choice: 0.05 is a good first guess. TODO 2 puts each stick through `deadband` and hands the two on.

You'll know it worked when this prints `BUILD SUCCESSFUL`:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.L3a*'
```

If it didn't, open the report gradle names.

- outside the band, the stick is passed through untouched expected:<0.5> but was:<0.0>. Nothing reached the wheels, so TODO 2 has not handed the two shaped sticks to drivetrain.sticks().
- inside the deadband, so the robot does not creep expected:<0.0> but was:<0.029999999329447746>. The band is still 0, so nothing is ever inside it. That long number is the stick the test pushed: a gamepad axis is a float, which keeps fewer digits than a double, and 0.03 is not one of the numbers it can hold exactly.

### Step 2: drive it with your hands off the sticks

Send the code to the robot. Run L3a Deadband, press PLAY, and let go of both sticks.

**You'll know it worked when** the robot sits still. Then push a stick a long way and let go again, and it stops.

If it didn't, and it still creeps, the band is smaller than what this gamepad reads when it is let go. Raise it a little: 0.05 to 0.1 is the usual range. A band much bigger than that starts to throw away real driving.

### Step 3: make a gentle push mean gently

With the deadband in, the stick is either ignored or passed straight through. Passed straight through means a tenth of a push is a tenth of the power, and near the middle that is more robot than a driver wants. Squaring fixes it: half a push becomes a quarter of the power, and a full push is still full power.

Write squared in LessonsDriveTrain.java, TODO 8. Squaring a negative number the ordinary way loses the minus sign, and a driver asking for backwards and getting forwards is worse than no squaring at all. So work out the size first, then put the sign back.

Then fill L3bSquaredOpMode.java, TODO 1 and TODO 2. It deadbands each stick and then squares it, one step to a line. Each lesson keeps its own band, so the number you picked in step 1 gets picked again here.

**You'll know it worked when** this prints BUILD SUCCESSFUL:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.L3b*'
```

If it didn't, open the report.

- and squaring keeps the sign expected:<-0.25> but was:<0.25>. The sign was not put back, so backwards came out forwards.
- and squaring keeps the sign expected:<-0.25> but was:<0.0>. This one is deadband, not squared: Math.abs is missing, so every negative stick reads as smaller than the band and gets thrown away. The robot drives forwards and refuses to back up. The L3a test does not catch it, because every stick it pushes outside the band is a forwards one.

### Step 4: drive it gently

Run L3b Squared. Push a stick about a third of the way and hold it.

**You'll know it worked when** the robot creeps along under control. Push the stick all the way and it still gives everything the robot has.

If it didn't, and it backs up but will not go forward, the sign is wrong. Or the other way round. Step 3's second message is the same fault, seen from the bench.

## 2.4.4 What you just did

Both new methods sit between the stick and the wheels. Neither one knows what a wheel is. That is

why they live in `LessonsDriveTrain` and not in the lesson. L15 and L16 drive through the same two methods, and they call the ones you wrote here. There is no second copy. Squaring is also the first time this guide bends what a stick means. Half a push really is a quarter of the power. So the numbers in Panels are no longer the numbers your hand is making, and that is why `stick/left_raw` sits beside `stick/left_shaped`. You can watch both at once.

## 2.4.5 Where next

- `task.l4` gives the robot one stick to drive with and one to turn with.

## 2.5 L4: one stick to drive, one to turn

### 2.5.1 What you will have when this is done

Arcade drive. One stick decides how fast the robot goes. The other decides how hard it turns. The robot takes the curve you asked for, even when you ask for more power than it has.

### 2.5.2 Before you start

- L3 is finished, so L3b Squared drives.
- Clear floor about 3 m across. This one drives in circles.

You write in three files: `L4ArcadeDriveTrain.java`, `L4ArcadeOpMode.java`, and one more method in `LessonsDriveTrain.java`.

### 2.5.3 The steps

#### Step 1: mix the two sticks into two sides

Tank drive asked the driver to think in left wheels and right wheels. Arcade asks for forward and turn instead, which is how a driver already thinks, and the robot works out the two sides.

Write sticks in `L4ArcadeDriveTrain.java`. It takes a forward speed and a turn speed, and turns them into a left speed and a right speed. Turning counter-clockwise means the left side goes slower and the right side goes faster, so one of the two gets the turn taken off it and the other gets it added on. Then hand the two to `driveWheelsNow`, the way `L2TankDriveTrain` does.

Then fill the TODO in `L4ArcadeOpMode.java`. Forward comes from the left stick's y axis and turn from the right stick's x axis. Both get a minus sign on the way in, and both get logged.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l4*'
```

If it didn't, open the report gradle names.

- turning right speeds the left wheels up expected:<0.75> but was:<0.0>. Nothing reached the wheels, so `driveWheelsNow` is not being called with the two speeds.
- turning right speeds the left wheels up expected:<0.75> but was:<0.25>. The turn is going on the wrong side. Swap which one gets it added and which gets it taken off.

#### Step 2: share out the power when there is not enough

Push the drive stick all the way forward and the turn stick half over, and arcade asks one side for 1.5 . A motor cannot do 1.5, and the largest it will take is 1. Something has to give.

The wrong answer is to chop it off at 1, because that changes the *shape* of what the driver asked for: the forward part keeps all of what it wanted and the turn part loses a third of what it wanted, so

the robot drives a wider curve than the sticks describe. The right answer is to divide every wheel by the biggest of them, which keeps the shape and makes the whole thing slower.

Write `normalized` in `LessonsDriveTrain.java`, TODO 3. Find the biggest of the four with the minus signs taken off, or 1 if none of them reaches 1, and divide every power by it. Start the biggest at 1, not at 0: starting at 0 would scale every gentle request up to full power.

**You'll know it worked when** step 3 says so on the floor. No test catches this one. The L4 test passes with `normalized` not written at all. The motors already cut anything over 1 down to 1, and the two stick values that test uses come out the same either way.

**If it didn't**, the test that passed in step 1 may now fail. Look for turning right speeds the left wheels up expected: <0.75> but was: <1.0>. That one means the biggest starts at 0 rather than 1. A half-speed request is being scaled up to full.

### Step 3: drive two circles

Run L4 Arcade. Hold the drive stick half forward and the turn stick a quarter over, and let the robot go all the way round. Then drive the same circle with both sticks twice as far over: drive all the way, turn half.

**You'll know it worked when** the two circles are the same size. The second one is faster, and that is the only difference.

**If it didn't**, and the second circle is much wider than the first, `normalized` is not dividing. In the simulator the first circle has a radius of 32.2 in either way. The second is 32.2 in with `normalized` written and 48.3 in without it, which is half again as wide.

---

### Where those two radii come from

The simulator, not a floor. L4 Arcade was run for 1 s at each pair of sticks, with `normalized` written and then deleted, and the radius worked out from where the robot ended up. A real floor has grip and slip in it, so read the shape rather than the number.

---

## 2.5.4 What you just did

`normalized` is the first code here that does not decide where the robot goes. It decides what to do when the driver asks for more than the robot has. Its answer is *the same thing, slower*. That is why it divides instead of cutting off. It also writes down what it divided by, as `powerScale`. A log can then say how much of what the driver asked for really happened. Every lesson from here on gets this for free, because `driveWheelsNow` already calls it.

## 2.5.5 Where next

- [task.15](#) gives the robot a third thing to do with the sticks: going sideways.

## 2.6 L5: slide it sideways

### 2.6.1 What you will have when this is done

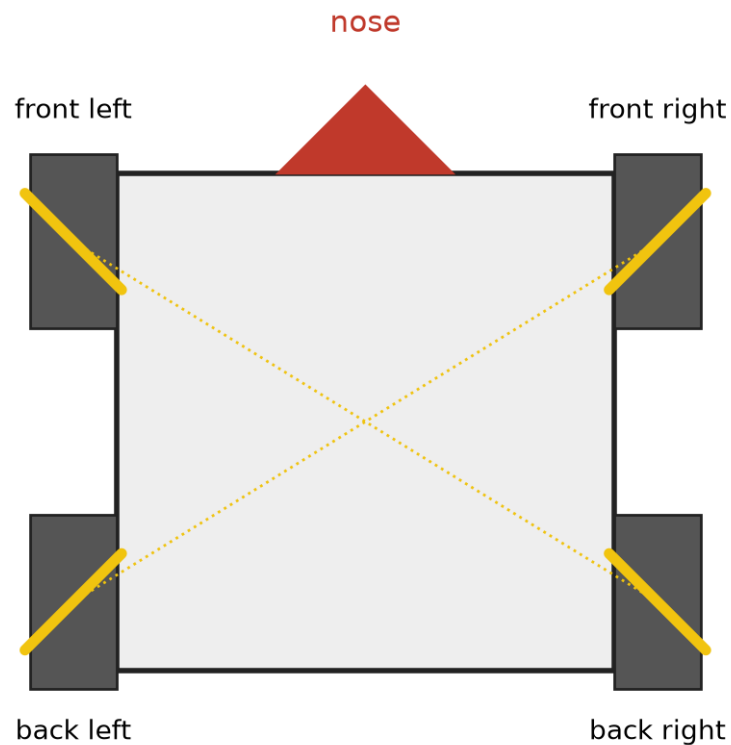
The robot slides sideways without turning. The left stick drives it and slides it, the right stick turns it, and all three can happen at once. Everything is relative to the robot's own nose, whichever way the nose happens to be pointing.

### 2.6.2 Before you start

- L4 is finished, so `normalized` is written. L5 needs it. Asking for forward and sideways in the same push asks for more than full power, and `normalized` is what sorts that out.
- `task.directions` is done.

- Mecanum wheels on all four corners, with their rollers making an X seen from above. Step 2 says how to read that.
- About 2 m by 2 m of clear floor.

You write in two files, `L5HolonomicDriveTrain.java` and `L5HolonomicOpMode.java`, and the test needs both.



the top roller of each wheel, at 45 degrees: together they make an X

Figure 2.10. The robot from above with its nose marked, and the top roller of each of the four wheels drawn as a line at 45 degrees, so the four lines make an X across the robot.

### 2.6.3 The steps

#### Step 1: mix all four wheels

A mecanum wheel has rollers set at 45 degrees. Spinning it pushes the robot along and sideways at the same time. Spin all four forward and the sideways pushes cancel, so the robot drives forward. Spin one diagonal forward and the other diagonal back and the forward pushes cancel instead. What is left is a slide.

So there are three numbers now, not two: forward, slide left, turn. Each corner adds the three up its own way.

Open `L5HolonomicDriveTrain.java`. The TODO asks for four named powers and one call to `driveWheelsNow`. The four lines are written out in the class javadoc just above, and so are three places that draw where the signs come from.

Then open `L5HolonomicOpMode.java`. The left stick's x axis is the new one, and it needs negating

like the other two. The stick reads positive to the right, and `strafeLeftSpeed` counts positive to the left.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.15*'
```

**If it didn't**, open the report gradle names.

- driving forward turns every wheel the same way expected:<1.0> but was:<0.0>. Nothing reached the wheels at all, so one of the two files is still blank.
- sliding left runs one diagonal back and the other forward expected:<-1.0> but was:<1.0>. The slide goes the wrong way. Either the minus sign is missing on `gamepad1.left_stick_x`, or the strafe signs are swapped in the four lines. The two mistakes look the same from here.
- turning right runs the left side forward expected:<0.5> but was:<-0.5>. Same fault, on the turn instead of the slide.
- expected:<1.0> but was:<-1.0>, with no words in front of it. Only the first check of each group carries words, so this one is the back left wheel while sliding left. The two back lines have the two front lines' strafe signs. Copying the first two lines down and editing only the turn does exactly this.

### Step 2: slide it sideways on the floor

Send the code to the robot. Run L5 Holonomic and press PLAY. Push the left stick straight left and hold it there.

**You'll know it worked when** the robot slides to its own left with the nose still pointing the same way. Push straight right and it comes back.

**If it didn't**, and it slid the wrong way, that is step 1's second message seen from the floor. The test catches it, so run the test again.

**If it didn't**, and it turned instead of sliding, or ground along hardly moving, a wheel is on the wrong corner. Look down on the robot. The top roller of each wheel runs at 45 degrees, and the four of them make an X. Forward still works with a wheel in the wrong place, because all four still push forward. It is the sideways halves that stop cancelling.

---

### Where the X comes from

L5HolonomicDriveTrain's own javadoc and Game Manual 0's mecanum page, which it links. No robot here has been driven with a wheel on the wrong corner, so the symptom above is what the arithmetic says, not something that has been watched.

---

### Step 3: drive a diagonal

Push the left stick into a corner, forward and left together, and hold it.

**You'll know it worked when** the robot goes diagonally with the nose still forward. It covers less ground each way than it would going straight, which is the point of step 3.

**If it didn't**, and it drifted into a slow spin, that is the right stick at rest. L5 does not deadband anything. L3's two methods belong to L3's two lessons, and this one reads the sticks raw, so a stick that does not quite read 0 turns the robot.

---

### Where these numbers come from

The simulator, held at one stick position for 1 s each time. Full forward covered 54.75 in. Full left covered 37.07 in. The corner of the stick covered 27.38 in forward and 18.53 in left, both halves of the full numbers, because `normalized` divided all four powers by 2.

Half into the corner measured the same as all the way into the corner. At half, the front right wheel

asks for exactly 1. At full it asks for 2, and `normalized` divides by 2 and gives 1 again. The corner of a square stick is already full power halfway out.

A real floor is slower than this and scrubs the rollers sideways. Read the ratios rather than the numbers.

---

## 2.6.4 What you just did

Four wheels, three numbers, one line each. Those four lines are the whole of mecanum. The path follower works with the same four lines later. So a robot that slides correctly here follows a path correctly in L6.

Sliding is slower than driving. The simulator keeps two different top speeds, one along the robot and one across it, and a second of driving beat a second of sliding by 54.75 in to 37.07 in. Expect that on a floor too, and plan a route that needs less sideways than forwards.

This is also the last lesson where the drivetrain owns the wheels by itself. `L5HolonomicDriveTrain` has no `mix`, so the path follower cannot drive it: hand one to the follower and it stops at the first update and says so. That refusal is on purpose. It is what lets L2 to L5 exist without any of the follower's arithmetic, and L6 is where the missing method arrives.

## 2.6.5 Where next

- `task.panels` puts the numbers a lesson sends on a screen while somebody drives. From here on the lessons are easier to debug when you can watch them.
- `task.l6` hands the wheels to the path follower, and hands them back when the driver wants them.

## 2.7 Watch the numbers while it drives

### 2.7.1 What you will have when this is done

The robot's own numbers on a screen while somebody drives it. You get a table of every value the lesson sends, a graph of any two of them, and a picture of where the robot thinks it is. After this, "it went crooked" is a graph you can point at. It stops being an argument.

### 2.7.2 Before you start

- L3 is finished, because step 3 graphs the two numbers L3b makes.
- A robot with its battery in, running a lesson.
- A laptop or a phone that can join the robot's Wi-Fi.

### 2.7.3 The steps

#### Step 1: open Panels

Join the robot's Wi-Fi, then open `http://192.168.43.1:8001`.

Panels is a web page the Robot Controller app serves. Nothing in the lesson starts it and nothing turns it on.

**You'll know it worked when** the page loads and shows its panels, even before an OpMode runs.

**If it didn't**, and the page never loads, the laptop is on the wrong Wi-Fi. Check which network it joined.

**If it didn't**, and you get a different page full of tuning buttons, that is AutoTune on port 10158. It is a second server in the same app, and it is not this.

### Step 2: find the numbers your lesson sends

Run L3b Squared and press PLAY. The Telemetry panel fills with one line per value, each shaped name: number.

The Graph panel has a list to pick from, and a name only joins that list **after it has been sent once**. So start the OpMode first, then go looking. The list is flat, in the order the names first arrived.

**You'll know it worked when** `stick/left_raw` and `stick/left_shaped` are both in the list, and both numbers move when you move the left stick. The [cheat sheet](#) lists every name each lesson sends.

**If it didn't**, and a name you expected is missing, nothing sent it. Either that lesson does not publish it, or the line that publishes it is still a blank.

### Step 3: graph the two against each other

Pick `stick/left_raw` and `stick/left_shaped` and push the stick slowly from nothing up to full.

**You'll know it worked when** the two lines leave zero together and the shaped one stays under the raw one the whole way up, until they meet at full stick. That is L3b drawn: squaring a number below 1 makes it smaller.

**If it didn't**, and the two lines lie exactly on top of each other, nothing is being shaped. That is L3b's blank still open.

**If it didn't**, and both lines look like a staircase rather than a curve, that is the sampling and it is normal. The robot sends to Panels every 75 ms and drops the loops in between, so with a loop of about 5 ms, roughly one loop in 15 reaches the graph. For the same reason one slow loop will not show in `loop/period_ms`. Graph `loop/max_period_ms` instead: it only ever steps up, and a step means some loop blocked.

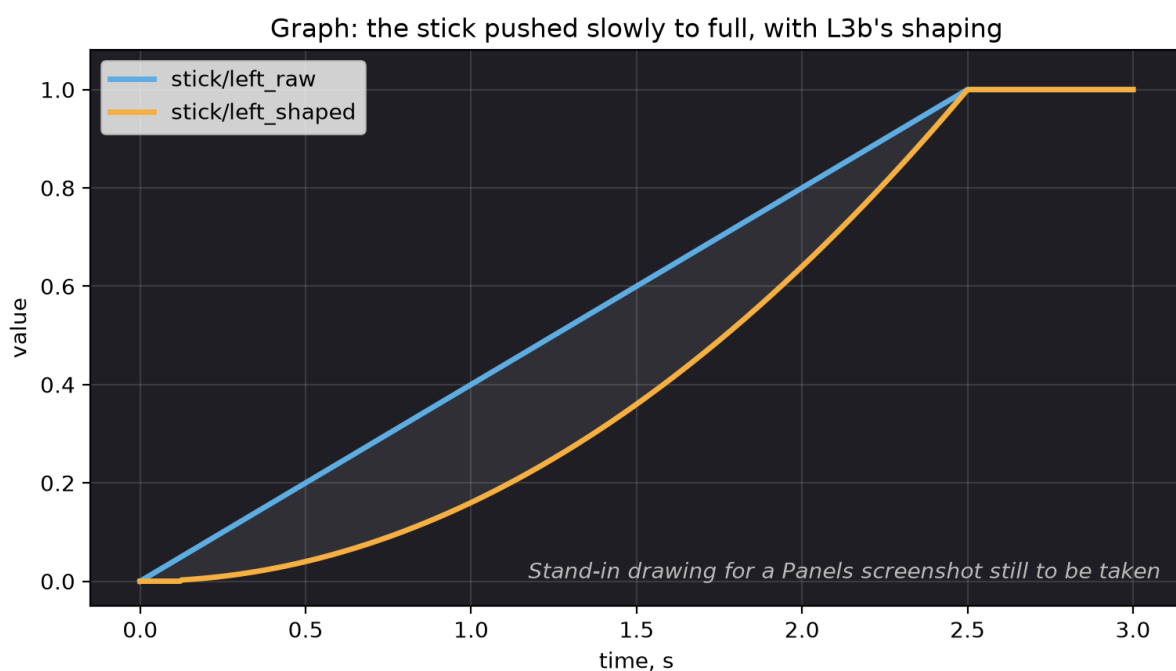


Figure 2.11. The Panels Graph panel with `stick/left_raw` and `stick/left_shaped` both plotted while the stick is pushed slowly to full, so the gap between the two curves is the thing a student is looking for.

### Step 4: look at where it thinks it is

Open the Field panel. There is nothing to set up: the lesson sends the field's coordinates with every frame.

A blue square 18 in by 18 in is the robot, and the line out of it is the way the robot faces. A grey line

behind it is where it has been since the OpMode started. In a lesson that follows a path you also get an orange line for the piece of path being followed, and a green square for the nearest point on that path at the heading the path wants there.

**You'll know it worked when** the blue square moves as the robot moves. Turn the robot, and the line out of the square swings round with it.

**If it didn't**, and the robot draws in a corner with most of the square off the field, nothing is wrong. Pedro puts (0, 0) in a corner and the field runs 0 in to 144 in, so a lesson that never sets a pose starts there.

**If it didn't**, and there is no green square or orange line, you are in a teleop lesson. There is no path to draw. You get the robot and its trail.

---

### What has not been done here

Nobody opened Panels for this page. Every step is taken from the team's own `PANELS.md`, and the names in step 2 are read out of the lesson code, the same way the [cheat sheet](#) gets them. What each *if it didn't* line describes follows from those two, and none of them was watched on a robot.

---

## 2.7.4 What you just did

Panels has no separate graph call. It reads the telemetry text, and any line shaped `name: number` becomes a table row and a graph series at once. That is why one `Tracker.publish` call gets you both, and why a name may hold anything except a colon.

The same call also writes the number into the flight log on the robot. Panels is the live view and the log is the recording, and they carry the same names, so a graph you found while driving is a graph you can find again afterwards.

The gap between the blue square and the green one is worth knowing. The green square is the nearest point on the path, not where the robot should have got to by now. So the gap says how far off the line the robot is and how crooked it is sitting. It does not say whether the robot is running late.

## 2.7.5 Where next

- [task.bench](#) is the other half of this. It runs the same lessons on a laptop, with no robot at all.

## 2.8 Check your work without a robot

### 2.8.1 What you will have when this is done

Two ways to try a lesson with no robot in front of you. One runs every test and tells you which lessons are finished. The other runs a single lesson on the laptop and draws the robot moving, so you can see what your code does before it is your turn at the field.

### 2.8.2 Before you start

- Android Studio opens the project, and you can type a Gradle command into its terminal. Getting a laptop that far has no page yet, so ask a mentor.
- [L2](#) is written, or step 2 has no lesson to show you.
- AdvantageScope installed on the same laptop, for step 2 and step 3.

## 2.8.3 The steps

### Step 1: run the tests and read the report

```
./gradlew :TeamCode:test
```

That runs the tests for everything the lessons are built on, and leaves the lesson tests out. It says so while it runs:

```
Skipping org.firstinspires.ftc.teamcode.lessons.*. Run them with  
:TeamCode:testLessons.
```

**You'll know it worked when** it ends with `BUILD SUCCESSFUL`. Nothing underneath the lessons is broken, so whatever fails next is yours.

**If it didn't**, and it says `SDK location not found`, the laptop has no `local.properties` file yet. That file is not kept in git, and Android Studio writes it the first time it opens the project. Open the project once, then run the command again.

Now the tests for your own work:

```
./gradlew :TeamCode:testLessons
```

On a fresh copy, where nobody has written a lesson yet, the report counts 66 tests and 54 failures. Measured on 2026-09-28. Fifty-four failures is not a broken copy. Each one is a lesson nobody has written, and the count drops as you write them.

**You'll know it worked when** the last lines name a report:

```
> There were failing tests. See the report at: file:///.../index.html
```

Open that link in a browser, click the class and then the test, and you get the message the test wrote. The console does not give you that. All the console gives you is this:

```
MecanumEncoderLocalizerTest > countsStraightAhead FAILED  
java.lang.AssertionError at MecanumEncoderLocalizerTest.java:63
```

A file and a line, and no word about what went wrong. The report for that same run also holds `robot relative while held expected:<1.0> but was:<0.0>`, which is L12's blank still open. Every *if it didn't* line in this guide quotes a message out of the report, so every one of them starts here.

### Step 2: watch one lesson move

A test says where the robot ended up. It does not show you the robot. For that, run one lesson on its own:

```
./gradlew :TeamCode:simRun --args="lessons.L2bTankOpMode --left_stick_y=-1  
--right_stick_y=-1"
```

The first word inside the quotes is the lesson's class name, with the package it sits in before it. Each option after it holds one gamepad control at one value for the whole run, named the way the gamepad names it. So `--left_stick_y=-1` is the left stick pushed fully forward, and `--a=true` is the A button held down. Ask for `--args="--help"` and it lists every control you can set.

To push the sticks yourself, plug a gamepad into the laptop and add `--pad`:

```
./gradlew :TeamCode:simRun --args="lessons.L2bTankOpMode --pad"
```

One gamepad becomes `gamepad1` on its own. With two plugged in, hold Start and press A on the one you want to drive with.

Plug it in before the run or during it, whichever suits. Pull it out and the sticks go back to rest, so the robot stops. Plug it back in and it is yours again.

Use `--pad`, or set the controls yourself, but not both. Together they are an error, and the run names the setting that clashed.

It prints where it is listening, then runs:

```
NT: Listening on NT3 port 1735, NT4 port 5810
L2bTankOpMode running. Connect AdvantageScope to 127.0.0.1 as NetworkTables 4, and
Ctrl-C to stop.
```

Open AdvantageScope and connect to `127.0.0.1` as NetworkTables 4. The topics arrive under `sim/`. Add a 2D field and give it `sim/Pose`.

**You'll know it worked when** the robot moves. `sim/wheels/frontLeft` and its three neighbours say what reached each motor, and `sim/vel/forward_ips` says how fast the robot is going.

**If it didn't**, and the robot sits still with all four wheels reading 0, that lesson is still blank. Watched with L2b's blanks open and both sticks forward: the pose never changed for the whole run. Nothing is wrong with the simulator. It ran a lesson that writes nothing to the motors.

**If it didn't**, and no topics arrive at all, nothing connected. The address is `127.0.0.1`, and the kind is NetworkTables 4 rather than a log file.

Ctrl-C stops it. The flight log is in the project's top folder, which the first line it prints names.

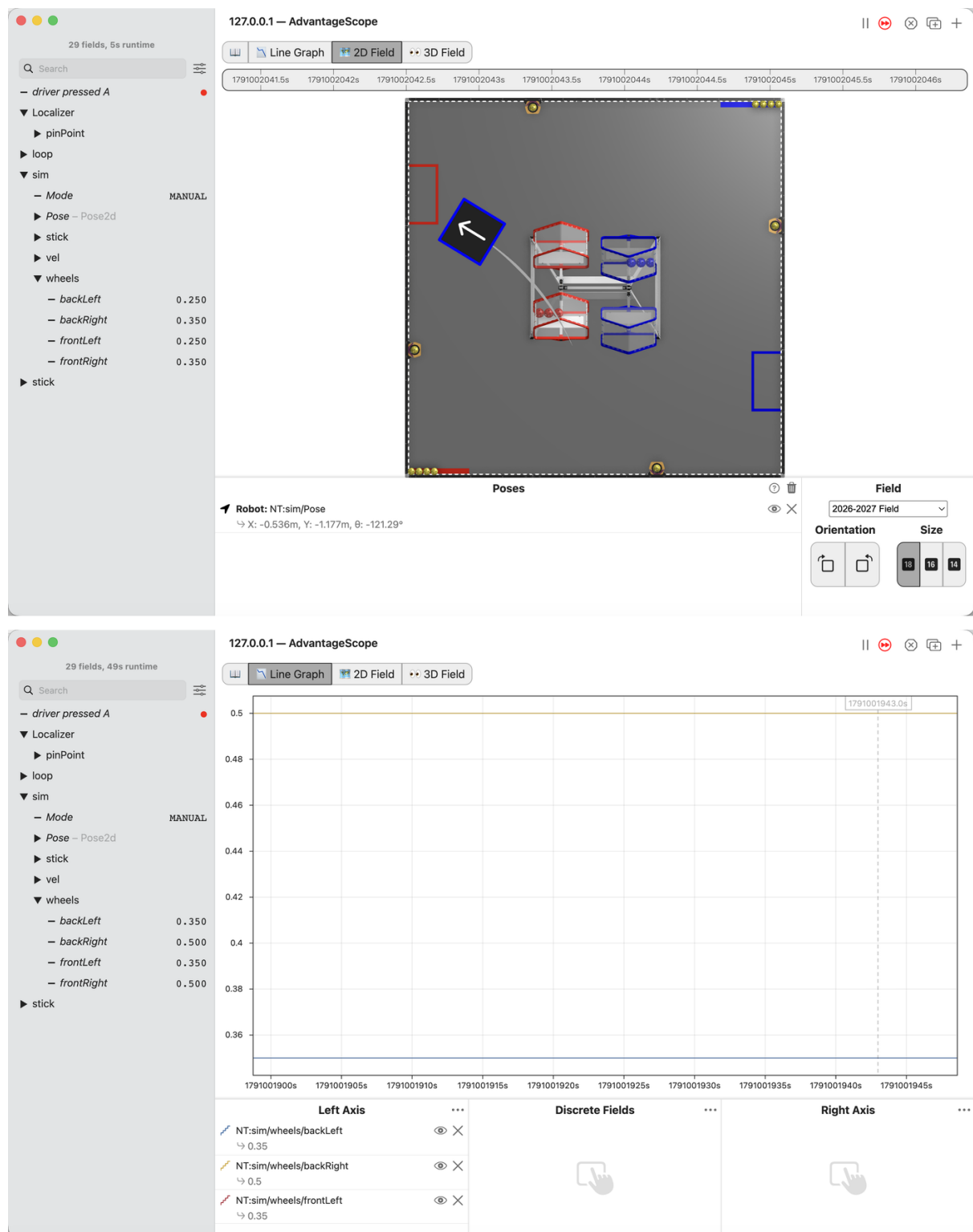


Figure 2.12. AdvantageScope with a 2D field showing `sim/Pose`, the topic list open beside it, and a graph of the four `sim/wheels` values, so a student can match their own screen to what this step describes.

### Which way round the field gets drawn is being looked into

The corner the arrow starts in and the way it points do not yet agree with the numbers going out. Trust that the robot moves, and how far it moves, rather than which way it faces on the screen. It is

filed as `sim.field_orientation`.

### Step 3: open the log a failing test left

Some test classes hand you their flight log when they fail. Watched, by asking the simulator's own test for 400 in when the robot drives 54.75 in:

```
java.lang.AssertionError: drove forward, and got a fair way: 54.75097300967889
AdvantageScope can open what ran:
/var/folders/.../Tank-20260928-065400.wpilog (37502 bytes)
```

**You'll know it worked when** that file opens in AdvantageScope as a log, not as a connection, and holds the run up to the moment the test gave up. There is more in it than `simRun` sends out: `/Robot/Pose`, `/Robot/Speeds`, `/Robot/Twist`, all four wheels, and the loop timing.

**If it didn't**, and the failure names no log, that class never asked for one. The lesson tests are all like that today. So `l2b_theSticksMoveTheSimulatedRobot` fails with `drove forward`, and got a `fair way: 0.0` and no path to open. A log does get written, and it holds 0 bytes, because nothing closed it. Watched. Use step 2 instead, and tell a mentor.

## 2.8.4 What you just did

Both halves run the same model of the robot, and it is a small one. Each loop, the four wheel powers are mixed back into forward, sideways and turn. Each of those three chases what was asked of it with a 0.15 s lag. The answer is added up into a pose.

| Axis     | Flat out  |
|----------|-----------|
| Forward  | 64.4 in/s |
| Sideways | 43.6 in/s |
| Turn     | 4.0 rad/s |

So the model has no slip, no scrub, no sagging battery, no floor, no field wall, no motor wired backwards and no Pinpoint. It will drive straight through the perimeter and never mention it, and it cannot tell you a path is too fast for the wheels. When the simulator and the robot disagree, the robot is right.

That is why a green test run is not a finished lesson. These two catch the mistakes that live in the arithmetic, which are most of them. What is left over is the kind of mistake only a robot on a floor will show you, and that is why every lesson's last step is a robot on a floor.

## 2.8.5 Where next

- [L6](#) hands the wheels to the path follower. From here on you can try each lesson on the laptop before you take it to the field.

## 2.9 L6: let the path follower hold the drivetrain

### 2.9.1 What you will have when this is done

The robot drives on the sticks exactly as well as it did in L5, and the path follower is holding the drivetrain. The sticks still win while the driver is pushing them.

That matters because a follower holding the drivetrain can do things a lesson cannot. It knows where the robot is. It can hold a heading, and it can drive a path. None of that is possible while the follower is holding a stand-in, and all of it is what L8 onwards is about.

## 2.9.2 Before you start

- L5 is finished, so the four mixing sums are written once and work.
- L4 is finished, so normalized is written. This lesson calls it twice.
- About 2 m by 2 m of clear floor.

You write in three files. `LessonsDriveTrain.java` has three blanks, `L6FollowerDriveTrain.java` has two, and `L6WheelsFollowerOpMode.java` has two.

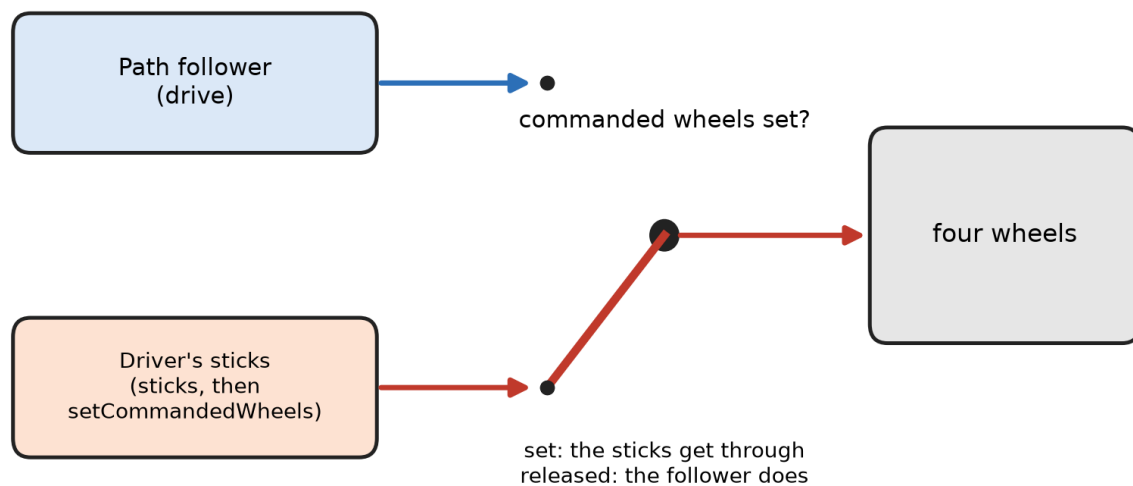


Figure 2.13. Two arrows reaching the same four wheels, one from the path follower and one from the driver's sticks, with the commanded-wheels switch between them showing which one gets through.

## 2.9.3 The steps

### Step 1: let the follower ask for wheels

Up to L5 the drivetrain wrote the motors the moment the lesson asked. From here the follower calls `drive` on it every update and hands it three numbers: forward, slide left, turn. The drivetrain has to turn those into four powers.

Write `mix` in `L6FollowerDriveTrain.java`, TODO 2. It is the same four sums as L5, in a four-slot array, named with FL, FR, BL and BR.

Then write `drive` in `LessonsDriveTrain.java`, TODO 6. It picks where the four powers come from and sends them on. Step 2 is the other half of that choice, so for now the `if` takes the branch that asks `mix`.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests
'*L6FollowerDriveTrainTest.the*WheelIs*'
```

There is one test per wheel, so a sum that is wrong names its own wheel.

**If it didn't**, open the report gradle names.

- sliding left runs it forward expected:<1.0> but was:<-1.0> from theBackLeftWheelIs..., and sliding left runs it backwards expected:<-1.0> but was:<1.0> from theBackRightWheelIs.... The two back sums are swapped. The test

names the wheel, and the message says which of the three numbers is going the wrong way into it.

- driving forward runs it forward expected:<1.0> but was:<0.0> from all four. Nothing reached the motors at all. Either mix still hands back the empty array it was given, or drive stops before calling writeWheels().
- NullPointerException: Cannot load from double array because "sourcePowers" is null. The if in drive is the wrong way round. commandedWheels == null is the case that asks mix.

### Step 2: let the sticks win

The follower is not the only one who wants the wheels. The driver does too, and the driver wins. A lesson says so by **commanding** the four wheels, and a commanded wheel ignores whatever the follower worked out until the lesson hands it back.

Write setCommandedWheels, TODO 4, and releaseCommandedWheels, TODO 5, both in LessonsDriveTrain.java. One remembers four powers, the other forgets them.

Then write sticks in L6FollowerDriveTrain.java, TODO 1. It goes through mix on purpose, so the driver's three numbers and the follower's three numbers mean the same thing at every wheel.

**You'll know it worked when** the whole class passes:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*L6FollowerDriveTrainTest*'
```

### If it didn't:

- the wheels are commanded, not left to the follower, with no numbers after it. sticks ran and nothing got commanded. Either sticks is still blank, or setCommandedWheels is.
- expected:<0.1> but was:<0.0> from writeWheelsSendsEachPowerToItsOwnMotor. That one commands four different numbers itself, so sticks is not the problem. setCommandedWheels is not storing them.
- released, so the follower's zero reaches the motor expected:<0.0> but was:<0.1>, on its own. releaseCommandedWheels is the blank that is left, and setting the field to null is the whole line.
- IllegalStateException: L6FollowerDriveTrain is driven by driveWheelsNow and by the follower at the same time. sticks called driveWheelsNow, which is L2b's door and writes the motors straight away. The follower is holding this drivetrain now, so two things would be writing the same four motors. The check stops it, and the message says which door to use instead.

---

### One mistake the tests here cannot see

Leaving normalized out of sticks passes all nine of them. CachedMotor chops any power over 1 back down to 1, and on the diagonal that gives the same four numbers normalized would have. It is the lopsided asks that come out wrong. Watched, with full forward and half a turn: 1.000 and 0.333 with normalized, and 1.000 and 0.500 without. The right side runs faster than it should, so the robot turns less than the driver asked. Step 4 is where you feel it.

---

### Step 3: hand the drivetrain over

Two blanks in L6WheelsFollowerOpMode.java. TODO 1 changes the field to an L6FollowerDriveTrain, and TODO 2 passes it to initAfter(drivetrain). L2 to L5 called initAfter() with nothing, because they drove their own wheels.

**No test watches this step.** The harness gives every OpMode its own follower and ignores what is passed to initAfter. Both blanks were left wrong on purpose, and the whole suite still passed.

**You'll know it worked when** step 4 drives. That is the only check there is.

**If it didn't**, and the robot does not move at all, this step is why. Commanding the wheels only remembers four numbers. Something has to call `drive` to send them, and that something is the follower. Watched: after `sticks(1, 0, 0)` and no follower update the front left motor holds `0.00`, and one update later it holds `1.00`.

#### **Step 4: drive it**

Send the code to the robot. Run `L6 Follower Wheels` and press **PLAY**. Drive it the way you drove L5.

**You'll know it worked when** it feels like L5. Forward, sideways, turn, and a diagonal, all the same as before.

**If it didn't**, and nothing moves at all, go back to step 3.

**If it didn't**, and it drives but turns lazily with the drive stick held all the way forward, that is `normalized` missing from `sticks`. It is the same fault L4 measured on the floor, and no test here catches it.

## **2.9.4 What you just did**

`drive` asks one question every update: has a lesson commanded the wheels? If not, mix what the follower asked for. If so, use what was commanded and throw the follower's answer away. That is nine lines, and every lesson from L8 on stands on them.

`mix` is also the method L5's drivetrain refuses to write. Hand an `L5HolonomicDriveTrain` to the follower and it stops at the first update and says why. This is the lesson where the missing method arrives, so the refusal is over.

This door is ours, not Pedro's. Pedro's own mecanum drivetrain does what the follower says. Ours lets the driver reach past it, to the wheels. The teleop lessons after L8 all need that. The follower keeps doing its own job while they do.

## **2.9.5 Where next**

- `task.l8` puts a second localizer beside the first one and logs both. It needs a follower that is holding a real drivetrain, which is what you just handed it.

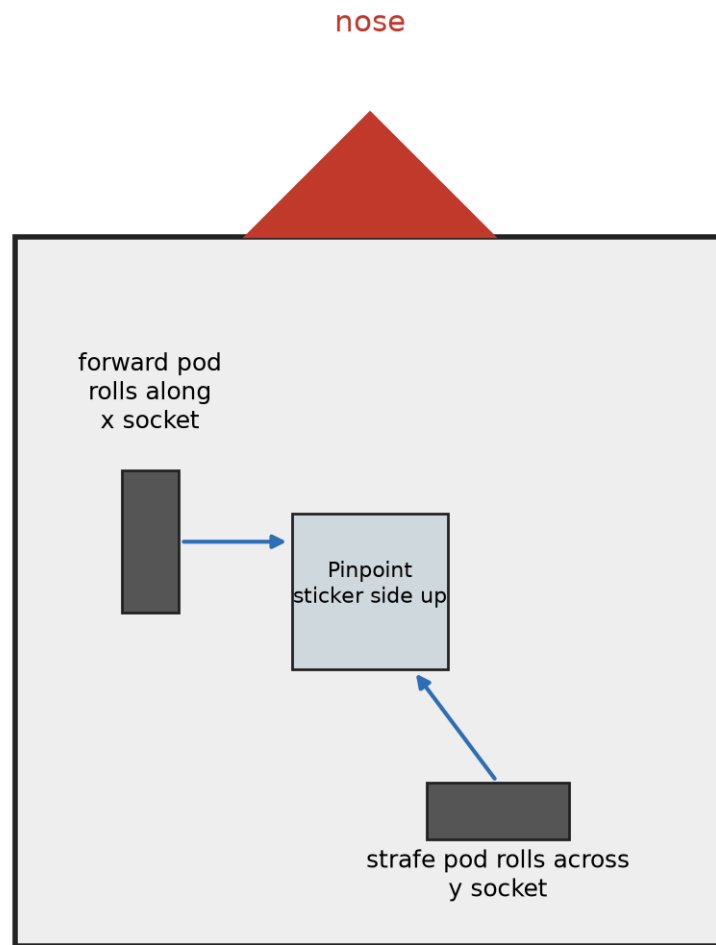
## **2.10 Mount the Pinpoint and plug it in**

### **2.10.1 What you will have when this is done**

The Pinpoint is bolted on, its two pods are in the right sockets, and the Control Hub finds it when you press **INIT**. The robot does not yet know how far the pods sit from its middle. That is `task.autotune`, and this page is what it needs first.

### **2.10.2 Before you start**

- L6 is finished. From L6 on the follower holds the drivetrain, and the follower is what builds the Pinpoint.
- The robot, the two odometry pods, and whatever driver the robot's own screws take.
- The I2C cable the Pinpoint came with.



*Stand-in drawing for a photo still to be taken*

Figure 2.14. The Pinpoint on the robot from above, with the forward pod and the strafe pod drawn at right angles to each other, each labelled with the socket it plugs into, and the sticker side marked facing up.

### 2.10.3 The steps

#### Step 1: put one pod along the robot and one across it

The Pinpoint counts two wheels that nothing drives. One rolls along the length of the chassis, and that one is the **forward** pod. The other rolls across it, and that one is the **strafe** pod.

Both have to touch the floor the whole time, and neither may be able to lift. A pod that skips loses count, and nothing downstream can tell that it did.

Then plug each one into its own socket. The forward pod goes in the x socket. The strafe pod goes in the y socket.

**You'll know it worked when** both pod wheels move as you push the robot forward. The forward pod turns. The strafe pod drags sideways.

**If it didn't**, and a pod wheel does not turn at all, it is not reaching the floor. Most mounts have a slot for that.

### Step 2: use an I2C port that is not port 0

The Pinpoint talks to the Control Hub over I2C, and it has to be on a port **other than port 0**. Port 0 already has the Control Hub's own IMU on it.

While you are in there, the sticker side of the Pinpoint, the side the ports are on, faces **up**.

Then open the Robot Configuration on the Driver Station and give it the name `pinpoint`, spelled exactly that way. That is the name the code asks for:

```
c.name.set("pinpoint");
```

**You'll know it worked when** L6 Follower Wheels gets through INIT without an error. Building the follower is what looks the Pinpoint up, so INIT finishing means it was found.

**If it didn't**, and the Driver Station shows this, the name in the configuration is not the name in the code:

```
Unable to find a hardware device with name "pinpoint" and type  
GoBildaPinpointDriver
```

### Step 3: hold the robot still while it inits

Pressing INIT builds the follower. The follower builds the Pinpoint. The Pinpoint then recalibrates its own gyro, which takes about a quarter of a second. The robot has to sit still for it.

So press INIT with the robot on the floor and your hands off it. If somebody was carrying it, wait until it is down.

**You'll know it worked when** the heading holds steady. Leave the robot alone after INIT and watch heading in Panels, or drive a slow square and see whether the nose angle comes back to where it started.

**If it didn't**, and the heading crawls one way while the robot sits still, INIT happened while the robot was moving. Press STOP and INIT again without touching it.

### Step 4: let AutoTune measure the pods

Four numbers are still missing: which way each pod counts, and how far each one sits from the middle of the robot. They are in `Constants.java`, in `localizerConfig`, and they were put there by AutoTune.

What they mean, from the goBILDA driver's own notes: the x offset is how far **sideways** the forward pod is from the point the robot turns about, and left counts up. The y offset is how far **forward** the strafe pod is from that point, and forward counts up. This robot's are about 1.06 in and about 3.36 in.

Running the procedure that measures them is `task.autotune`. It pushes the robot forward, then to the left, then spins it 180 degrees, and prints a `PinpointConfig` to paste in.

**You'll know it worked when** AutoTune's localization test agrees with the robot. The pose starts at (0, 0). Driving forward raises x. Strafing left raises y.

**If it didn't**, and x and y swap over, the two pods are in each other's sockets. Step 1 is where that gets fixed, and no amount of tuning works around it.

---

### Where this page comes from

Not from a robot. The three rules in steps 1 and 2 are Pedro's, from [its Pinpoint page](#), read on 2026-09-28, and no Pinpoint was wired each wrong way to see what each mistake looks like.

What is read from the code rather than recalled: the missing-device message is the one `HardwareMap.get` throws in the SDK; the recalibration at INIT is `PinpointConfig`'s default `resetMode`, which `PinpointLocalizer`'s constructor acts on; the stationary robot and the quarter second are the goBILDA driver's own javadoc; and the two offsets are this robot's, from

---

Constants.localizerConfig.

---

### 2.10.4 What you just did

Every lesson from L8 on believes the Pinpoint. L8 puts a second localizer beside it, and shows the drive encoders drifting away from it over a run. L17 measures the robot against it. A pod in the wrong socket makes all of that quietly wrong instead of loudly broken. That is why it gets a page of its own.

The Pinpoint also explains one thing. The follower needs nothing from the drive motors to know where the robot is. Two wheels that nothing drives cannot be scrubbed sideways by a mecanum roller, and cannot be spun by a wheel that slips.

### 2.10.5 Where next

- `task.autotune` measures the pods and fills in the four numbers. It is not written yet.
- `task.l8` compares the Pinpoint against the drive encoders and logs both.

## 2.11 L8: work out where the robot is from the drive wheels

### 2.11.1 What you will have when this is done

Your own localizer, running beside the robot's own one, with both of them published so you can graph them against each other. Neither of them is the truth. A tape measure is the truth, and finding out how far each guess has drifted from it is the point of the lesson.

This is the longest lesson in the course: about 70 lines in two files.

### 2.11.2 Before you start

- L6 is finished, so the follower is holding a real drivetrain. A shadow localizer needs one.
- A tape measure, and about 2 m by 2 m of clear floor.
- You write in `MecanumEncoderLocalizer.java`, which has seven TODOs and some lines no TODO asks for, and in `L8CompareLocalizersOpMode.java`, which has two.

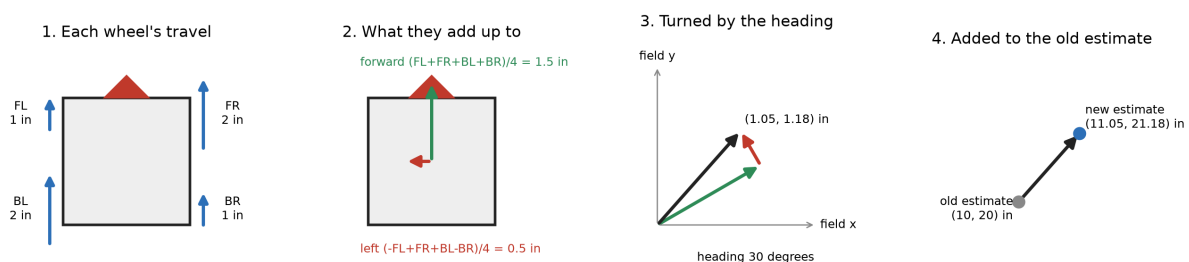


Figure 2.15. One loop of dead reckoning: the four wheel travels since the last loop, the forward and left arrows those add up to, the same pair turned to point along the robot's heading, and the old estimate with that pair added on the end.

### 2.11.3 The steps

#### Step 1: give the localizer somewhere to keep its answer

`update()` runs once a loop, and each run can only see how far the wheels have rolled since the last

one. Everything else has to be remembered between runs, which means fields.

No TODO asks for these, and nothing works without them. The class needs `x`, `y`, `headingOffset`, `lastWheels`, `lastRawHeading`, `lastNanos` and `state`, and a `POSES` factory to build a pose with. Three imports come with them: `Twist`, `PoseFactory` and `HeadingHold`. The answers page shows the whole block.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:compileDebugJavaWithJavac
```

It compiles with every TODO in the file still open, so this step stands on its own.

**If it didn't**, `error: cannot find symbol` and a line number is a field or an import that is not there yet. The line number is the line that wanted it.

### Step 2: one loop of dead reckoning

Five TODOs, one after the other, and they are one story. Read the wheels, the heading and the clock, TODO 1. On the very first loop there is nothing to compare against, so remember the three and return, TODO 2. Work out how far each wheel rolled since last time and add the four up two ways, TODO 3: all four together is forward, and one diagonal against the other is sideways. Turn that pair to point the way the robot is facing and add it to where you thought you were, TODO 4. Then publish the answer, TODO 5.

`state()` is where the answer leaves the class, and its one line is also marked TODO 5. The same number is used twice in this file. A small publish helper is worth writing, because `update`, `setPose` and `reset` all end by publishing.

Two details that are easy to miss. Subtract the two headings and wrap the difference with `HeadingHold.wrap`. And the heading to turn the pair by is the one halfway through the step, not the one at the start of it.

**You'll know it worked when the only test still failing is** `setPoseMovesTheEstimateWithoutMovingTheRobot`, which is step 3:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*MecanumEncoderLocalizerTest*'
```

**If it didn't**, the test that fails names the piece.

- Everything reads 0.0, starting with `countsStraightAhead` `expected:<24.0>` but was:`<0.0>`. `state()` still hands back `MotionState.zero()`.
- `countsStraightAhead` `expected:<24.0>` but was:`<96.0>`. Four wheels were added up and not divided by 4. Each wheel rolled the whole 24 in, and the robot went 24 in, not 96 in.
- `left` is `+y` when facing `+x` `expected:<12.0>` but was:`<0.0>`. The sideways sum is adding up the wrong diagonal. `-fl + fr - bl + br` is the pattern for turning, and turning is what it measured.
- `left` is `+y` when facing `+x` `expected:<12.0>` but was:`<-12.0>`. The two signs in the rotation are swapped, so left came out as right.
- `addsUpOverManyLoops` `expected:<24.0>` but was:`<1212.0>`, and nothing else fails. The wheel readings went in where the differences belong. One loop of 24 in looks the same either way, and 100 loops of 0.24 in does not. That is what that test is for.
- `NullPointerException: Cannot load from double array because "this.lastWheels" is null`. TODO 2's first loop is missing. There is nothing to subtract from yet.

### Step 3: start counting from somewhere else

TODO 6 is `setPose`: somebody knows where the robot really is and says so. TODO 7 is `reset`, which is back to the origin.

The IMU's heading cannot be moved. So `setPose` remembers the difference between the heading it

was handed and the heading the IMU is reading, and adds that from then on. That is what `headingOffset` is for.

**You'll know it worked when the whole class passes:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*MecanumEncoderLocalizerTest*'
```

**If it didn't**, `setPoseMovesTheEstimateWithoutMovingTheRobot` expected:<72.0> but was:<0.0> means `setPose` is still blank.

---

### Three corners no test here reaches

Each of these was tried on its own, with the simpler version written in, and the whole suite passed every time.

`HeadingHold.wrap` left off the heading difference. It matters when the robot turns through the back of the circle. The raw heading jumps from just under half a turn to just past minus half a turn, and a plain subtraction reads that step as nearly a full turn the other way.

The heading taken at the start of the step instead of halfway through. It matters while driving and turning at the same time, which no test here does.

`reset()` left blank. Nothing watches it at all. It is `setPose` to the origin, and it forgets `lastWheels` as well, so the next loop starts fresh.

---

### Step 4: run it beside the robot's own

TODO 1 in `L8CompareLocalizersOpMode.java` adds your localizer as a shadow, under the name `driveWheelEncoders`. Both localizers count encoders, so `encoders` would not tell them apart. What differs is which wheels: yours reads the four that push, and the Pinpoint reads two that only measure. TODO 2 drives, with the same call L6 ended on.

A shadow is read and logged, never driven. The test says so, and so does the arithmetic: nothing in your localizer writes to a motor.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l8*'
```

**If it didn't:**

- the shadow localizer is logged: `[Localizer/pinPoint/x_in, Localizer/pinPoint/y_in, ...]`. TODO 1. That list is every name the lesson did publish, and yours is not among them.
- expected:<[]> but was:<[L8CompareLocalizersOpMode turned 0.0]>. TODO 2. The sticks reach nothing.

---

### Step 5: drive it, then measure it

Send the code to the robot. Run `L8 Compare Localizers` and press PLAY. Drive it a long way in a straight line, stop, and read both estimates: `Localizer/driveWheelEncoders/x_in` and `Localizer/pinPoint/x_in`. Then put the tape measure on the floor and find out what really happened.

**You'll know it worked when** both numbers moved, and they do not agree with each other.

Then make the wheels slip. Hold the robot still and push a stick. Your estimate walks off across the field while the robot sits in your hands, and nothing in the arithmetic can tell. The test does the same thing on a laptop: one wheel rolling 24 in on its own moves the estimate 6 in, a quarter of a lie shared four ways.

**If it didn't**, and one of the two never moves, that localizer is not being updated. Check step 4 for yours.

---

### What has not been done here

Step 5 has not been performed on a robot for this page. The test numbers in it are measured; the floor part is written from what the code does.

---

## 2.11.4 What you just did

Dead reckoning believes the wheels. It has no other source, so a wheel that spins without carrying the robot is a wheel that moves the estimate. The Pinpoint's two wheels only measure and never push, which is why they slip less, and that is the whole argument for having them.

Every localizer is a guess that drifts. Yours drifts faster, and knowing by how much is worth more than not knowing. That is what publishing both under one prefix is for: two series on one graph, and a tape measure to settle them.

## 2.11.5 Where next

- [task.19](#) lets go of the sticks. The follower drives 24 inches on its own, using the pose you just learned not to trust.

## 2.12 L9: the first autonomous

### 2.12.1 What you will have when this is done

The robot drives 24 inches forward on its own and stops there. Nobody touches a stick. This is the first lesson where the follower does the driving, and it is three lines long.

### 2.12.2 Before you start

- [L6](#) is finished. The follower has to be holding a real drivetrain to drive anything.
- About 1 m of clear floor in front of the robot, and a tape measure.
- [L8](#) comes before this one in the guide, and nothing here needs it. The follower uses the robot's own localizer, not yours.

You write three lines in one file, `L9Drive24OpMode.java`.

### 2.12.3 The steps

#### Step 1: say where to go

An autonomous has no `loop` full of sticks. It has a `routine`, and the scheduler runs it. Your job is to hand back a command that says what to do.

The command is a sequence of one step: follow a straight line from `start` to `end`, holding the heading the robot starts at. Both poses are already declared above, 24 inches apart along `y`. The robot faces `+y`, so that straight line is straight forward.

`startPose()` is already written, and it matters. The follower has no idea where the robot is until something tells it, and everything after that is measured from there.

**You'll know it worked when this prints `BUILD SUCCESSFUL`:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.19*'
```

**If it didn't**, open the report gradle names.

- the robot never started following a path. The routine still hands back

`Command.NOOP`, which is the command that does nothing.

- ends 24 inches further along `y`. The line runs from end to start. The robot is standing at that path's finish already, so the command is done the moment it starts and nothing moves.
- and `still faces +y.constant(90)` does not mean 90 degrees. The number form is radians. Pass a pose instead, `constant(start)`, and it takes the heading from that pose.

---

### Why `.constant()` and not `.linear()`

`.constant()` holds one heading for the whole line, which is what this path wants: the heading is the same at both ends, so there is nothing to interpolate.

`.linear()` interpolates between two headings, and on Pedro 3.0.1 it does it backwards on a line path, reported in its issues #176 and #181. Nothing in this lesson can show that, because both ends hold the same heading.

---

### Step 2: drive it on the floor

Put the robot on the floor with a piece of tape at its front edge. Send the code. Choose `L9 Drive 24` from the autonomous list, and press PLAY.

**You'll know it worked when** the robot drives forward, slows down, and stops. Measure from the tape to the front edge: it should be close to 24 inches.

**If it didn't**, and it overshoot or stopped short, that is the follower's tuning, not your three lines. `task.autotune` is where the numbers behind it come from.

**If it didn't**, and it curved instead of going straight, check `task.directions` first. A backwards wheel curves every drive, with the sticks or without them.

The driver station shows the pose, the speed and the loop rate while it runs, because `loop` asks it to. Watch the `y` number climb by 24 while `x` stays put.

---

### What has not been done here

Step 2 has not been performed on a robot for this page. The 24 inches and the two poses are read from the lesson and from its test, which drives the same path in the simulator.

---

## 2.12.4 What you just did

The follower is a library, and this lesson uses it the way a team uses one: you say where to go and it works out the powers. You did not write the path algorithm and you do not need to be able to. What you did write is everything the follower needs to know about this robot, back in L6.

`sequential` takes a list, and this one has a single item in it. L10 puts a second item after it, and that is the whole of what an autonomous routine is: a list of things to do in order.

## 2.12.5 Where next

- `task.l10` adds a second leg, and the robot slides sideways along it without turning.

## 2.13 L10: two legs, forward then sideways

### 2.13.1 What you will have when this is done

The robot drives forward, then slides 24 inches to its right without turning, and holds where it ends up. Five lines, and the first autonomous that does more than one thing.

---

## 2.13.2 Before you start

- L9 is finished, so one leg works and the follower knows where the robot starts.
- About 1 m by 2 m of clear floor, and a tape measure.

You write five lines in one file, `L10ForwardThenStrafeOpMode.java`. Three poses are declared above it: `start`, `corner` and `end`. Read their headings before you write anything. All three are the same.

## 2.13.3 The steps

### Step 1: two legs and a hold

`sequential` takes a list of commands and runs them in order. L9's list had one item. This one has three.

TODO 1 is the first leg, `start` to `corner`. TODO 2 is the second leg, `corner` to `end`. Both hold the heading the robot starts at, so it never turns. The second leg moves it sideways instead, which is what mecanum wheels are for. TODO 3 is `hold(follower, end)`, which keeps the robot where it finished.

The robot faces +y the whole way. So the first leg is straight forward, and the second one is a pure strafe to its right.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l10*'
```

If it didn't, open the report gradle names.

- the robot never started following a path. The routine still hands back `Command.NOOP`.
- never turned, so it still faces +y. The heading changed somewhere. Pass a pose to `constant()` and it takes the heading from that pose, which is what you want here.
- strafed 24 inches in x. The first leg went straight to `end` instead of stopping at `corner`. The robot drives one diagonal instead of two legs, and arrives somewhere between them.

---

### `constant()` counts in radians

`constant(90)` does not mean 90 degrees. The number form is radians, and 90 radians is about 14 full turns; the path settles at 116.6 degrees once that wraps around. Measured on Pedro 3.0.1.

The poses use `POSES.of(x, y, degrees)` because a pose factory converts for you. `constant()` does not. So pass it a pose, `constant(start)`, or convert first with `Math.toRadians(90)`.

---

### Step 2: drive it on the floor

Put the robot down with tape at its front edge. Send the code. Choose `L10 Forward Then Strafe` and press `PLAY`.

**You'll know it worked when** the robot drives straight forward, stops, then slides sideways to its right and stops again. Its nose points the same way the whole time. It should end up about 24 inches to the right of the line it drove up.

**If it didn't**, and it drove both legs but the corner was a sharp stop and restart rather than a curve, that is what two separate legs do. The follower finishes the first line before it starts the second.

**If it didn't**, and it slid left instead of right, check which way +x runs from where the robot is standing. Facing +y, the robot's right hand points at +x.

---

### What has not been done here

Step 2 has not been performed on a robot for this page. `hold` is not watched by anything either: the test passes with `TODO 3` left out. What it does on a floor is keep the wheels working against a push once the path is over.

---

## 2.13.4 What you just did

An autonomous routine is a list. That is all it is. Two legs and a hold here, and a real match autonomous is the same list with more items and a few of them moving something other than the wheels.

A leg carries a heading, held from one end of it to the other. Hold the same heading on both legs and the robot never turns; where it goes and which way it faces are two separate things. That is what mecanum wheels buy, and `L11` spends it.

## 2.13.5 Where next

- `task.l11` goes back to the sticks. A stick then means a direction on the field. It stops meaning a direction on the robot.

## 2.14 L11: make the sticks mean the field

### 2.14.1 What you will have when this is done

Push the stick away from you and the robot goes away from you. It does that whichever way the robot happens to be facing, so a driver never has to think about which end of the robot is the front.

### 2.14.2 Before you start

- `L6` is finished, because this uses the same three lines it ended on.
- `L5` is worth running again first. Turn the robot to face you, push the left stick away from you, and watch it drive at you. That is the problem this lesson fixes.
- About 2 m by 2 m of clear floor.

You write in two files. `LessonsDriveTrain.java` gets the rotation, `TODO 9`, and `L11FieldRelativeOpMode.java` calls it.

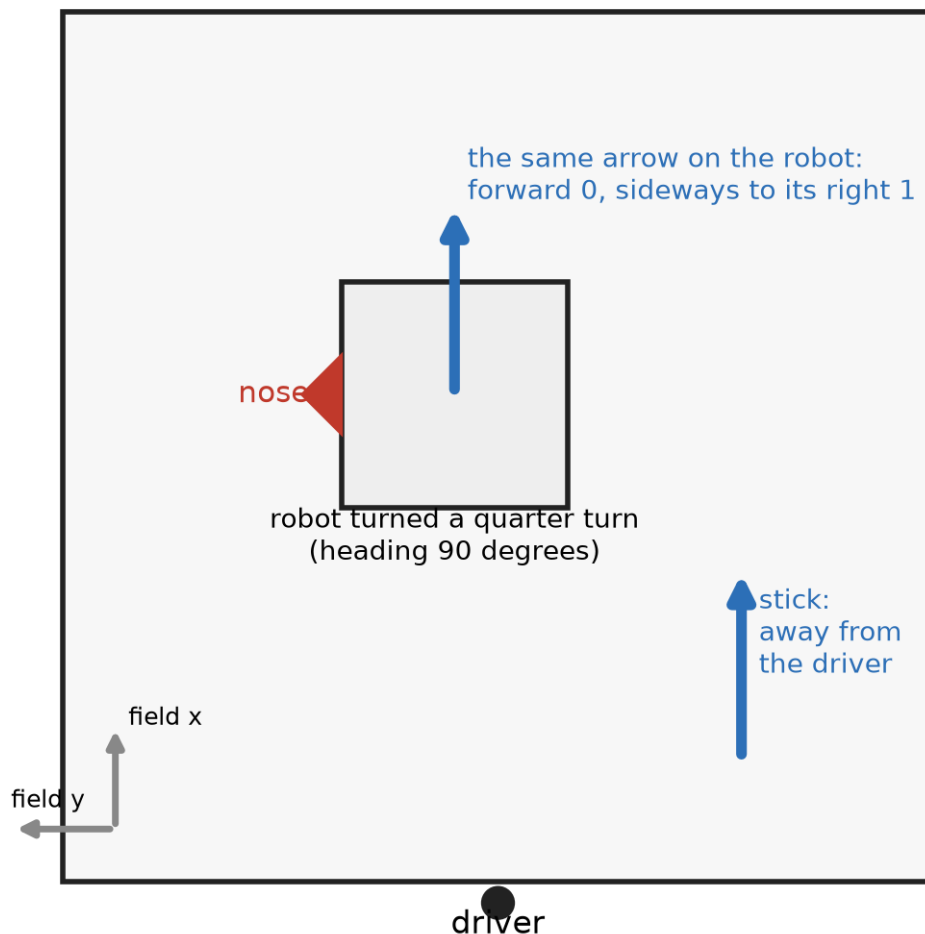


Figure 2.16. The field from above with the driver at the bottom, the robot turned a quarter turn, the stick's direction as an arrow on the field, and the same arrow drawn again on the robot showing what forward and sideways it turns into.

### 2.14.3 The steps

#### Step 1: rotate the driver's directions into the robot's

The driver's two numbers are about the field. The wheels only understand the robot: forward along its nose, and sideways across it. Turning one into the other is a rotation, and the angle is the robot's heading.

Write `fieldRelative` in `LessonsDriveTrain.java`, TODO 9. Take the cosine and the sine of the heading into their own variables. Then rotate the field pair **backwards** through the heading, which is what the two lines in the TODO do. Then mix, scale and command the wheels, which is the same three lines `sticks` ends on.

Backwards is the part worth a second look. The heading says how far the robot has turned away from the field. To ask "what does the field's +x look like from where the robot is standing?", you have to undo that turn.

#### Step 2: hand it the heading

The drivetrain cannot work out the heading for itself. It talks to motors; it has no idea where the robot is. The follower does, so the OpMode reads `follower.pose().heading()` and passes it in first.

Then the same three sticks as L5, negated the same way.

**You'll know it worked when** this prints BUILD SUCCESSFUL:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l11*'
```

**If it didn't**, open the report gradle names.

- expected:<1.0> but was:<0.0>, on the very first check. Nothing reached the wheels, so the OpMode's blank is still open.
- same stick, still moves away from the driver expected:<0.0> but was:<1.0>. The OpMode is still calling sticks, which is L5's door and knows nothing about the field.
- expected:<-1.0> but was:<1.0>. The rotation runs the wrong way. Either the two signs in fieldRelative are the other way round, or the heading was negated on the way in. Both undo the same thing twice.
- same stick, still moves away from the driver expected:<0.0> but was:<-0.3338674754196965>. The heading went in as degrees. Math.cos wants radians, and 90 radians is about 14 turns, so the answer is nonsense of exactly this shape.

### Step 3: drive it facing the wrong way

Send the code to the robot. Run L11 Field Relative and press PLAY. Turn the robot to face you, by pushing the right stick until its nose points at you. Then push the left stick away from you.

**You'll know it worked when** the robot drives away from you while facing you. Turn it a quarter turn and try again, and the stick still means the same direction on the field.

**If it didn't**, and the robot drives away from you at first but the directions slide as you drive around, the follower's pose is drifting. That is the localizer, not this lesson. [task.l8](#) is where the drift comes from.

---

### What has not been done here

Step 3 has not been performed on a robot for this page. The messages in step 2 are measured; the floor part is written from what the code does.

---

## 2.14.4 What you just did

Field relative is one rotation, and nothing else. There is no new way of driving here: the same mix, the same normalized, the same commanded wheels. All that changed is which pair of numbers gets handed to them.

It also depends on the follower knowing where the robot is pointing. A localizer that has drifted 20 degrees gives a driver a robot whose sticks are 20 degrees crooked. This is the first lesson where somebody else's mistake shows up in your hands.

## 2.14.5 Where next

- [task.l12](#) puts the robot's own directions back on a bumper, so a driver can have both.

## 2.15 L12: give the driver both

### 2.15.1 What you will have when this is done

The sticks mean the field, the way L11 left them, until the right bumper is held. While it is held they mean the robot. Let go and they mean the field again.

## 2.15.2 Before you start

- L11 is finished. Both doors this lesson chooses between are already written.
- About 2 m by 2 m of clear floor.

You write seven lines in one file, `L12RobotRelativeButtonOpMode.java`. The three sticks are already read for you above the blank.

## 2.15.3 The steps

### Step 1: choose a door with the bumper

Field relative is the better way to drive most of the time. It is not the better way to line up against something. Backing straight out of a corner, or nudging along a wall, is easier when forward means the robot's own forward.

So both are worth having, and an `if` is all it takes. Held bumper: `drivetrain.sticks`, the robot's own directions. Not held: `drivetrain.fieldRelative`, with the heading first. Publish which one is in force while you are there.

Held, not toggled. With a hold, letting go always puts the driver back where they started, and their thumb tells them which mode they are in. With a toggle they have to remember, in a match, with somebody shouting.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l12*'
```

If it didn't, open the report gradle names. The test pushes one stick with the robot turned a quarter turn, so field relative reads 0 at the wheels and robot relative reads 1.

- `robot relative while held expected:<1.0> but was:<0.0>`. The blank is still open. Nothing chooses, so nothing drives.
- `field relative by default expected:<0.0> but was:<1.0>`. The two branches are swapped. The bumper is the exception, and the `if` should read the bumper, not the lack of it.
- `and back again on release expected:<0.0> but was:<1.0>`. The bumper is flipping a remembered flag instead of being read fresh every loop. This is the toggle, and this is the test that catches it.

No test watches the log line. It is one call, and Panels is where it earns its place: a driver who says "it went crooked" and a graph that says which mode was in force settle the question between them.

### Step 2: drive both ways without stopping

Send the code to the robot. Run `L12 Robot Relative Button` and press PLAY. Drive field relative for a while. Then turn the robot so its nose points sideways, hold the right bumper, and push the stick away from you.

**You'll know it worked when** the robot drives along its own nose while you hold the bumper. Let go, and it goes back to driving away from you.

**If it didn't**, and holding the bumper changes nothing, look at your two branches. The one that calls `sticks` is the robot relative one.

---

### What has not been done here

Step 2 has not been performed on a robot for this page. The messages in step 1 are measured.

---

## 2.15.4 What you just did

This is the first lesson that is a driver's decision rather than a robot's. Nothing here is new arithmetic. Both doors existed already, and what you added is who chooses and when.

That is most of what a competition teleop turns out to be: a small number of ways to drive, and a gamepad laid out so a driver can reach them without looking.

## 2.15.5 Where next

- `task.l13` holds a heading for the driver. The robot stops drifting round when nobody is turning it.

## 2.16 L13: hold the heading the driver left

### 2.16.1 What you will have when this is done

Let go of the turn stick and the robot keeps pointing where it was left. Push the turn stick and the driver has it back at once. The nose stops wandering while the robot drives, which is what a driver notices first.

### 2.16.2 Before you start

- `L11` is finished, because this drives field relative through the same call.
- About 2 m by 2 m of clear floor.

You write six lines in one file, `L13HeadingHoldOpMode.java`. The class that does the holding, `HeadingHold`, is written for you.

### 2.16.3 The steps

#### Step 1: build the hold from the tuned controller

`bindings()` is where an `OpMode` sets up the things it will use. `TODO 1` builds a `HeadingHold` out of `Constants.foresightConfig.headingFeedback.get()`.

That is the same controller the autonomous steers with. Tuning the robot for autonomous therefore tunes the teleop hold as well, and there is no second set of numbers to keep in step.

#### Step 2: ask it for the turn power

`TODO 2` replaces the turn stick with what `HeadingHold` says. Hand it the follower and the turn stick, and it hands back the turn power to use. Then drive field relative with that in place of the stick, and publish the two heading values so Panels can draw the hold.

What the class does with it, worth reading before you call it: while the stick is pushed it forgets any target and passes the stick straight through. The loop the stick is released on becomes the target. After that it feeds the difference to the controller, which steers back.

Its own deadband is 0.05, so a stick that is nearly centred counts as released. That is L3's idea again, inside somebody else's class.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l13*'
```

If it didn't, open the report gradle names.

- while steering, the stick wins `expected:<-0.5>` but `was:<0.0>`. Both blanks are still open, so nothing turns at all.

- `NullPointerException`: Cannot invoke `... HeadingHold.turn ...` because `"this.heading"` is null. `TODO 2` is written and `TODO 1` is not. The loop is asking a hold that was never built.
- while steering, the stick wins expected:`<-0.5>` but was:`<0.5>`. The turn stick is not negated. Pushing the right stick right swings the nose right, which is clockwise, which is a negative turn.
- it steers back, with no numbers. The stick went straight to `fieldRelative` and `HeadingHold` was never asked. Steering still works, which is why the first two checks pass, and nothing holds.

No test watches the two log lines.

### Step 3: drive it and then push it

Send the code to the robot. Run `L13 Heading Hold` and press `PLAY`. Drive a long diagonal with the left stick and do not touch the turn stick.

**You'll know it worked when** the nose stays where it was. Then take hold of the robot and twist it a few degrees. It should come back on its own.

**If it didn't**, and the robot fights you and overshoots, that is the controller's tuning rather than this lesson. `task.autotune` is where those numbers come from.

**If it didn't**, and the nose slowly drifts anyway, the hold is working and the follower's idea of the heading is drifting with it. That is the localizer, and [task.l8](#) is where that comes from.

---

### What has not been done here

Step 3 has not been performed on a robot for this page. The messages in step 2 are measured.

---

## 2.16.4 What you just did

Three things now want to turn the robot, and they take turns in a fixed order. The driver's stick beats the hold. The hold beats doing nothing. Nothing here beats the follower, because the follower is not steering in teleop.

That order is the whole design. It is the same shape as `L6's` commanded wheels. The person with their hands on the gamepad wins, and everything else fills in when they let go.

## 2.16.5 Where next

- [task.l14](#) gives a button a pose, and the robot drives there by itself while the driver watches.

## 2.17 L14: give the driver a button that parks the robot

### 2.17.1 What you will have when this is done

Press `Y` and the robot drives itself to one spot on the field and stays there. Touch a stick and the driver has it back at once, from wherever the robot got to.

### 2.17.2 Before you start

- `L11` is finished, because the driver's half of this drives field relative.
- `L10` is finished, so a follower holding a pose is not new.
- [task.l8](#) is done, so the robot knows where it is. The target is a spot on the field, not a distance from the robot, and a robot with the wrong pose drives to the wrong place.

- About 3 m by 3 m of clear floor.

You write 21 lines in one file, `L14DriveToPoseOpMode.java`. The target is given: `POSES.of(120, 72, 90)`.

### 2.17.3 The steps

#### Step 1: put a command on the Y button

`bindings()` runs once, when the `OpMode` starts. TODO 1 ties `gamepad1.y` to an instant command that calls `follower.hold(TARGET)` and sets `drivingItself = true`.

An **instant** command does its work and finishes in the same breath. This one sets the follower's mode and ends, and the follower goes on holding long after the command is gone. The only thing that takes it back out of HOLD is a call to `manual()`. So the lesson has to remember by itself that it started something, which is what `drivingItself` is for.

Watched in the test harness: the command body runs inside `loopBefore()`, on the same loop the button goes down, before your own driving code. So `drivingItself` is already true the first time step 2 reads it.

#### Step 2: decide who gets the wheels

TODO 2 and TODO 3 are one `if` inside another. The three sticks and `driverWantsControl` are read for you above the blank, with a band of 0.1, so what is left is the choosing. There are three cases and they go in this order.

- The command is running and the sticks are near zero. Publish `AUTO`, hand the wheels back with `drivetrain.releaseCommandedWheels()`, and return. The follower is holding the pose, and the lesson's job is to keep out of its way.
- The command is running and the driver has moved a stick. Forget the command with `drivingItself = false` and call `follower.manual(0, 0, 0)`, which is the one call that leaves HOLD. The three zeros are thrown away; the next line commands the wheels.
- Nothing is running. Publish `DRIVER` and drive field relative, exactly as L11 does.

The `return` in the first case is the whole shape of it. Leave it out and the code walks straight on into the take-over lines, so pressing Y cancels its own command on the same loop.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l14*'
```

**If it didn't**, open the report gradle names. Two mistakes share each message, so both are worth reading.

- the command is driving, not the sticks expected:<HOLD> but was:<MANUAL>. Nothing put the follower in HOLD and left it there. Either TODO 1 is still open, so the button does nothing, or the `return` is missing and the loop cancels the hold the instant it starts.
- manual control returns expected:<MANUAL> but was:<HOLD>. The follower is holding and the driver cannot get it back. Either the outer `if` is never entered, because `drivingItself` is never set true, or it is entered and `follower.manual(0, 0, 0)` is not called.

Two of those are worth doing on purpose once. Take the `return` out and watch the first message appear, then put it back and watch it go.

#### Mistakes the test here cannot see

Leaving `releaseCommandedWheels()` out passes. Nothing is commanded yet in the test, so there is nothing to hand back. On the floor it is the mistake you notice most: the wheels stay at whatever

the driver last asked for, and the follower's answer never reaches them. Watched in the harness, sliding right and then pressing Y, the follower says HOLD for a whole second while the robot slides on from 68 in to 26 in and never turns toward the target.

Leaving out `drivingItself = false`, and swapping `fieldRelative` for `sticks`, both pass as well. So does dropping both `drive/mode` publishes. The mode is for Panels, and a driver who says "it took off on its own" and a graph that says which of the three cases was in force settle it between them.

---

### Step 3: drive it and then take it back

Send the code to the robot. Run `L14 Drive To Pose` and press PLAY. Drive somewhere with the left stick, let go, and press Y.

**You'll know it worked when** the robot drives off to the target on its own and sits there. Then push the left stick and it is yours again, without pressing anything else.

**If it didn't**, and the robot drives somewhere that is not the target, the pose is wrong rather than this lesson. [task.18](#) is where the pose comes from.

**If it didn't**, and it creeps up to the target and hunts back and forth, that is the follower's tuning. `task.autotune` is where those numbers come from.

---

### What has not been done here

Step 3 has not been performed on a robot for this page. Every message and number above was measured in the test harness, where the robot covers the 48 in and settles within 0.02 in of the target, pointing 90 deg, about 2 s after the press.

---

## 2.17.4 What you just did

The robot now has two drivers, and they never argue, because one line settles it. A stick past 0.1 wins. The wheels the sticks commanded stay commanded, and commanded wheels beat the follower, so the only way the follower can drive at all is for the lesson to stand back and say so.

That is [task.16](#)'s door used the other way round. L6 through L13 used it to take the wheels off the follower. This lesson uses it to give them back for a while, and it is how a real teleop runs a scoring move: a button starts it, the driver's thumb stops it.

## 2.17.5 Where next

- [task.15](#) puts every teleop lesson in one OpMode, which is the one a driver would take to a match.

## 2.18 L15: one OpMode a driver could take to a match

### 2.18.1 What you will have when this is done

One teleop with all of it in. The sticks are shaped. They drive field relative. They hold the heading they were left at. The right bumper means the robot's own directions. A points the robot at 45 deg while the driver drives. Y hands the robot to the follower until a stick moves.

### 2.18.2 Before you start

- [L13](#) is finished. It is the last of the pieces this one puts together.
- [task.18](#) is done, because this runs the encoder localizer beside the real one.
- About 3 m by 3 m of clear floor.

You write two lines. Everything else in `L15CombinedOpMode.java` is given, and every part of it was written in an earlier lesson.

### 2.18.3 The steps

#### Step 1: run the second localizer again

TODO 1 adds your `MecanumEncoderLocalizer` as a shadow, under the name `driveWheelEncoders`, exactly as [task.l8](#) did. It is two lines and they are the same two lines.

The name is the whole point of copying it rather than inventing one. Panels draws a series per name, so L8's plot and this one land on top of each other, and a run of L15 can be compared with a run of L8.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l15*'
```

If it didn't, one message covers every way of getting this wrong:

- the shadow localizer is still running, with no numbers. The test looks for one published key, `Localizer/driveWheelEncoders/x_in`. It cannot tell a missing localizer from a misspelled name, so check the spelling before you go looking for anything else. Watched, with the name typed as `encoders`: this same message, and nothing else in the suite noticed.

In Panels you should see six values: `x_in`, `y_in` and `heading_deg` under `Localizer/driveWheelEncoders/`, and the same three under `Localizer/pinPoint/`.

#### Step 2: read what you are about to drive

Nothing below is new, and that is the point of the lesson. Read `driveTheRobot()` before you run it, and find the lesson each part came from.

- Both drive sticks go through deadband and then squared, from [L3](#). The turn stick is dead-banded only. Squaring the turn as well makes a small correction almost nothing, and steering wants that hundredth.
- Y hands the robot to the follower, and any stick past the deadband takes it back, from [L14](#).
- The turn power comes from `HeadingHold`, from [L13](#). A held stick steers; letting go sets the target.
- A aims at 45 deg, which is `HeadingHold` again through a different door. It sets a target the driver did not leave, and translation stays with the driver.
- The right bumper chooses sticks or `fieldRelative`, from [L11](#) and [L12](#).

Four things want to turn the robot and they are settled in one fixed order: the driver's turn stick, then A's aim, then the held heading, then the follower. Each one only gets a say when the one above it is quiet.

**You'll know it worked when** you can name the lesson every line came from. There is no test for that, and it is the check that matters here.

#### Step 3: drive all four of them

Send the code to the robot. Run `L15 Combined` and press PLAY. Try them in this order: field relative, bumper held, A, Y.

**You'll know it worked when** each one does what its lesson did, and switching between them needs nothing but a thumb.

If it didn't, and one of them misbehaves on its own, go back to that lesson's page. This page adds no arithmetic, so a fault here is a fault there.

If it didn't, and A turns the robot but the driving stops while it turns, check that you are reading the given code and not your own. Aiming and driving happen in the same loop, on purpose.

---

**What has not been done here**

Step 3 has not been performed on a robot for this page. Watched in the test harness instead, with A pressed and the drive stick held: the robot swings from 0 deg to 48.7 deg in half a second, overshoots 45 deg by about 4 deg, and is inside a degree of it 1.25 s after the press, all while the drive stick keeps its own two powers near 0.5.

---

**2.18.4 What you just did**

You wrote two lines and got a competition teleop. That is what the last seven lessons were for: every part lives in one place, so putting them together is choosing an order rather than writing code.

The order is the design, and it is worth saying out loud. The driver beats everything. A button beats nothing at all. The follower gets what nobody else has asked for. A teleop built the other way round, where the robot decides and the driver argues with it, is the one that loses matches.

**2.18.5 Where next**

- `task.l16` stops asking the wheels for power and starts asking them for a speed.

**2.19 L16: ask the wheels for a speed****2.19.1 What you will have when this is done**

Full stick means 40 in/s, not “full power”. The robot goes that fast on a fresh battery. It goes that fast on a flat one too, because the code measures what the wheels are doing and makes up the difference.

Panels gains three numbers for every wheel. The speed you asked for, the speed the encoder sees, and the power in between.

**2.19.2 Before you start**

- `L6` is finished, because this commands the wheels the same way.
- `L3` is finished, because the sticks are still deadbanded.
- About 3 m of clear floor, and something gentle to drive up, like a folded mat.

You write in two files: three short blanks in `L16VelocityDriveOpMode.java`, and one 16-line blank in `LessonsDriveTrain.java`.

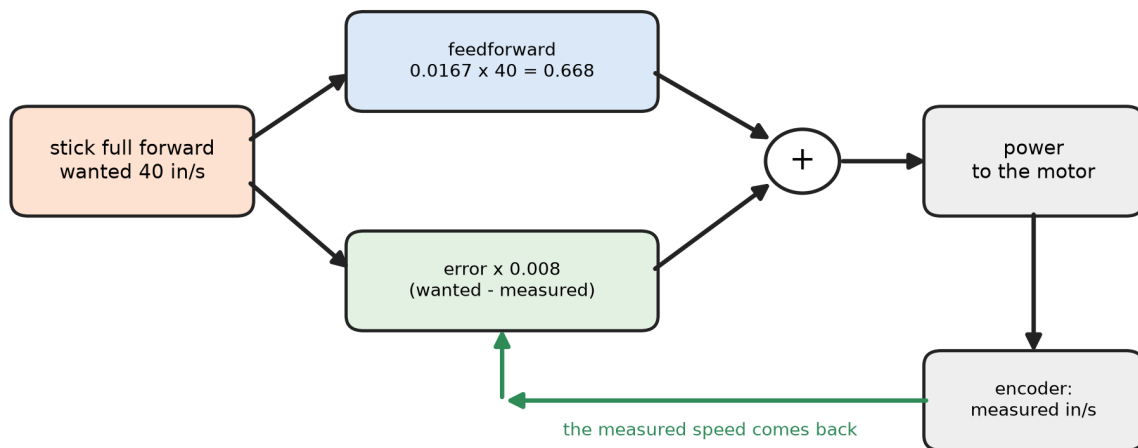


Figure 2.17. The loop drawn once round: the stick asking for 40 in/s, the feedforward guess at the power, the encoder's measured speed coming back, and the error being added in.

### 2.19.3 The steps

#### Step 1: three lines that mean inches, not power

TODO 1, 2 and 3 in the OpMode, one line each in the loop.

- TODO 1 reads the sticks as a **speed**. Full forward asks for `MAX_IPS`, which is 40 in/s. Full turn asks for `MAX_TURN_RADPS`, which is  $\pi$  rad/s, one half turn a second. Deadband each stick at 0.05 and keep the minus signs from L5.
- TODO 2 works out what each wheel has to do for the robot to move like that. `WheelTargets.forMecanum` does the arithmetic. The fourth thing it needs is `Constants.turnRadiusInches`, how far a wheel sits from the middle.
- TODO 3 hands those four speeds to `drivetrain.setCommandedWheelSpeeds(...)`, which is what step 2 writes.

You'll know it worked when the code compiles:

```
./gradlew :TeamCode:compileDebugJavaWithJavac
```

The tests cannot pass yet. Watched, with these three lines right and step 2 still blank: every L16 test fails with `NullPointerException: Cannot invoke "java.lang.Double.doubleValue()" because the return value of "java.util.Map.get(Object)" is null`. That is the test asking the log for a number nobody published. It is not a crash on the robot, and step 2 is the fix.

#### Step 2: turn a wanted speed into a power

TODO 10 in `LessonsDriveTrain.java` writes `setCommandedWheelSpeeds`, and it is the lesson. Every other door into this drivetrain takes a power. This one takes a speed and gets it, by adding two answers together for each wheel.

- **Feedforward** is a guess made before anything is measured: `Constants.powerPerInchPerSecond * wanted[i]`. Power and speed are roughly proportional, so the guess gets most of the way there on the first loop.
- **Feedback** is the correction: `VELOCITY_KP * (wanted[i] - measured[i])`, where `measured` comes from `measuredSpeeds.all()`. It is the only part that knows what is really happening.

Put `clampToPower(feedforward + feedback)` in each wheel's slot, command the four wheels, and call `publishWheelSpeeds(wanted, measured, powers)`.

Neither half works alone. The guess is always a little off, and the correction has to build up an error before it does anything, so it lags behind. Together they are how nearly every speed controller works, on a robot or anywhere else.

You'll know it worked when this prints `BUILD SUCCESSFUL`:

```
./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.116*'
BUILD SUCCESSFUL
```

If it didn't, open the report gradle names. The test asks for 40 in/s with the wheels at rest, so the numbers below are all about one wheel wanting 40 in/s and getting nothing yet.

- full stick asks for a speed, in inches per second `expected:<40.0>` but was:`<0.0>`. `TODO 1` is still three zeros.
- `expected:<40.0>` but was:`<1.0>`. The stick is being handed on as it comes, without `MAX_IPS`. A full stick is 1, and 1 in/s is a slow walk for an ant.
- `expected:<40.0>` but was:`<-40.0>`, and turning counter-clockwise drives the left side backwards. The minus signs are missing from `TODO 1`.
- and every wheel must travel at it `expected:<40.0>` but was:`<0.0>`. `TODO 2` is still new `double[4]`, so every wheel is asked for nothing.
- `expected:<0.9878391425450086>` but was:`<0.6678391425450086>`. The feedback is missing. The difference is exactly `0.008 * 40`, which is the correction that is not being added.
- `expected:<0.9878391425450086>` but was:`<0.32>`, and the power is the feedforward alone `expected:<0.6678391425450086>` but was:`<0.0>`. The feedforward is missing. A wheel already at speed then gets no power at all.
- `expected:<0.9878391425450086>` but was:`<0.34783914254500864>`. The error is the wrong way round. `measured - wanted` subtracts the correction instead of adding it.
- `measured` speed matches the command `expected:<40.0>` but was:`<0.0>`. `measuredSpeeds.all()` is never read, so `measured` is four zeros and the log says the wheels are stopped while they spin.
- turning counter-clockwise drives the left side backwards, on its own. Two speeds went into each other's slots.

---

### Mistakes the tests here cannot see

Each test pushes one stick, so anything that only goes wrong when two are mixed gets through. All four of these pass the whole suite, and all four were measured in the harness with the turn stick held for 1 s.

- **Strafe and turn handed to forMecanum the wrong way round.** Right answer: the robot turns 120.9 deg in that second. Wrong: it slides 2.9 in sideways and never turns at all.
- **The turn radius left out, 1 in in place of 8 in.** The robot turns the right way, 15.1 deg instead of 120.9 deg. Eight times too slow, and the sticks feel dead.
- **setCommandedWheels never called.** Nothing moves, and the log looks perfect: 40 in/s asked for and 0.988 power, with the robot standing still. Worth knowing as a shape, not just a bug. A number in a log is what the code decided, not what happened.
- **clampToPower left out.** Asking for 400 in/s publishes a power of 9.878 instead of 1.000, and the robot drives exactly as far either way, because `CachedMotor` chops it to 1 before the motor sees it. So the clamp is there to keep the log honest.

The deadband can also be left out and everything passes, because every stick the tests push is already outside it.

---

### Step 3: drive it up something

Send the code to the robot. Run `L16 Velocity Drive` and press PLAY. Drive it on the flat first, then straight up the mat with the stick held at the same place.

**You'll know it worked when** the robot slows a little at the foot of the slope and then picks back up. Watch `wheel/frontLeft/target_ips` and `wheel/frontLeft/actual_ips` in Panels. The measured one dips, then climbs back to the one you asked for, and the power between them rises to do it.

**If it didn't**, and the robot is slower than the number says everywhere, `powerPerInchPerSecond` is the guess and it is measured on a robot. `task.autotune` is where it comes from.

**If it didn't**, and turning is the wrong speed while driving straight is right, look at `Constants.turnRadiusInches`. It is still 8 in. Your robot may not be. `task.l17` measures it.

---

### What has not been done here

Step 3 has not been performed on a robot for this page, and it is the step that shows what the lesson is for. Every message and number above was measured in the test harness, where the fake encoders read zero speed unless a test sets them. So the correction never shrinks there the way it does on carpet.

---

## 2.19.4 What you just did

A power is an instruction and a speed is a result. Every lesson up to here sent instructions and hoped. This one states the result it wants and closes the loop, which is the difference between code that drives a robot and code that pushes at it.

Both numbers it leans on are measurements of your robot, not constants of nature: `powerPerInchPerSecond` and `turnRadiusInches`. Nothing in this lesson can tell that they are wrong, which is the whole reason L17 exists.

## 2.19.5 Where next

- `task.l17` measures the two numbers this lesson trusted: ticks per inch, and the turn radius.

## 2.20 L17: measure the two numbers L16 trusted

### 2.20.1 What you will have when this is done

Two numbers in `Constants.java` that came off your robot instead of somebody else's: how many encoder ticks make an inch, and how far a wheel sits from the middle. You get them by pushing and spinning the robot by hand while it watches itself.

### 2.20.2 Before you start

- L16 is finished. It is the lesson that uses both numbers.
- The Pinpoint is mounted and tuned, because this measurement believes it. `task.pinpoint` is where the mounting comes from.
- About 1.5 m of clear straight floor for the push, and room to spin the robot on the spot.

You write four lines, all in `LessonsDriveTrain.java`. Both `OpModes` are given.

## 2.20.3 The steps

### Step 1: open a door to the encoders

TODO 11 writes `wheelTicks()`. Four lines, each one putting what a motor's encoder has counted into that wheel's slot: `ticks[FL] = hardware.frontLeft.getCurrentPosition();` and three more.

Nothing here is new. It is the same shape as `writeWheels()` from L2b, pointing the other way: that one sends four numbers out to the motors, this one brings four numbers back.

**You'll know it worked when this prints BUILD SUCCESSFUL:**

```
./gradlew :TeamCode:testDebugUnitTest --tests
'*LessonsDriveTrainTest.theTicksDoor*'
```

**If it didn't**, the test puts 10, 20, 30 and 40 on the four encoders. So the numbers in the message name the wheel they came from.

- front left, front right, back left, back right: arrays first differed at element [0]; expected:<10> but was:<0>. The blank is still open, so every measurement would say the robot never moved.
- expected:<10> but was:<20>. Two motors went into each other's slots. 20 is the front right, and it is in the front left's place.
- expected:<10> but was:<0>, with the door written. `getVelocity()` reads how fast a wheel is turning, and a wheel nobody is pushing is turning at 0. `getCurrentPosition()` is the one that counts.

### Step 2: push the robot and measure an inch

Send the code to the robot. Run L17a Measure ticks per inch and press PLAY. Push the robot slowly straight forward, at least 36 in, keeping it pointing the same way.

The robot is keeping two accounts of the same journey. The Pinpoint says how far it went in inches, because its own wheels measure the floor. The drive encoders say how many ticks they counted. One divided by the other is the number, and neither account knows what the other said.

Push slowly. A wheel that slips travels without turning. The encoders then count less than the floor did, and the answer comes out short. Do it three times. Three answers a tick or two apart are a measurement. Three that disagree mean something slipped.

**You'll know it worked when** the Driver Station shows `ticksPerInch` with a number after it. Put that number in `Constants.ticksPerInch`, which is 45.0 until you do, and run it again to check.

**If it didn't**, and it says `Encoders counted 0 ticks` while the inches climb, step 1 is not finished on the robot you just sent to.

**If it didn't**, and the wheels drag as you push, they are braking when they should coast. The `OpMode` asks for that itself, and the two tests below say whether it works.

### Step 3: spin the robot and measure the radius

Run L17b Measure turn radius and spin the robot on the spot, at least two full turns, keeping it in roughly the same place. **Run L17a first:** this one turns ticks into inches with the number you just measured, so a wrong one there is a wrong one here.

A wheel  $r$  inches from the middle travels  $r$  inches for every radian the robot turns. So the radius is wheel inches divided by radians, which is the same trick as step 2 with a different pair of accounts.

**You'll know it worked when** the Driver Station shows a turn radius, and a tape measure agrees with it. Measure the diagonal between two opposite wheels and halve it. That is the same distance.

**If it didn't**, and the number is about half what the tape says, check that you spun the robot rather than swinging it round the room. The Pinpoint counts the turning, and driving in a circle adds

turning the wheels never made.

---

### The wheels have to roll, and two tests say so

Both OpModes force the wheels to coast in `start()` and put braking back in `stop()`. It is given code, and it has been wrong twice, so both mistakes are watched.

- Put the coast back inside the loop instead of at the end and the coast lasts one loop: `L17aMeasureTicksPerInchOpMode` still coasts five loops in. That is how the lesson used to be written, and the wheels braked while the student was still pushing.
- Leave braking un-restored at the end and two messages appear: `L17aMeasureTicksPerInchOpMode` brakes again afterwards, and braking is back although no loop ran. The second one is a run where STOP came before the first loop, which used to leave the next OpMode coasting.

Both were measured by putting each mistake back.

---

### What has not been done here

Steps 2 and 3 have not been performed on a robot for this page, and no test can stand in for them: pushing a robot by hand is not something the simulator does. The arithmetic underneath is tested on its own, in `CalibrationTest`, and step 1's messages are measured.

---

## 2.20.4 What you just did

You turned two constants into measurements. `Constants.java` was handed to you with 45.0 and 8.0 in it, and those are somebody's robot, not yours. Nothing in L16 could tell they were wrong, because code cannot check a number it has no other way of knowing.

The method is worth more than either number. Have the robot measure the same thing two ways, with two instruments that do not talk to each other, and compare. That is how the wheel size, the gearing, and the tuning numbers are all found, and it is what `task.autotune` does with the rest of them.

## 2.20.5 Where next

- `task.autotune` measures the numbers that need the robot driving itself rather than being pushed.

That page is not written yet.

## 2.21 Pedro routes: a cheat sheet

Every way to say where the robot should go, and which ones to avoid. Measured against `com.pedropathing:core:3.0.1`, the version this project pins, on 2026-09-29. Where the documentation and the jar disagree, the jar is what is written here.

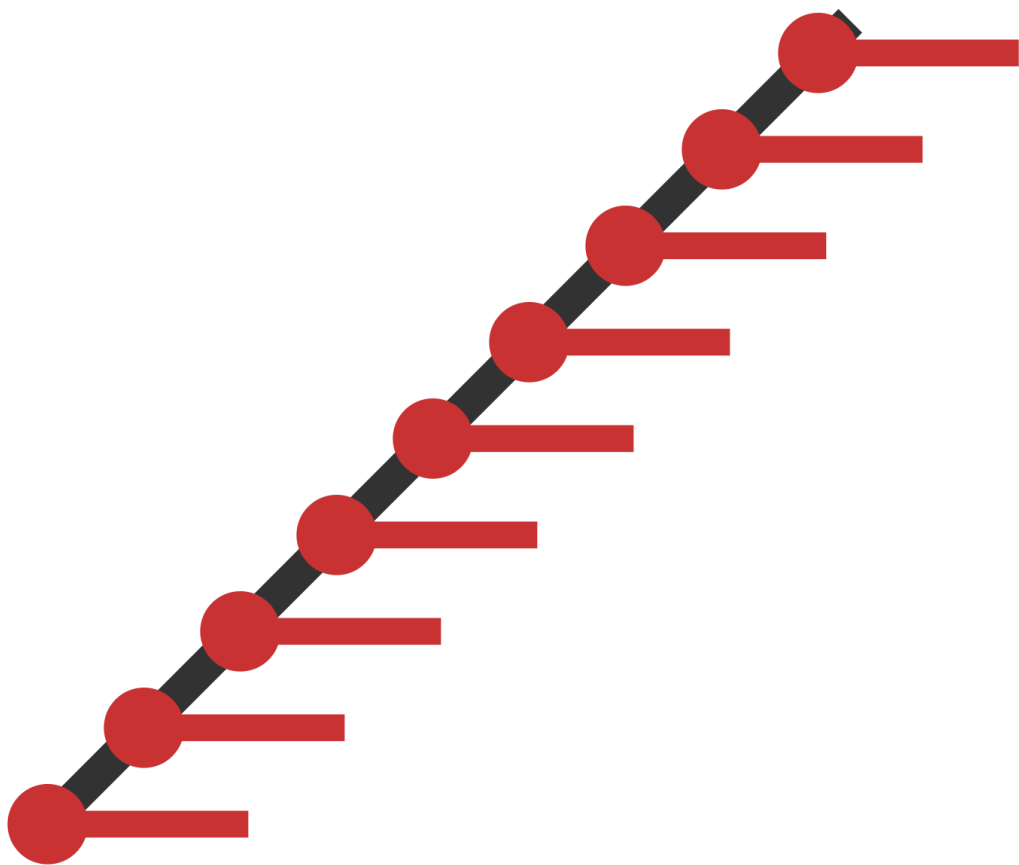
In the pictures the black line is where the robot drives and each red pin is its nose, sampled every eighth of the way along. Both come from `Path.get(t)` and `Path.heading(t)`, so they are the library's own output rather than a drawing of it.

### 2.21.1 The shapes

A route is a shape plus a heading rule. There are four shapes.

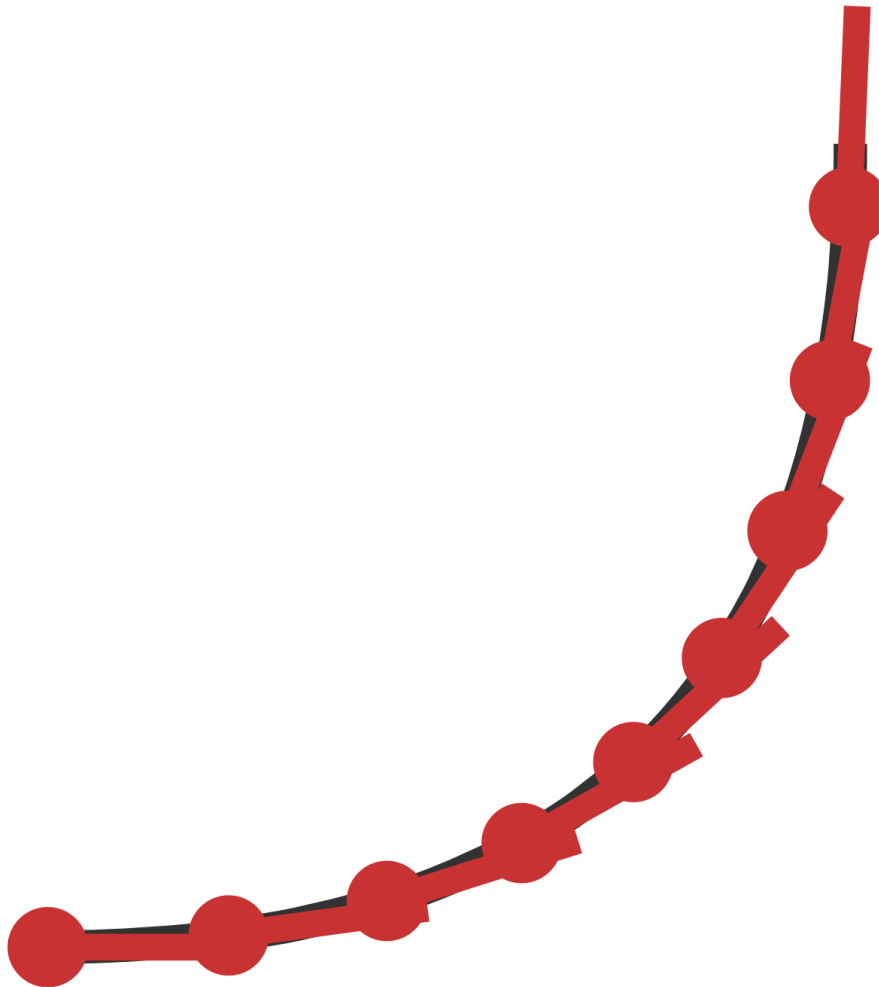
| Shape                                            | What it is                                                                                                            |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>Paths.line(start, end)</code>              | A straight line between two poses.                                                                                    |
| <code>Paths.curve(start, control..., end)</code> | A Bézier curve. The first and last poses are the ends; every pose between them pulls the curve without being touched. |
| <code>Paths.through(poses...)</code>             | A curve that passes through every pose given.                                                                         |
| <code>Paths.path(paths...)</code>                | Several paths run as one, in order.                                                                                   |

### line



Two poses, nothing between them. `line(start, end).constant(start)` is the plainest route there is and the one to reach for first.

curve



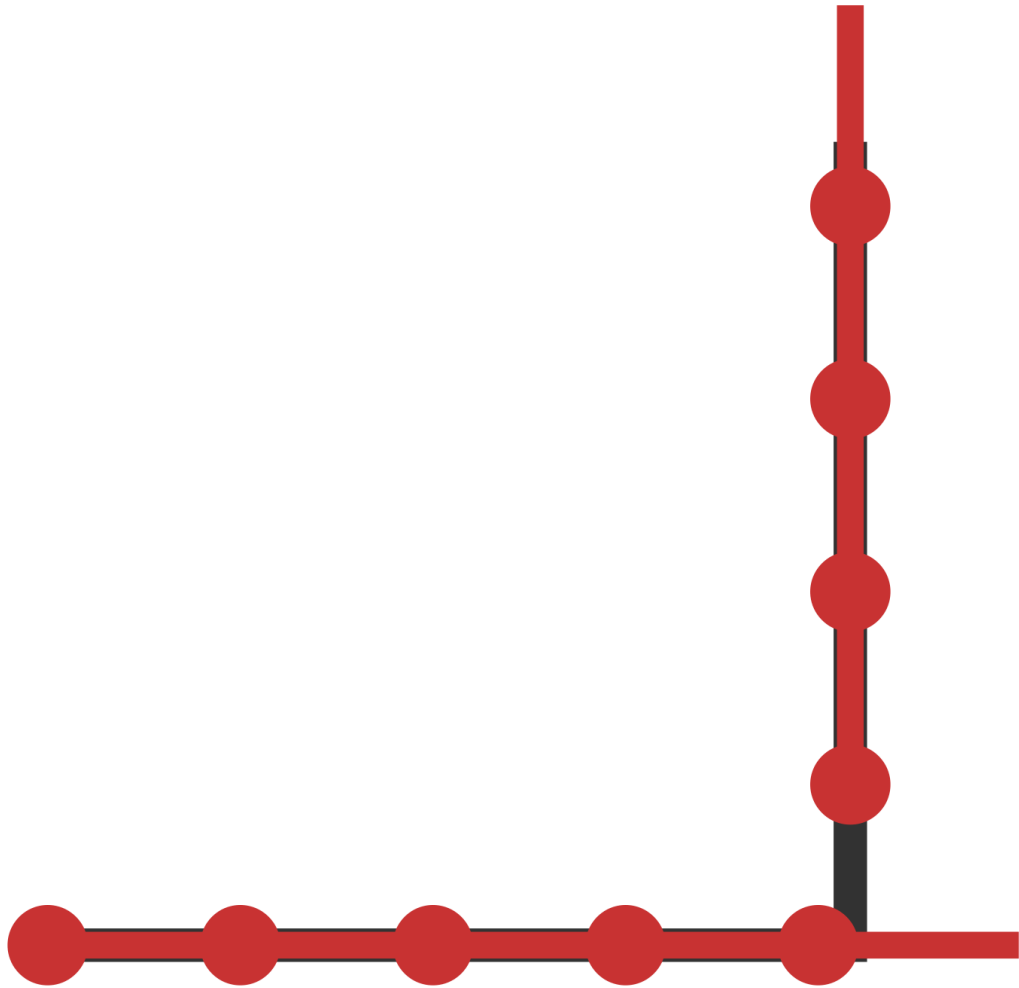
The control pose is a magnet, not a waypoint: the curve bends toward it and never reaches it. Its heading is ignored.

through



Every pose is on the route. The cost is at the ends: with `tangent()` the nose starts at -18.4 degrees and finishes at 108.4 degrees on the same three poses where `curve` runs 0 to 90, because the spline arrives at an angle. Check the end heading rather than assuming it.

path



Two legs of a route, joined. The nose snaps at the corner rather than easing round it: measured 0 degrees for the first leg and 90 for the second, with nothing in between.

### 2.21.2 The heading rules

| Rule                            | What it does                                        | Use it?             |
|---------------------------------|-----------------------------------------------------|---------------------|
| <code>constant (pose)</code>    | Holds that pose's heading the whole way.            | Yes.                |
| <code>constant (radians)</code> | The same, from a number.                            | Careful: radians.   |
| <code>tangent ()</code>         | Nose follows the direction of travel.               | Yes.                |
| <code>reverseTangent ()</code>  | Tangent plus 180 degrees, for driving backwards.    | Yes.                |
| <code>facingPoint (pose)</code> | Nose always aimed at one spot on the field.         | Yes, with one edge. |
| <code>linear (a, b)</code>      | Sweeps from one heading to another along the route. | Not on a line.      |

|                                        |                                                |                |
|----------------------------------------|------------------------------------------------|----------------|
| <code>longLinear(a, b)</code>          | The same, taking the long way round.           | No.            |
| <code>Interpolator.pieceswise()</code> | A different rule on each stretch of the route. | Untested here. |

**There is no default**

The documentation says `tangent()` “is the default interpolation type when you create a path”. It is not. A path with no heading rule throws as soon as anything asks for a heading:

```
java.lang.UnsupportedOperationException: No heading interpolator set.
```

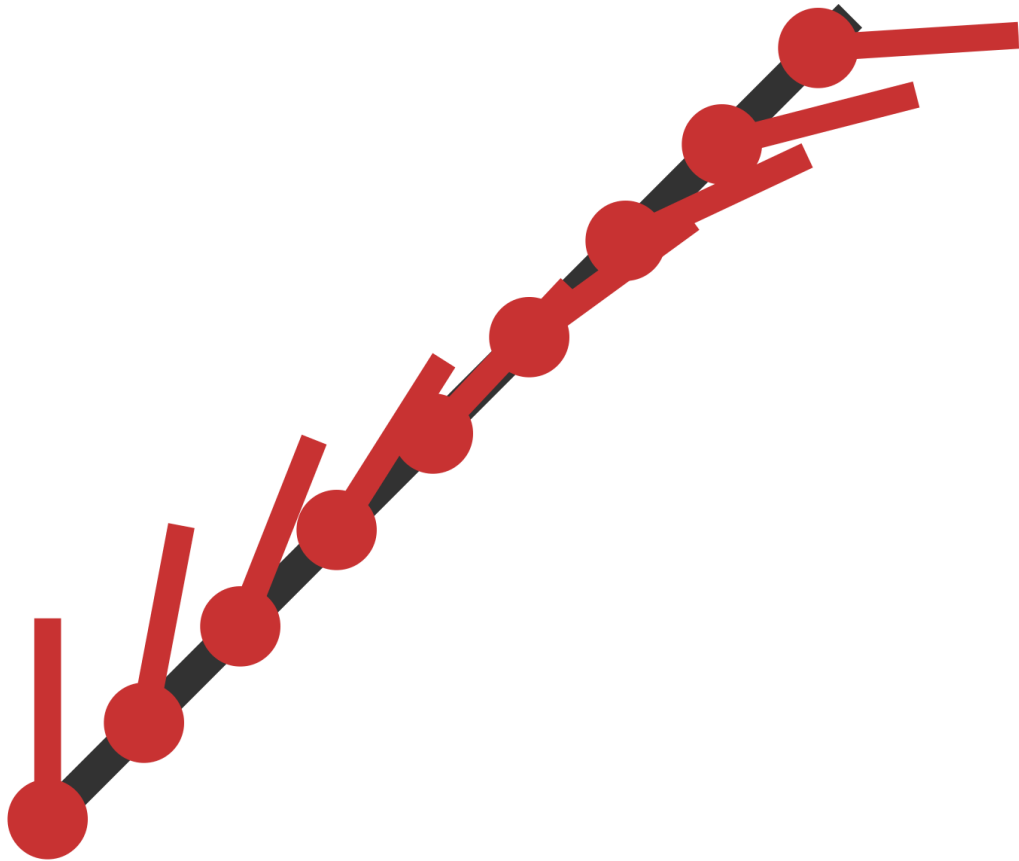
So every route needs a rule. It is not optional.

**The number form is radians**

`constant(90)` is not 90 degrees. It is 90 radians, which wraps to 116.6 degrees. `constant(0)` is safe only because zero is the same in both.

Two ways to avoid it. Pass a pose, `constant(start)`, and the heading comes from the pose. Or convert, `constant(Math.toRadians(90))`. A pose built by `PoseFactory.degrees()` converts for you; `constant` does not.

### 2.21.3 Do not use `linear()` on a line



Asked to sweep from 0 to 90 degrees along a line, 3.0.1 sweeps from 90 to 0. Measured:

|                                                  |                  |                  |                    |                    |                  |                       |
|--------------------------------------------------|------------------|------------------|--------------------|--------------------|------------------|-----------------------|
| <code>line(s, e).linear(0, toRadians(90))</code> | <code>t=0</code> | 90.0             | <code>t=0.5</code> | 45.0               | <code>t=1</code> | 0.0                   |
| <code>line(s, e).linear(e, s)</code>             | swapped          | <code>t=0</code> | 0.0                | <code>t=0.5</code> | 45.0             | <code>t=1</code> 90.0 |

The end pose is wrong too: `endPose().heading()` reports the start heading.

This is [issue 176](#), with [180](#) and [181](#) reporting the same thing. The cause given there is `Curve.pathCompletion()` returning the fraction of the path remaining rather than the fraction completed. All three are closed, and 3.0.1 still does it.

The workaround in those issues is to swap the arguments, and it works, and it reads as a mistake to whoever finds it next. It also fails for a 180 degree turn, where the two ways round are the same distance.

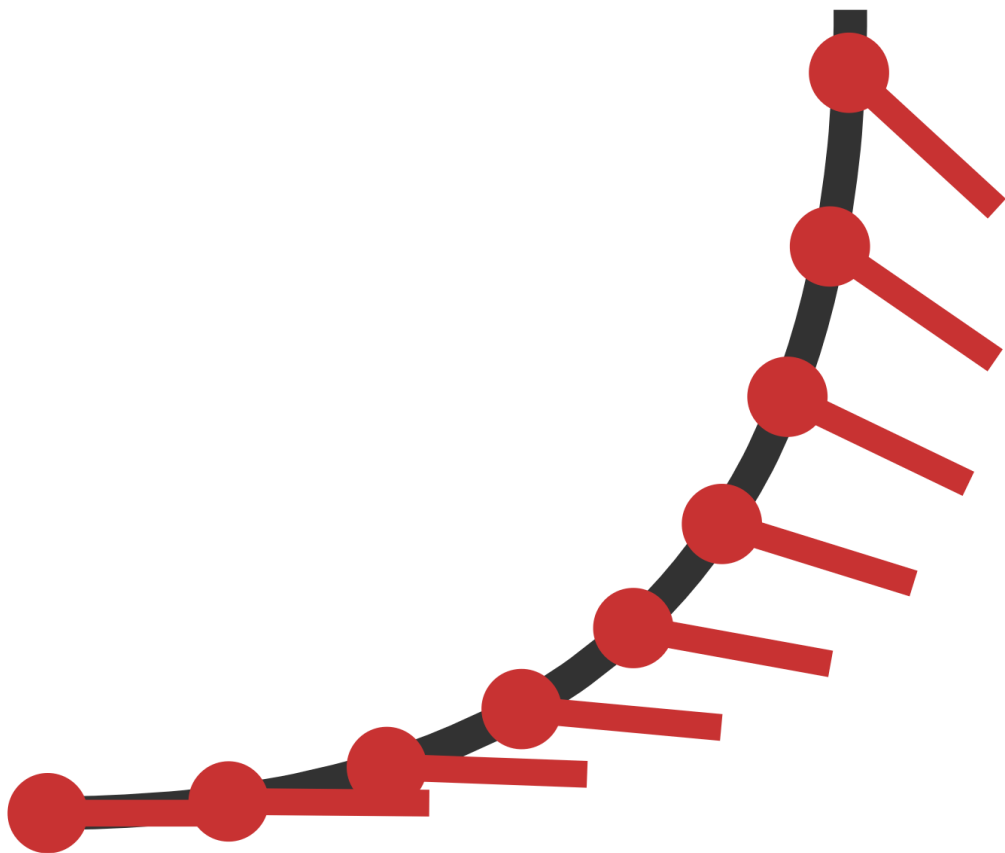
**On a curve, `linear()` is correct.** Measured on the same three poses: `curve(s, m, e).linear(0, toRadians(90))` runs 0, 24.6, 45, 90. So the bug is the line's, which is what [issue 176's](#) title says: line and compound paths.

Two ways round it on a straight leg. Hold one heading with `constant` and put the turn on its own leg. Or make the leg a shallow curve, where `linear` behaves.

#### 2.21.4 `longLinear()` is reversed the same way

`longLinear(0, toRadians(90))` runs 90, 157.5, 225, then 0 at the end. Neither end is what was asked for. It is undocumented as well: the documentation page lists `tangent`, `linear`, `constant`, `facingPoint`, `piecewise` and `custom`, and not this.

#### 2.21.5 `facingPoint()` has one edge



It works: the nose tracks the point as the robot moves. The edge is a route that starts *at* the point it faces, where there is no direction to face. Measured on a line leaving the point, the heading reads 0 at  $t=0$  and the correct -135 degrees from the next sample on, so the first instant is a discontinuity. [Issue 69](#) is about this interpolator; it is closed.

#### 2.21.6 What this page does not cover

- `Interpolator.piecewise()`, which nothing here has run.

- Custom interpolators.
- Deceleration, path and end constraints, and foresight, each of which has its own reference page and changes how a route is driven rather than what it is.
- Whether any of this behaves on a floor. Everything above is the library's own arithmetic, read out of the jar. When the simulator and the robot disagree, the robot is right.

## 2.22 Glossary

Every word here is one the task pages use, and each entry names the page that introduces it. Where the guide could have invented a friendlier word and did not, the reason is that the code, Pedro's documentation and the Driver Station all use this one.

### OpMode

One program the Driver Station can run. Each lesson is an OpMode, and its name in the list is the one in its `@TeleOp` or `@Autonomous` line. [L2](#)

### TeleOp and Autonomous

The two kinds of OpMode. A TeleOp reads the gamepads; an autonomous runs on its own. L9 and L10 are the two autonomous lessons. [L9](#)

### Driver Station

The phone or tablet that holds the gamepads and picks the OpMode. Lines printed with `Tracker.printToDs` show up there. [L2](#)

### Panels

The web page that draws what the code published while it ran. Every `Tracker.publish` key shows up there, and [the cheat sheet](#) lists them. [L3](#)

### Drivetrain

The four motors and the code that decides what power each one gets. Every lesson shares one class, `LessonsDriveTrain`, and adds to it. [L2](#)

### Deadband

A band of stick readings near the middle that count as let go. Without one the robot creeps when nobody is touching it. [L3](#)

### Squaring a stick

Making a small push mean a smaller power, by multiplying the stick by itself and putting the sign back. [L3](#)

### Normalizing

Scaling four wheel powers down together when the biggest one is over 1. The robot still goes where it was asked. It only goes slower. [L4](#)

### Mixing

Turning forward, sideways and turn into four wheel powers with four sums. [L5](#)

### Mecanum

A wheel with rollers set at 45 degrees on its rim, which lets the robot drive sideways. The four wheels' rollers make an X seen from above. [L5](#)

### Commanded wheels

Four powers a lesson has set itself. The path follower cannot overrule them. It gets the wheels back when the lesson hands them back. [L6](#)

### Follower

Pedro's code that drives the robot along a path, or holds it at a pose. From L6 on it holds the drivetrain and the lessons reach past it. [L6](#)

### Pose

Where the robot is and which way it faces: x, y and a heading. A pose is a place on the field, not a distance from the robot. [L8](#)

**Heading**

Which way the robot is facing, as an angle. Counter-clockwise counts up, and the code keeps it in radians even where a page says degrees. [L8](#)

**Localizer**

Whatever works out the pose. A robot can run more than one at a time. That is what L8 is for. [L8](#)

**Shadow localizer**

A second localizer that is read and logged but never steers anything. It is how two answers get compared on one run. [L8](#)

**Dead reckoning**

Working out where you are by adding up every step you took, with nothing to check it against. The drive encoders do it; the Pinpoint does it better. [L8](#)

**Pinpoint**

The odometry computer. It has its own two measuring wheels, and it works out the pose. The follower believes this one. L17 measures against it. [L8](#)

**Path**

A line or curve the follower drives along, with a heading to hold while it does. [L9](#)

**Hold**

Telling the follower to sit at one pose and stay there. If something pushes the robot, the follower puts it back. [L10](#)

**Field relative**

Sticks that mean the field. Push away from you and the robot moves away from you, whichever way its nose points. [L11](#)

**Robot relative**

Sticks that mean the robot's own directions. Better for lining up against something. [L12](#)

**Heading hold**

Keeping the nose where the driver left it, and giving it straight back the moment the turn stick moves. [L13](#)

**Instant command**

A command that does its work and finishes in the same breath, rather than running for a while. A button press is usually one. [L14](#)

**Feedforward**

A guess at the power a wanted speed needs, made before anything is measured. Gets most of the way there at once. [L16](#)

**Feedback**

A correction worked out from two speeds: the one asked for and the one measured. It cleans up what the guess got wrong. [L16](#)

**Ticks**

What an encoder counts as a motor turns. A tick is a fixed fraction of a turn, so ticks become inches once you know how many make one. [L17](#)

**Ticks per inch**

How many ticks a wheel counts for an inch of travel. Measured, not looked up. [L17](#)

**Turn radius**

How far a wheel sits from the middle of the robot, along the diagonal it pushes. It turns radians per second into inches per second at the wheel. [L17](#)

**Coast and brake**

What a motor does at zero power. Braking holds the robot still; coasting lets the wheels roll, which is what a measurement you push by hand needs. [L17](#)

## 2.23 Cheat sheet

Every table here is read straight out of the lesson code, so it says what the code says.

### 2.23.1 What to run

One OpMode name for the Driver Station, one command for the laptop. Each command runs only that lesson's own tests, which is what makes a red one easy to read. A dash means no test names that lesson: its check is on the floor.

| Lesson | On the robot                         | On the laptop                                                                           |
|--------|--------------------------------------|-----------------------------------------------------------------------------------------|
| L2a    | L2a Sticks (TeleOp)                  | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l2a*'</code>          |
| L2b    | L2b Tank (TeleOp)                    | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l2b*'</code>          |
| L3a    | L3a Deadband (TeleOp)                | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l3a*'</code>          |
| L3b    | L3b Squared (TeleOp)                 | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l3b*'</code>          |
| L4     | L4 Arcade (TeleOp)                   | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l4*'</code>           |
| L5     | L5 Holonomic (TeleOp)                | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l5*'</code>           |
| L6     | L6 Follower Wheels (TeleOp)          | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*L6FollowerDriveTrainTest*'</code> |
| L8     | L8 Compare Localizers (TeleOp)       | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l8*'</code>           |
| L9     | L9 Drive 24 (Autonomous)             | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l9*'</code>           |
| L10    | L10 Forward Then Strafe (Autonomous) | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l10*'</code>          |
| L11    | L11 Field Relative (TeleOp)          | <code>./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.l11*'</code>          |

|      |                                                     |                                                                                                                  |
|------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| L12  | L12<br>Robot<br>Relative<br>Button<br>(TeleOp)      | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.112*'                                                |
| L13  | L13<br>Heading<br>Hold<br>(TeleOp)                  | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.113*'                                                |
| L14  | L14<br>Drive<br>To<br>Pose<br>(TeleOp)              | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.114*'                                                |
| L15  | L15<br>Combined<br>(TeleOp)                         | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.115*'                                                |
| L16  | L16<br>Velocity<br>Drive<br>(TeleOp)                | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsTest.116*'                                                |
| L17a | L17a<br>Measure<br>ticks<br>per<br>inch<br>(TeleOp) | ./gradlew :TeamCode:testDebugUnitTest --tests '*LessonsDriveTrainTest.theTicksDoorReportsWhatEachEncoderCounted' |
| L17b | L17b<br>Measure<br>turn<br>radius<br>(TeleOp)       | –                                                                                                                |

### 2.23.2 Who writes what in the shared drivetrain

Every lesson adds to one class, and nothing is written twice. A method written in an early lesson is the one a later lesson calls.

| Lesson        | What it writes in LessonsDriveTrain                                                     |
|---------------|-----------------------------------------------------------------------------------------|
| L2b           | writeWheels(), driveWheelsNow()                                                         |
| L3a and L3b   | deadband(), squared(), fieldRelative()                                                  |
| L4            | normalized()                                                                            |
| L6            | mix(), setCommandedWheels(), releaseCommandedWheels(), commandedWheelsAreSet(), drive() |
| L16           | setCommandedWheelSpeeds()                                                               |
| L17a and L17b | wheelTicks()                                                                            |

### 2.23.3 What shows up in Panels

A key ending in . . . has a name added to the end of it, one per wheel or per localizer.

| Log key                              | Published by                                                 |
|--------------------------------------|--------------------------------------------------------------|
| Localizer<br>/driveWheelEncoders/... | L15CombinedOpMode, L8CompareLocalizersOpMode                 |
| command/forward                      | L4ArcadeOpMode, L5HolonomicOpMode,<br>L6WheelsFollowerOpMode |
| command/forward_ips                  | L16VelocityDriveOpMode                                       |
| command/left                         | L5HolonomicOpMode, L6WheelsFollowerOpMode                    |
| command/left_ips                     | L16VelocityDriveOpMode                                       |
| command/turn_ccw                     | L4ArcadeOpMode, L5HolonomicOpMode,<br>L6WheelsFollowerOpMode |
| command/turn_radps                   | L16VelocityDriveOpMode                                       |
| drive/aiming                         | L15CombinedOpMode                                            |
| drive/mode                           | L14DriveToPoseOpMode, L15CombinedOpMode                      |
| drive/robotRelative                  | L12RobotRelativeButtonOpMode                                 |
| drive/target_deg                     | L15CombinedOpMode                                            |
| driver pressed A                     | L2aSticksOpMode, L2bTankOpMode                               |
| heading/deg                          | L13HeadingHoldOpMode                                         |
| heading/holding                      | L13HeadingHoldOpMode                                         |
| lesson/event                         | L2aSticksOpMode, L2bTankOpMode                               |
| lesson/loop_count                    | L2aSticksOpMode                                              |
| lesson/seconds_running               | L2aSticksOpMode                                              |
| measure/inches                       | L17aMeasureTicksPerInchOpMode                                |
| measure/radians                      | L17bMeasureTurnRadiusOpMode                                  |
| measure/ticks                        | L17aMeasureTicksPerInchOpMode                                |
| measure/ticksPerInch                 | L17aMeasureTicksPerInchOpMode                                |
| measure/turnRadius_in                | L17bMeasureTurnRadiusOpMode                                  |
| measure/wheel_inches                 | L17bMeasureTurnRadiusOpMode                                  |
| stick/leftX                          | L2aSticksOpMode, L2bTankOpMode                               |
| stick/leftY                          | L2aSticksOpMode, L2bTankOpMode                               |
| stick/left_raw                       | L3aDeadbandOpMode, L3bSquaredOpMode                          |
| stick/left_shaped                    | L3aDeadbandOpMode, L3bSquaredOpMode                          |
| stick/rightX                         | L2aSticksOpMode, L2bTankOpMode                               |
| stick/rightY                         | L2aSticksOpMode, L2bTankOpMode                               |
| stick/right_raw                      | L3aDeadbandOpMode, L3bSquaredOpMode                          |
| stick/right_shaped                   | L3aDeadbandOpMode, L3bSquaredOpMode                          |
| wheel/...                            | LessonsDriveTrain                                            |

Generated by applying solutions/ on solutions-try, every lesson in order.

## 2.24 Where this came from

### 2.24.1 The guide's own terms

These pages are licensed [Creative Commons Attribution-NonCommercial 4.0 International](#), which is CC BY-NC 4.0 for short.

You may share these pages and build on them. Credit Catholic Central Spires Robotics, link the license, and do not sell what you make.

The repository's own LICENSE is FIRST's notice over the FTC SDK. It covers the code, not these

pages.

### 2.24.2 Game Manual 0

[Game Manual 0](#) is licensed [CC BY-NC 2.0 Generic](#). It is the best general FTC reference there is. This guide is not trying to replace it. These pages are about one robot and one set of lessons.

One page leans on it. [L5](#) explains why the mecanum rollers make an X, and says that GM0's mecanum page and the drivetrain's own javadoc are where that explanation comes from. No page here copies GM0's words. The guide's license is chosen to keep every restriction GM0's asks for anyway.

### 2.24.3 Pedro Pathing

[Pedro Pathing](#) is the path following library the lessons hand the drivetrain to. It is licensed BSD 3-Clause, and its name and logo are its own. This guide links Pedro's documentation rather than restating it, and the code the lessons write is ours.

### 2.24.4 The FTC SDK

The **FTC SDK** is FIRST's, under the BSD-style notice in the repository's root `LICENSE`. The Driver Station, the Control Hub and every `@TeleOp` in this guide are its.

### 2.24.5 AdvantageScope

**AdvantageScope**, from the Mechanical Advantage team, is what opens a flight log after a run.

### 2.24.6 Not affiliated with FIRST

FIRST and FIRST Tech Challenge are trademarks of For Inspiration and Recognition of Science and Technology.

This guide is written by one team for its own students. It is not produced by FIRST, endorsed by FIRST, or affiliated with FIRST.

## 2.25 Answers

What each lesson changes, file by file, out of the solutions line. Use it when you are stuck, not instead of being stuck.

### 2.25.1 L2aSticksOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying `solutions/` on `solutions-try`:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L2aSticksOpMode.java
```

#### L2a

*TODO 1*

Before L2a:

```
// TODO 1: read all four stick axes into four named doubles. Negate each  
//          y axis, so that pushing away from the driver is positive.
```

After L2a:

```
double leftSpeed = -gamepad1.left_stick_y;  
double rightSpeed = -gamepad1.right_stick_y;  
double leftSideways = gamepad1.left_stick_x;  
double rightSideways = gamepad1.right_stick_x;
```

*TODO 2*

Before L2a:

```
// TODO 2: log all four, so Panels can draw them:
//      Tracker.publish("stick/leftY", leftSpeed);
//      then stick/leftX, stick/rightY and stick/rightX.
```

After L2a:

```
Tracker.publish("stick/leftY", leftSpeed);
Tracker.publish("stick/leftX", leftSideways);
Tracker.publish("stick/rightY", rightSpeed);
Tracker.publish("stick/rightX", rightSideways);
```

*TODO 3*

Before L2a:

```
// TODO 3: read gamepad1.a into a boolean, and log it as
//      "driver pressed A". A boolean logs the same way a double does.
```

After L2a:

```
boolean buttonA = gamepad1.a;
Tracker.publish("driver pressed A", buttonA);
```

*TODO 4*

Before L2a:

```
// TODO 4: add one to loopCount, read getRuntime() into a double, and log
//      them as "lesson/loop_count" and "lesson/seconds_running".
An int
//      and a double, so three of Java's kinds of number are now
logged.
```

After L2a:

```
loopCount = loopCount + 1;
double secondsRunning = getRuntime();
Tracker.publish("lesson/loop_count", loopCount);
Tracker.publish("lesson/seconds_running", secondsRunning);
```

*TODO 5*

Before L2a:

```
// TODO 5: say when the button changes. If the boolean is not the same as
//      previousButtonA, log "button A pressed" when it is now true and
//      "button A released" when it is now false, as
//      "lesson/event". A String logs the same way. Then remember this
//      loop's value in previousButtonA for the next one.
//      Works when: LessonsTest.l2a_logsEveryStickAndTheAButton and
//      LessonsTest.l2a_saysWhenTheButtonIsPressedAndReleased pass.
```

After L2a:

```
if (buttonA != previousButtonA) {
    if (buttonA) {
        Tracker.publish("lesson/event", "button A pressed");
    } else {
        Tracker.publish("lesson/event", "button A released");
    }
}
previousButtonA = buttonA;
```

## 2.25.2 L2bTankOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L2bTankOpMode.java

### L2b

*A change*

Before L2b:

```
@TeleOp(name = "L2a Sticks", group = "Lessons")
```

After L2b:

```
@TeleOp(name = "L2b Tank", group = "Lessons")
```

*A change*

Before L2b:

```
/** How many times {@link #loop} has run. */  
private int loopCount;
```

After L2b:

```
private L2TankDriveTrain drivetrain;
```

*A change*

Before L2b:

```
(nothing)
```

After L2b:

```
drivetrain = new L2TankDriveTrain(hardware);
```

*A change*

Before L2b:

```
loopCount = 0;
```

After L2b:

```
(nothing)
```

*A change*

Before L2b:

```
double leftSideways = gamepad1.left_stick_x;  
double rightSideways = gamepad1.right_stick_x;
```

After L2b:

```
drivetrain.sticks(leftSpeed, rightSpeed);
```

*A change*

Before L2b:

```
Tracker.publish("stick/leftX", leftSideways);
```

After L2b:

```
Tracker.publish("stick/leftX", gamepad1.left_stick_x);
```

*A change*

Before L2b:

```
Tracker.publish("stick/rightX", rightSideways);
```

After L2b:

```
Tracker.publish("stick/rightX", gamepad1.right_stick_x);
```

*A change*

Before L2b:

```
loopCount = loopCount + 1;
double secondsRunning = getRuntime();
Tracker.publish("lesson/loop_count", loopCount);
Tracker.publish("lesson/seconds_running", secondsRunning);
```

After L2b:

```
(nothing)
```

*A change*

Before L2b:

```
(nothing)
```

After L2b:

```
drivetrain.stop();
```

It also changes 4 run(s) of comment lines, which are not shown.

### 2.25.3 L2TankDriveTrain

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/L2TankDriveTrain.java
```

**L2b**

*TODO (L2b)*

Before L2b:

```
// TODO (L2b): call driveWheelsNow with four powers. The left stick runs
both
//      left wheels and the right stick runs both right wheels, so two of
//      the four are the same number.
```

After L2b:

```
driveWheelsNow(leftSpeed, rightSpeed, leftSpeed, rightSpeed);
```

### 2.25.4 LessonsDriveTrain

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/LessonsDriveTrain.java
```

### L2b

#### TODO 1 (L2b)

Before L2b:

```
// TODO 1 (L2b): send each slot of wheelPowers to the motor it belongs to,
// naming the slot with FL, FR, BL or BR. One line each:
// motors[FL].setPower(wheelPowers[FL]); and the other three.
// Works when: LessonsTest.l2b_theSticksDriveTheWheelsLikeATank
// passes. Nothing drives at all until this is written.
```

After L2b:

```
motors[FL].setPower(wheelPowers[FL]);
motors[FR].setPower(wheelPowers[FR]);
motors[BL].setPower(wheelPowers[BL]);
motors[BR].setPower(wheelPowers[BR]);
```

#### TODO 2 (L2b)

Before L2b:

```
// TODO 2 (L2b): put each power into its own slot of wheelPowers, using
// FL, FR, BL and BR to say which slot is which. One line each.
// wheelPowers[FL] = frontLeftPower; and so on.
// Then call normalized(wheelPowers), which is L4's job and does
// nothing yet, and writeWheels(), which you wrote just above.
// Works when: the wheels turn the way the sticks say, and
// LessonsTest.l2b_theSticksDriveTheWheelsLikeATank passes.
```

After L2b:

```
wheelPowers[FL] = frontLeftPower;
wheelPowers[FR] = frontRightPower;
wheelPowers[BL] = backLeftPower;
wheelPowers[BR] = backRightPower;

normalized(wheelPowers);
writeWheels();
```

### L3a

#### TODO 7 (L3a)

Before L3a:

```
// TODO 7 (L3a): if the value is smaller than the band, ignoring its minus
// sign, the answer is 0. Otherwise the answer is the value
itself.
// An if and an else, and Math.abs takes the minus sign off.
// Works when: LessonsTest.l3a_aNearlyCentredStickCountsAsCentred
// passes, and the robot sits still with the sticks let go.
return value;
```

After L3a:

```
if (Math.abs(value) < band) {
    return 0.0;
} else {
    return value;
}
```

**L3b***TODO 8 (L3b)*

Before L3b:

```

// TODO 8 (L3b): the value times itself, with the sign it started with.
//      Squaring a negative number the ordinary way loses the minus
//      sign, which would drive the robot forwards when the driver
asked
//      for backwards, so put it back.
//      Works when: LessonsTest.l3b_halfAStickIsAQuarterOfThePower
//      passes.
return value;

```

After L3b:

```

double magnitude = value * value;
if (value < 0.0) {
    return -magnitude;
} else {
    return magnitude;
}

```

**L4***TODO 3 (L4)*

Before L4:

```

// TODO 3 (L4): find the biggest of the four, ignoring minus signs, or 1
if
//      none of them reaches 1. Math.abs takes the minus sign off, and
//      Math.max picks the bigger of two: give the magnitude its own
//      variable. Put 1 divided by that number into powerScale, so the
//      log says what the scaling was. Then divide every power by it,
in
//      a second loop with i as the index, and hand the array back.
//      Handing them back untouched is what happens now, which is why
//      L2b drives fine and L4 pulls to one side at full turn.
//      Works when:
LessonsTest.l4_arcadeUsesOneStickToDriveAndOneToTurn
//      passes, and the robot drives straight with the drive stick and
//      the turn stick both all the way forward.

```

After L4:

```

double max = 1.0;
for (double power : powers) {
    double magnitude = Math.abs(power);
    max = Math.max(max, magnitude);
}

powerScale = 1.0 / max;
for (int i = 0; i < powers.length; i++) {
    powers[i] = powers[i] / max;
}

```

**L6***TODO 4 (L6)*

Before L6:

```

// TODO 4 (L6): make a new four-slot array in commandedWheels and put each
//      power in its own slot, the same way driveWheelsNow does.

```

After L6:

```
commandedWheels = new double[4];
commandedWheels[FL] = frontLeftPower;
commandedWheels[FR] = frontRightPower;
commandedWheels[BL] = backLeftPower;
commandedWheels[BR] = backRightPower;
```

*TODO 5 (L6)*

Before L6:

```
mix(). // TODO 5 (L6): forget the four powers, so drive() goes back to asking
//      Setting commandedWheels to null is how a field says "nothing
//      here".
```

After L6:

```
commandedWheels = null;
```

*TODO 6 (L6)*

Before L6:

```
on. // TODO 6 (L6): work out where the four powers come from, and send them
//      Declare a double[] sourcePowers with nothing in it. If a lesson
//      has not commanded the wheels, ask mix() for them into its own
//      variable, hand that to normalized(), and point sourcePowers at
//      what comes back. If a lesson has commanded them, point
//      sourcePowers at commandedWheels instead. An if and an else, not
//      a ?. Then copy every slot of sourcePowers into wheelPowers in a
//      loop with i as the index, and call writeWheels().
//      Works when: L6FollowerDriveTrainTest passes, the robot drives
on //      the sticks in L6, and L9 drives its 24 inches.
```

After L6:

```
double[] sourcePowers;
if (commandedWheels == null) {
    double[] mixed = mix(powers);
    sourcePowers = normalized(mixed);
} else {
    sourcePowers = commandedWheels;
}
for (int i = 0; i < wheelPowers.length; i++) {
    wheelPowers[i] = sourcePowers[i];
}

writeWheels();
```

**L11**

*TODO 9 (L11)*

Before L11:

```
// TODO 9 (L11): turn the driver's two field speeds into the robot's own
//      forward and strafe, by rotating them backwards through the
//      heading. Take the cosine and the sine of headingRad into their
//      own variables, then
//      forwardSpeed = fieldXSpeed * cos + fieldYSpeed * sin
//      strafeLeftSpeed = -fieldXSpeed * sin + fieldYSpeed * cos
//      Then mix those three, normalize what comes back, and command
```

```

the      //      four wheels, which is the same three lines L6's sticks() ends
on.      //      Works when:
          //      LessonsTest.l11_fieldRelativeIgnoresWhichWayTheRobotFaces
passes,  //      and pushing the stick away from the driver moves the robot away
          //      from the driver whichever way it is facing.

```

After L11:

```

double cos = Math.cos(headingRad);
double sin = Math.sin(headingRad);
double forwardSpeed = fieldXSpeed * cos + fieldYSpeed * sin;
double strafeLeftSpeed = -fieldXSpeed * sin + fieldYSpeed * cos;

double[] mixed = mix(new DrivePowers(forwardSpeed, strafeLeftSpeed,
turnCcwSpeed));
double[] scaled = normalized(mixed);
setCommandedWheels(scaled[FL], scaled[FR], scaled[BL], scaled[BR]);

```

## L16

### TODO 10 (L16)

Before L16:

```

// TODO 10 (L16): put the four speeds asked for into a four-slot array, in
//      wheel order, and ask measuredSpeeds.all() for the four speeds
the    //
        //      encoders see. Then work out a power for each wheel, in a loop
        //      with i as the index:
        //      feedforward = Constants.powerPerInchPerSecond * wanted[i]
        //      feedback    = VELOCITY_KP * (wanted[i] - measured[i])
        //      and put clampToPower(feedforward + feedback) in that wheel's
        //      slot. Command the four wheels with them, then call
        //      publishWheelSpeeds(wanted, measured, powers) so the log shows
        //      what was asked for, what happened, and the power in between.
        //      Works when:
        //
LessonsTest.l16_theSticksCommandASpeedAndTheWheelsAreCorrectedTowardsIt
//      passes.

```

After L16:

```

double[] wanted = new double[4];
wanted[FL] = frontLeftInPerS;
wanted[FR] = frontRightInPerS;
wanted[BL] = backLeftInPerS;
wanted[BR] = backRightInPerS;

double[] measured = measuredSpeeds.all();
double[] powers = new double[4];
for (int i = 0; i < powers.length; i++) {
    double feedforward = Constants.powerPerInchPerSecond * wanted[i];
    double feedback = VELOCITY_KP * (wanted[i] - measured[i]);
    powers[i] = clampToPower(feedforward + feedback);
}

setCommandedWheels(powers[FL], powers[FR], powers[BL], powers[BR]);
publishWheelSpeeds(wanted, measured, powers);

```

**L17a***TODO 11 (L17a)*

Before L17a:

```
// TODO 11 (L17a): fill each slot with what that wheel's encoder has
// counted, naming the slot with FL, FR, BL or BR and the motor
// through hardware:
// ticks[FL] = hardware.frontLeft.getCurrentPosition(); and three
// more. Four zeros is what comes back now, so a measurement says
// the robot never moved.
// Works when:
// LessonsDriveTrainTest.theTicksDoorReportsWhatEachEncoderCounted
// passes.
```

After L17a:

```
ticks[FL] = hardware.frontLeft.getCurrentPosition();
ticks[FR] = hardware.frontRight.getCurrentPosition();
ticks[BL] = hardware.backLeft.getCurrentPosition();
ticks[BR] = hardware.backRight.getCurrentPosition();
```

**2.25.5 L3aDeadbandOpMode**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L3aDeadbandOpMode.java

**L3a***TODO 1 (L3a)*

Before L3a:

```
// TODO 1 (L3a): pick the number. A stick let go reads a few hundredths, so
// 0.05 is a good first guess; 0 leaves the creep exactly where it
// was.
private static final double DEADBAND = 0;
```

After L3a:

```
private static final double DEADBAND = 0.05;
```

*TODO 2 (L3a)*

Before L3a:

```
// TODO 2 (L3a): put each raw stick through drivetrain.deadband(), with
// DEADBAND as the band both times, and hand the two to
// drivetrain.sticks().
double leftSpeed = 0;
double rightSpeed = 0;
```

After L3a:

```
double leftSpeed = drivetrain.deadband(leftRawSpeed, DEADBAND);
double rightSpeed = drivetrain.deadband(rightRawSpeed, DEADBAND);
drivetrain.sticks(leftSpeed, rightSpeed);
```

**2.25.6 L3bSquaredOpMode**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L3bSquaredOpMode.java

### L3b

*A change*

Before L3b:

```
@TeleOp(name = "L3a Deadband", group = "Lessons")
```

After L3b:

```
@TeleOp(name = "L3b Squared", group = "Lessons")
```

*A change*

Before L3b:

```
double leftSpeed = drivetrain.deadband(leftRawSpeed, DEADBAND);
double rightSpeed = drivetrain.deadband(rightRawSpeed, DEADBAND);
```

After L3b:

```
double leftDeadbanded = drivetrain.deadband(leftRawSpeed, DEADBAND);
double rightDeadbanded = drivetrain.deadband(rightRawSpeed, DEADBAND);

double leftSpeed = drivetrain.squared(leftDeadbanded);
double rightSpeed = drivetrain.squared(rightDeadbanded);
```

It also changes 4 run(s) of comment lines, which are not shown.

## 2.25.7 L4ArcadeOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L4ArcadeOpMode.java

### L4

*TODO*

Before L4:

```
// TODO: read the two sticks and hand them to
drivetrain.sticks(forwardSpeed, turnCcwSpeed).
//      forward comes from the left stick's y axis, turn from the right
//      stick's x axis, and BOTH need a minus sign -- the note above says
//      why the turn one does. Log them as command/forward and
//      command/turn_ccw so Panels can draw them.
double forwardSpeed = 0;
double turnCcwSpeed = 0;
```

After L4:

```
double forwardSpeed = -gamepad1.left_stick_y;
double turnCcwSpeed = -gamepad1.right_stick_x;
```

*A change*

Before L4:

```
(nothing)
```

After L4:

```
Tracker.publish("command/forward", forwardSpeed);
Tracker.publish("command/turn_ccw", turnCcwSpeed);
```

It also changes 1 run(s) of comment lines, which are not shown.

## 2.25.8 L4ArcadeDriveTrain

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/L4ArcadeDriveTrain.java
```

### L4

*A change*

Before L4:

```
public void sticks(double leftSpeed, double rightSpeed) {
```

After L4:

```
public void sticks(double forwardSpeed, double turnCcwSpeed) {
    double leftSpeed = forwardSpeed - turnCcwSpeed;
    double rightSpeed = forwardSpeed + turnCcwSpeed;
```

It also changes 5 run(s) of comment lines, which are not shown.

## 2.25.9 L5HolonomicOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/L5HolonomicOpMode.java
```

### L5

*A change*

Before L5:

```
@TeleOp(name = "L4 Arcade", group = "Lessons")
```

After L5:

```
@TeleOp(name = "L5 Holonomic", group = "Lessons")
```

*A change*

Before L5:

```
private L4ArcadeDriveTrain drivetrain;
```

After L5:

```
private L5HolonomicDriveTrain drivetrain;
```

*A change*

Before L5:

```
drivetrain = new L4ArcadeDriveTrain(hardware);
```

After L5:

```
drivetrain = new L5HolonomicDriveTrain(hardware);
```

*A change*

Before L5:

(nothing)

After L5:

```
double strafeLeftSpeed = -gamepad1.left_stick_x;
```

*A change*

Before L5:

```
drivetrain.sticks(forwardSpeed, turnCcwSpeed);
```

After L5:

```
drivetrain.sticks(forwardSpeed, strafeLeftSpeed, turnCcwSpeed);
```

*A change*

Before L5:

(nothing)

After L5:

```
Tracker.publish("command/left", strafeLeftSpeed);
```

It also changes 3 run(s) of comment lines, which are not shown.

**2.25.10 L5HolonomicDriveTrain**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/L5HolonomicDriveTrain.java
```

**L5***A change*

Before L5:

```
public void sticks(double forwardSpeed, double turnCcwSpeed) {
    double leftSpeed = forwardSpeed - turnCcwSpeed;
    double rightSpeed = forwardSpeed + turnCcwSpeed;
    driveWheelsNow(leftSpeed, rightSpeed, leftSpeed, rightSpeed);
}
```

After L5:

```
public void sticks(double forwardSpeed, double strafeLeftSpeed, double
turnCcwSpeed) {
    double frontLeftPower = forwardSpeed - strafeLeftSpeed - turnCcwSpeed;
    double frontRightPower = forwardSpeed + strafeLeftSpeed + turnCcwSpeed;
    double backLeftPower = forwardSpeed + strafeLeftSpeed - turnCcwSpeed;
    double backRightPower = forwardSpeed - strafeLeftSpeed + turnCcwSpeed;
    driveWheelsNow(frontLeftPower, frontRightPower, backLeftPower,
backRightPower);
}
```

It also changes 6 run(s) of comment lines, which are not shown.

**2.25.11 L6WheelsFollowerOpMode**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L6WheelsFollowerOpMode.java

**L6**

*A change*

Before L6:

```
@TeleOp(name = "L5 Holonomic", group = "Lessons")
```

After L6:

```
@TeleOp(name = "L6 Follower Wheels", group = "Lessons")
```

*A change*

Before L6:

```
private L5HolonomicDriveTrain drivetrain;
```

After L6:

```
private L6FollowerDriveTrain drivetrain;
```

*A change*

Before L6:

```
drivetrain = new L5HolonomicDriveTrain(hardware);  
initAfter();
```

After L6:

```
drivetrain = new L6FollowerDriveTrain(hardware);  
initAfter(drivetrain);
```

It also changes 3 run(s) of comment lines, which are not shown.

## 2.25.12 L6FollowerDriveTrain

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L6FollowerDriveTrain.java

**L6**

*TODO 1*

Before L6:

```
// TODO 1: wrap the three numbers in a new DrivePowers(...) in its own  
//         variable, hand that to mix() into a second variable, hand that  
//         to normalized() into a third, and pass the four slots to  
//         setCommandedWheels(). One call to a line; nothing nested.  
//         Going through mix() is the point: the sticks and the follower  
//         then agree about which wheel does what.
```

After L6:

```
DrivePowers drivePowers = new DrivePowers(forwardSpeed, strafeLeftSpeed,  
turnCcwSpeed);  
double[] mixed = mix(drivePowers);  
double[] scaled = normalized(mixed);  
setCommandedWheels(scaled[FL], scaled[FR], scaled[BL], scaled[BR]);
```

**TODO 2**

Before L6:

```
// TODO 2: fill each slot in, one line each, naming it with FL, FR, BL or
//          BR. The four sums are the same ones L5HolonomicDriveTrain uses.
```

After L6:

```
wheels[FL] = forwardSpeed - strafeLeftSpeed - turnCcwSpeed;
wheels[FR] = forwardSpeed + strafeLeftSpeed + turnCcwSpeed;
wheels[BL] = forwardSpeed + strafeLeftSpeed - turnCcwSpeed;
wheels[BR] = forwardSpeed - strafeLeftSpeed + turnCcwSpeed;
```

**2.25.13 L8CompareLocalizersOpMode**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L8CompareLocalizersOpMode.java

**L8****TODO 1 (L8)**

Before L8:

```
// TODO 1 (L8): run your localizer alongside the robot's own, named
//          "driveWheelEncoders" because that is which encoders it reads:
//          shadowLocalizers.add("driveWheelEncoders",
//          new MecanumEncoderLocalizer(new
//          HardwareWheelSource(hardware)));
```

After L8:

```
shadowLocalizers.add("driveWheelEncoders",
    new MecanumEncoderLocalizer(new HardwareWheelSource(hardware)));
```

**TODO 2 (L8)**

Before L8:

```
// TODO 2 (L8): holonomic driving, same as L5 -- but through
the drivetrain
//          the follower is holding now, the way L6 handed it over. One
call
//          to drivetrain.sticks() with the three sticks.
```

After L8:

```
drivetrain.sticks(
    -gamepad1.left_stick_y, -gamepad1.left_stick_x,
    -gamepad1.right_stick_x);
```

It also changes 1 run(s) of comment lines, which are not shown.

**2.25.14 MecanumEncoderLocalizer**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/MecanumEncoderLocalizer.java

## L8

*A change*

Before L8:

```
(nothing)
```

After L8:

```
import com.pedropathing.math.Twist;
import com.pedropathing.api.PoseFactory;
```

*A change*

Before L8:

```
(nothing)
```

After L8:

```
import org.firstinspires.ftc.teamcode.base.HeadingHold;
```

*A change*

Before L8:

```
(nothing)
```

After L8:

```
private static final PoseFactory POSES = PoseFactory.radians();
```

*A change*

Before L8:

```
(nothing)
```

After L8:

```
private double x;
private double y;
private double headingOffset;
private double[] lastWheels;
private double lastRawHeading;
private long lastNanos;
private MotionState state = MotionState.zero();
```

*TODO 1 (L8)*

Before L8:

```
// TODO 1 (L8): read source.wheelInches() and source.headingRadians().
// TODO 2 (L8): on the first loop there is nothing to compare against --
//             remember the values and return.
// TODO 3 (L8): work out how far each wheel moved since last time, then
//             forward = (fl + fr + bl + br) / 4
//             left    = (-fl + fr + bl - br) / 4
//             (Check that second one against a robot: to strafe LEFT, the
//             front-left wheel rolls backwards.)
// TODO 4 (L8): rotate (forward, left) by the heading and add it to x and
y:
//             x += forward * cos(h) - left * sin(h)
//             y += forward * sin(h) + left * cos(h)
// TODO 5 (L8): publish the new pose with MotionState.ofTwist(...).
```

After L8:

```

double[] wheels = source.wheelInches();
double rawHeading = source.headingRadians();
long now = clockNanos.getAsLong();

if (lastWheels == null) {                                     // first loop: nothing to
compare with
    lastWheels = wheels.clone();
    lastRawHeading = rawHeading;
    lastNanos = now;
    publish(0, 0, 0);
    return;
}

double dFrontLeft = wheels[0] - lastWheels[0];
double dFrontRight = wheels[1] - lastWheels[1];
double dBackLeft = wheels[2] - lastWheels[2];
double dBackRight = wheels[3] - lastWheels[3];

double forwardIn = (dFrontLeft + dFrontRight + dBackLeft + dBackRight) /
4;
double strafeLeftIn = (-dFrontLeft + dFrontRight + dBackLeft - dBackRight)
/ 4;

double dHeading = HeadingHold.wrap(rawHeading - lastRawHeading);
double midHeading = lastRawHeading + headingOffset + dHeading / 2; //
halfway through the step

x += forwardIn * Math.cos(midHeading) - strafeLeftIn *
Math.sin(midHeading);
y += forwardIn * Math.sin(midHeading) + strafeLeftIn *
Math.cos(midHeading);

double seconds = (now - lastNanos) / 1e9;
lastWheels = wheels.clone();
lastRawHeading = rawHeading;
lastNanos = now;

if (seconds > 0) {
    publish(forwardIn / seconds, strafeLeftIn / seconds, dHeading /
seconds);
} else {
    publish(0, 0, 0);
}

private void publish(double forwardSpeedInPerS, double strafeLeftSpeedInPerS,
double turnCcwSpeedRadPerS) {
    state = MotionState.ofTwist(
        POSES.of(x, y, lastRawHeading + headingOffset),
        new Twist(forwardSpeedInPerS, strafeLeftSpeedInPerS,
turnCcwSpeedRadPerS));
}

```

TODO 6 (L8)

Before L8:

```

// TODO 6 (L8): start counting from this pose. The IMU's heading can't be
// moved, so remember the DIFFERENCE and add it from now on.

```

After L8:

```

x = pose.x();
y = pose.y();
headingOffset = pose.heading() - lastRawHeading;

```

```
publish(0, 0, 0);
```

TODO 5 (L8)

Before L8:

```
return MotionState.zero(); // TODO 5 (L8): return the pose you worked  
out
```

After L8:

```
return state;
```

TODO 7 (L8)

Before L8:

```
// TODO 7 (L8): back to zero.
```

After L8:

```
x = 0;  
y = 0;  
headingOffset = -lastRawHeading;  
lastWheels = null;  
publish(0, 0, 0);
```

It also changes 2 run(s) of comment lines, which are not shown.

## 2.25.15 L9Drive24OpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L9Drive24OpMode.java
```

L9

TODO

Before L9:

```
// TODO: return a sequence with one step: follow a straight line from  
// start to end, holding the heading it starts at.  
// return sequential(follow(follower, line(start,  
end).constant(start)));  
// .constant(pose) takes the heading from a pose. The number form,  
// .constant(0), is radians, so .constant(90) is not 90 degrees.  
// Use .constant(), not .linear() -- Pedro 3.0.1 issues #176 and  
#181.  
return Command.NOOP;
```

After L9:

```
return sequential(  
    follow(follower, line(start, end).constant(start))  
);
```

## 2.25.16 L10ForwardThenStrafeOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L10ForwardThenStrafeOpMode.java
```

**L10***A change*

Before L10:

```
(nothing)
```

After L10:

```
import static com.pedropathing.ivy.pedro.PedroCommands.hold;
```

*A change*

Before L10:

```
@Autonomous(name = "L9 Drive 24", group = "Lessons")
```

After L10:

```
@Autonomous(name = "L10 Forward Then Strafe", group = "Lessons")
```

*A change*

Before L10:

```
private final Pose end = POSES.of(72, botStartYIn+24, 90);
```

After L10:

```
private final Pose corner = POSES.of(72, 72, 90);
private final Pose end = POSES.of(96, 72, 90);
```

*A change*

Before L10:

```
follow(follower, line(start, end).constant(start))
```

After L10:

```
follow(follower, line(start, corner).constant(start)),
follow(follower, line(corner, end).constant(start)),
hold(follower, end)
```

It also changes 2 run(s) of comment lines, which are not shown.

**2.25.17 L11FieldRelativeOpMode**

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

```
TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry
/L11FieldRelativeOpMode.java
```

**L11***TODO (L11)*

Before L11:

```
// TODO (L11): the same three sticks as L5, but through
// drivetrain.fieldRelative(...) instead of drivetrain.sticks(...).
// The heading goes in first, and the drivetrain cannot get it
// itself: follower.pose().heading() is where it comes from.
// Try L5's version with the robot turned 180 degrees first, so you
// can feel the difference.
```

After L11:

```
drivetrain.fieldRelative(follower.pose().heading(),
    -gamepad1.left_stick_y,      // away from the driver
    -gamepad1.left_stick_x,      // to the driver's left
    -gamepad1.right_stick_x);    // counter-clockwise
```

It also changes 1 run(s) of comment lines, which are not shown.

## 2.25.18 L12RobotRelativeButtonOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L12RobotRelativeButtonOpMode.java

### L12

*TODO (L12)*

Before L12:

```
// TODO (L12): if gamepad1.right_bumper is held, drive robot relative
//             (drivetrain.sticks); otherwise field relative
//             (drivetrain.fieldRelative, with follower.pose().heading() first).
//             Held, not toggled -- ask a driver why.
//             Log it too: Tracker.publish("drive/robotRelative",
gamepad1.right_bumper);
```

After L12:

```
if (gamepad1.right_bumper) {
    drivetrain.sticks(forwardSpeed, strafeLeftSpeed, turnCcwSpeed);
} else {
    drivetrain.fieldRelative(follower.pose().heading(),
        forwardSpeed, strafeLeftSpeed, turnCcwSpeed);
}
Tracker.publish("drive/robotRelative", gamepad1.right_bumper);
```

It also changes 1 run(s) of comment lines, which are not shown.

## 2.25.19 L13HeadingHoldOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L13HeadingHoldOpMode.java

### L13

*TODO 1*

Before L13:

```
// TODO 1: build a HeadingHold from the tuned controller:
//             new
HeadingHold(Constants.foresightConfig.headingFeedback.get())
```

After L13:

```
heading = new
HeadingHold(Constants.foresightConfig.headingFeedback.get());
```

*TODO 2 (L13)*

Before L13:

```

the      // TODO 2 (L13): ask heading.turn(follower, -gamepad1.right_stick_x) for
        //
        //      turn power, then drive field relative with it through
        //      drivetrain.fieldRelative(...). Log heading/holding and
        //      heading/deg so Panels can show the hold.

```

After L13:

```

double turnCcwSpeed = heading.turn(follower, -gamepad1.right_stick_x);
drivetrain.fieldRelative(follower.pose().heading(),
    -gamepad1.left_stick_y, -gamepad1.left_stick_x, turnCcwSpeed);
Tracker.publish("heading/holding", heading.target() != null);
Tracker.publish("heading/deg", Math.toDegrees(follower.pose().heading()));

```

It also changes 1 run(s) of comment lines, which are not shown.

## 2.25.20 L14DriveToPoseOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L14DriveToPoseOpMode.java

### L14

#### TODO 1

Before L14:

```

// TODO 1: when gamepad1.y is pressed, run an instant command that
//      calls follower.hold(TARGET) and sets drivingItself = true.
//      (PedroCommands.hold() is instant: it sets the mode and ends.
//      The follower stays in HOLD until something calls manual().)

```

After L14:

```

// PedroCommands.hold() is an INSTANT command: it tells the follower to
// hold the pose and finishes straight away. The follower stays in HOLD
// until something calls manual() -- so we track that ourselves.
buttons.whenPressed() -> gamepad1.y, Commands.instant(() -> {
    follower.hold(TARGET);
    drivingItself = true;
}));

```

#### TODO 2 (L14)

Before L14:

```

// TODO 2 (L14): if drivingItself and the sticks are near zero, publish
//      Tracker.publish("drive/mode", "AUTO"), hand the wheels back
with
//      drivetrain.releaseCommandedWheels(), and return -- the follower
//      is holding the pose and must not be argued with.
// TODO 3 (L14): if the driver DOES move a stick, set drivingItself =
false,
//      call follower.manual(0, 0, 0) to leave HOLD -- only manual()
can,
//      and the three zeros are never used -- then publish
//      Tracker.publish("drive/mode", "DRIVER") and drive field
relative.

```

After L14:

```

if (drivingItself) {

```

```

    if (!driverWantsControl) {
        Tracker.publish("drive/mode", "AUTO");
        drivetrain.releaseCommandedWheels();
        return; // leave the follower holding
    }
    drivingItself = false; // the driver takes over
    // Out of HOLD, which only manual() can do. The three zeros are
    // never used -- the sticks command the wheels on the next line.
    follower.manual(0, 0, 0);
}
Tracker.publish("drive/mode", "DRIVER");
drivetrain.fieldRelative(follower.pose().heading(),
    forwardSpeed, strafeLeftSpeed, turnCcwSpeed);

```

It also changes 1 run(s) of comment lines, which are not shown.

### 2.25.21 L15CombinedOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L15CombinedOpMode.java

#### L15

##### TODO 1 (L15)

Before L15:

```

// TODO 1 (L15): add your encoder localizer as "driveWheelEncoders", as in
// L8, so the two localizers plot under the same names as they did
// there.

```

After L15:

```

shadowLocalizers.add("driveWheelEncoders",
    new MecanumEncoderLocalizer(new HardwareWheelSource(hardware)));

```

It also changes 1 run(s) of comment lines, which are not shown.

### 2.25.22 L16VelocityDriveOpMode

What each lesson's patch does to this file, in the order the lessons come, from applying solutions/ on solutions-try:

TeamCode/src/main/java/org/firstinspires/ftc/teamcode/mytry  
/L16VelocityDriveOpMode.java

#### L16

##### TODO 1 (L16)

Before L16:

```

// TODO 1 (L16): read the sticks as a SPEED, not a power. Full forward
asks // for MAX_IPS inches per second; full turn, MAX_TURN_RADPS
radians // per second. Use drivetrain.deadband(value, 0.05) as usual, and
// remember the minus signs from L5.
double forwardSpeedInPerS = 0;
double strafeLeftSpeedInPerS = 0;
double turnCcwSpeedRadPerS = 0;

```

After L16:

```

// 1. The sticks ask for a speed, not a power.
double forwardSpeedInPerS = drivetrain.deadband(-gamepad1.left_stick_y,
0.05) * MAX_IPS;
double strafeLeftSpeedInPerS = drivetrain.deadband(-gamepad1.left_stick_x,
0.05) * MAX_IPS;
double turnCcwSpeedRadPerS =
    drivetrain.deadband(-gamepad1.right_stick_x, 0.05) *
MAX_TURN_RADPS;

```

*TODO 2 (L16)*

Before L16:

```

// TODO 2 (L16): work out how fast each wheel has to travel for the robot
to
//          move like that. WheelTargets.forMecanum(forward, strafeLeft,
//          turnCcw, radius) does the arithmetic, and the radius is
//          Constants.turnRadiusInches.
double[] target = new double[4];

```

After L16:

```

// 2. What each wheel must do for the robot to move like that.
double[] target = WheelTargets.forMecanum(forwardSpeedInPerS,
strafeLeftSpeedInPerS,
    turnCcwSpeedRadPerS, Constants.turnRadiusInches);

```

*TODO 3 (L16)*

Before L16:

```

// TODO 3 (L16): ask the drivetrain for those four speeds, with
//          drivetrain.setCommandedWheelSpeeds(...). It measures, guesses
and
//          corrects, and publishes wheel/... for every wheel.

```

After L16:

```

// 3. Ask the drivetrain for those speeds. It measures, guesses and
corrects,
// and publishes wheel/... for every wheel.
drivetrain.setCommandedWheelSpeeds(target[0], target[1], target[2],
target[3]);

```

It also changes 1 run(s) of comment lines, which are not shown.





